ACCIDENTAL CANCELLATIONS IN CHOLESKY FACTORIZATION

Nam Trang
with Muse Spark via meta.ai

Reviewed by
Paul Bendich and Patrick Tam

Abstract. Sparse Cholesky factorization allocates storage from the symbolic fill pattern G^+ alone, assuming that no position predicted nonzero ever cancels. Over-allocation never threatens correctness, so the assumption is safe in practice, but it leaves open an existential question, recorded by the first author in unpublished notes circulated in 2021 as two conjectures. Call an ordering σ clean for an SPD matrix A if every position predicted nonzero in G^+_σ is nonzero in the factor L. Conjecture 1: every irreducible SPD matrix admits a clean ordering. Conjecture 2: every irreducible SPD matrix admits an ordering whose first Schur complement has no cancellation at a predicted-nonzero position.

We refute both. Conjecture 1 fails at every even order $n \geq 6$, via an explicit family M_{2m} on the complete graph. The family is not exhibited but derived: classifying the zero-configurations available at a level with four surviving vertices forces a fixed-point-free involution — hence even order — forces the magnitudes AB within pairs and blocks $[[A, B], [B, A]]$ across them, and forces the parameters onto the measure-zero curve $A^2 + (1 - B^2)A - 1 = 0$. Conversely every point of that curve with $0 < B < 1$ yields a counterexample at every even order $n \geq 6$, so the construction is a one-parameter family rather than a single matrix. Conjecture 2 already fails at order 4, via A^* with spectrum $\{9, 3, 3, 1\}$: none of its 24 orderings has a cancellation-free first Schur complement, yet 8 of them give a clean factor — the refill phenomenon that separates the two conjectures, and the trap that invalidates any argument conflating Schur-complement zeros with factor zeros.

Positively, Conjecture 1 holds for every irreducible SPD matrix of order at most 4, by an elementary argument; order five we leave open, and we settle no further order exhaustively. At order six the complete graph K_6 fails, by the family above. Forests are clean in the strong form that every ordering is clean — recovering, in factorization language, a covariance-faithfulness theorem of Malouche and Rajaratnam — and so are 2-path strips and dominating-vertex fans, chordal graphs with exactly one triangle, and chordal graphs of clique number at most 3 with pairwise edge-disjoint triangles; on the last four classes the clean ordering must adapt to the matrix, since no fixed ordering suffices. Odd orders $n \geq 7$ remain open, and we explain why the pairing argument that settles the even orders does not extend to them.

1. Introduction

Human

1.1. Sparse Cholesky and the price of fill. Let A be a real symmetric positive definite matrix of order n. Its Cholesky factorization $A = LL^\top$ is the workhorse of direct methods for the linear systems arising in finite-element analysis, interior-point methods, Gaussian process regression, and maximum-likelihood estimation in graphical models. When A is sparse, the cost of the factorization is governed not by n but by the number of nonzeros in L, and that number depends on the order in which variables are eliminated. Eliminating a variable makes its surviving neighbors mutually adjacent, so positions that were structurally zero in A may become nonzero in L. These are the fill

2020 *Mathematics Subject Classification.* 65F50; 15A23, 05C50, 15B48, 05C75, 62H22.

Human

entries, and controlling them is the central problem of sparse direct methods.

The combinatorial description is due to Rose, Tarjan and Lueker [RTL76]. Represent the sparsity of A by its pattern graph G_A, with an edge $\{i, j\}$ whenever $A_{ij} \neq 0$. For an ordering (or permutation) σ of $\{0, 1, \ldots, n-1\}$, play the elimination game: repeatedly delete the next vertex (in the ordering given by σ) and complete its neighborhood into a clique. The resulting graph G_σ^+ contains every position that can be nonzero in L, and the fill-path theorem characterizes it exactly: $\{j, k\}$ is an edge of G_σ^+ precisely when G_A contains a j–k path all of whose interior vertices precede both endpoints. Choosing σ to minimize fill is NP-hard [Yan81], which is why practical codes rely on heuristics and on the elimination-tree machinery surveyed by Heath, Ng and Peyton [HNP91] and Davis [Dav06].

1.2. The no-cancellation assumption. Every sparse Cholesky implementation performs a symbolic factorization before touching a single floating-point number: it computes G_σ^+ from G_A alone and allocates exactly that much storage. This step is purely graph-theoretic, and it is where an assumption enters. The code takes for granted that a position predicted nonzero is nonzero – that once an entry fills in, it never cancels back to zero. Demmel's lecture notes state it without apology: "assume that there is no cancellation, i.e. that once an entry of A gets filled in, it never cancels out later (which would be highly unlikely anyway)" [Dem24].

The assumption is harmless for its intended purpose, and it is worth being precise about why. Symbolic factorization produces an upper bound on the structure of L. If a predicted nonzero happens to cancel, the code has reserved a slot it does not need: it over-allocates slightly and computes the correct answer. Cancellation is a performance question, never a correctness question. Nor is the parenthetical wrong: a cancellation requires an exact algebraic coincidence among the entries of A, and such coincidences form a set of Lebesgue measure zero.

So the practice is sound and the heuristic is true. But both are statements about a fixed ordering and about typical matrices, and neither answers the question a mathematician would ask next.

In sum, the results in this paper do not contradict expert expectations, they live on its complement. Experts expected the generic statement, and it holds; the conjectures are existential and quantify over all SPD matrices, including the measure-zero locus where the counterexamples lie.

1.3. The existential question. Fix a matrix, not an ordering, and ask whether some ordering is free of cancellation. Call σ clean for A if for every edge $\{j, k\} \in G_\sigma^+$, letting S be the set of vertices preceding $\min(\sigma^{-1}(j), \sigma^{-1}(k))$ in σ, we have $(A/A_S)_{jk} \neq 0$ — i.e. the entry is nonzero at the stage where the earlier of j, k is pivoted, so the predicted nonzero is realized in L. This is the factor definition, not the Schur-complement definition requiring $(A/A_S)_{jk} \neq 0$ for every stage S whenever $\{j, k\} \in G_S$; the latter conflates Schur zeros with factor cancellations and would give A^* no clean ordering at all, unlike the eight of Theorem 5.1, because it ignores the refill phenomenon. Two conjectures, recorded by Trang in unpublished notes in 2021, ask:

Conjecture 1. Every irreducible SPD matrix admits a clean ordering.

Conjecture 2. Every irreducible SPD matrix admits an ordering whose first Schur complement has no cancellation at a predicted-nonzero position.

Human

These are genuinely different from the implementation's assumption, in two ways. First, they are existential: they permit the ordering to depend on the numerical entries of A, whereas symbolic factorization must choose σ from G_A alone. Second, they quantify over all SPD matrices, including the measure-zero ones the genericity heuristic sets aside. A counterexample must therefore be an exact algebraic object, and this is exactly why "cancellation is unlikely" and "Conjecture 1 is false" can both be true: the two statements live on complementary sets.

Conjecture 2 is the stronger demand, and the gap between them is the phenomenon we call refill. A zero created in one Schur complement is not yet a cancellation in L: the entry (j, k) is committed only when the earlier of j, k is pivoted, and an intervening elimination can restore it to a nonzero value. Any analysis that conflates Schur-complement zeros with factor zeros will reach false conclusions.

1.4. The same question in statistics. The existential question has a second life in Gaussian graphical models, where a zero in a Schur complement is a conditional independence: $(A/A_S)_{jk} = 0$ says exactly that X_j and X_k are conditionally independent given X_S. A distribution is faithful to its graph when no conditional independence holds beyond those the graph implies, so a cancellation at a predicted-nonzero position is a failure of faithfulness.

That literature has settled one case decisively. Malouche and Rajaratnam [MR11] proved, using the covariance–precision path formula of Jones and West [JW05], that a Gaussian whose covariance graph is a tree is necessarily covariance faithful. The dual statement for concentration graphs is older still: Becker, Geiger and Meek [BGM05] proved that a Gaussian whose concentration graph is a tree is necessarily concentration faithful. In our language the covariance version is Conjecture 1 for forests, in the strong form that every ordering is clean – proved fifteen years ago under a different name, its dual a decade earlier again, and neither cited in the numerical literature that assumes it. Beyond trees the two communities part company: Meek [Mee95] and Uhler, Raskutti, Bühlmann and Yu [URBY13] study when faithfulness holds generically, showing that unfaithful distributions have measure zero while strong-faithfulness failures occupy a set of positive measure, but neither literature asks whether some ordering can always be chosen to avoid the failures that do occur.

1.5. Results. We answer the existential question negatively in general and positively on a substantial and now precisely delimited class.

Conjecture 2 is false at order 4 (Theorem 5.1): an explicit matrix A^* with spectrum $\{9, 3, 3, 1\}$ has no ordering whose first Schur complement is cancellation-free, although 8 of its 24 orderings give a clean factor – the refill phenomenon in its simplest form.

Conjecture 1 holds for $n \leq 4$ by an elementary case analysis of the six connected patterns on four vertices, with no computation (Theorem 5.2), and is false at every even order at least 6 (Theorem 5.8), via a family M_{2m} built from a pairing of the vertices, with within-pair entries of magnitude AB and cross-pair blocks $[[A, B], [B, A]]$. The family is not exhibited but derived: at any level with four surviving vertices, exactly three of the $4096 = 2^{12}$ candidate zero-configurations are feasible and use all four vertices as pivots, and each is a fixed-point-free involution (Lemma 5.5). The three magnitudes and the block structure are forced as the unique solution of the resulting additive system; the requirement of a fixed-point-free involution is precisely why the order must be even. The parameters are forced onto the measure-zero curve $A^2 + (1 - B^2)A - 1 = 0$, and conversely every point of that curve with $0 < B < 1$ yields a counterexample (Remark 5.9), so the construction is a one-parameter family; the instance $B = 1/2$, with $A = (-3 + \sqrt{73})/8$, is the one

Human

we verify explicitly.

Order 5 remains open (Section 7). At order six the complete graph K_6 fails, by Theorem 5.8; the other connected patterns of order six, the 14-edge pattern $K_6 - e$ among them, remain open.

Structural classes. Independently of the order-by-order picture we prove Conjecture 1 for forests in the strong form above; for 2-path strips and dominating-vertex fans; for chordal graphs with exactly one triangle; and for chordal graphs of clique number at most 3 whose triangles are pairwise edge-disjoint. Beyond forests the clean ordering must adapt to the matrix: the triangle lies in each of these classes, and no fixed ordering is clean for every SPD matrix whose pattern is a triangle (Remark 4.9).

The counterexamples refuting both conjectures, and the structural classes of Section 4 beyond forests, were found with AI assistance and insight; AI finds the counterexamples in Section 5, provides the calculations showing the counterexamples work, and comes up with key ideas for the proofs in Section 4.

Odd orders at least 7 remain open. The reason is structural rather than computational: the classification of Lemma 5.5 that forces the even construction requires exactly four surviving vertices, and its injectivity step fails at five, so parity confers no protection at odd orders. Remark 5.7 records this together with the two computations that delimit the position — no scheme leaving one survivor at every vertex exists at order five, and an exhaustive search at order seven did not terminate.

1.6. Method. Three devices carry the analysis. We recast cleanliness as a covering problem: a slot $(S, \{j,k\})$ asserts $(A/A_S)_{jk} = 0$, and Conjecture 1 fails exactly when some blocking family of slots – one hitting every ordering – is realizable by an SPD matrix. Second, since a cancellation reads $r_{jk} = r_{uj}r_{uk}$, the substitution $t_{jk}^{(S)} = -\log|r_{jk}^{(S)}|$ makes every cancellation additive, $t_{jk}^{(S)} = t_{uj}^{(S)} + t_{uk}^{(S)}$, with $t_{jk}^{(S)} > 0$ forced by strict definiteness; obstructions then become statements about a linear system, refutable by Stiemke's lemma [Sti15, Gor73] with exact rational certificates. Third, unrealizability is monotone in the slot family (Lemma 3.18), which is what makes exhaustive search over patterns finite and sound.

1.7. Related work. Four strands of literature bear on the existential question, and it is worth saying what we take from each.

Sparse direct methods. The combinatorial model is due to Rose, Tarjan and Lueker [RTL76]: the elimination game, the fill graph G_σ^+, and the fill-path theorem characterizing its edges by paths whose interior has already been eliminated. Everything symbolic in this paper is a consequence of that characterization, recorded here as Proposition 2.10; what we add is the question of when the symbolic prediction is numerically exact. Minimizing fill is NP-hard [Yan81], which is why production codes choose the ordering by heuristics, and the accounts of Heath, Ng and Peyton [HNP91] and of Davis [Dav06] describe the two-phase practice that Section 6 addresses: a symbolic phase allocates for G_σ^+, and a numeric phase fills that structure in. Demmel's lecture notes [Dem24] state the underlying assumption explicitly. Our results separate two readings of it: as an allocation bound it is correct, since the numeric pattern is always contained in the symbolic one (Definition 2.12), but as an existential statement about orderings it is false at every even order from six onwards.

Chordal graphs. The positive results of Section 4 proceed by choosing simplicial pivots, so they rest on the classical theory: Dirac's theorem that a chordal graph has a simplicial vertex [Dir61], the characterization of chordality by perfect elimination orderings [FG65], and the standard treatment

Human

of Golumbic [Gol80]. Our setting differs from the usual one in a way worth emphasizing. A perfect elimination ordering is optimal for fill, but optimality for fill is not cleanliness: Remark 4.6 exhibits a 2-path strip and SPD realizations of it for which no fill-free ordering is clean, so the induction of Theorem 4.8 must sometimes create fill deliberately. The strip and fan classes are partial 2-trees in the sense of Bodlaender [Bod98], and general partial 2-trees are the natural next case, which is why they head the list of open problems in Section 7.

Gaussian graphical models and faithfulness. Under the dictionary of Lauritzen [Lau96], a Schur-complement zero is a conditional independence, so a cancellation at a predicted-nonzero position is a failure of faithfulness. Malouche and Rajaratnam [MR11] proved that a Gaussian whose covariance graph is a tree is covariance faithful; in our language that is the Fill-Graph Conjecture for forests in the strong form that every ordering is clean, and Theorem 4.1 re-derives it in factorization language using the path formula of Jones and West [JW05], which is also the identity that drives the proof. Trek separation [STD10] gives the general combinatorial criterion for vanishing minors in such models, and our slots are instances of the vanishing conditions it classifies. The faithfulness literature then parts company with our question, and the difference is worth stating precisely: Meek [Mee95], Zhang and Spirtes [ZS08], and Uhler, Raskutti, Bühlmann and Yu [URBY13] ask when faithfulness holds *generically* or *strongly*, and show that unfaithful distributions form a set of measure zero while strong-faithfulness failures occupy a set of positive measure. Those are statements about typical matrices. Ours is existential and quantifies over all SPD matrices, so a counterexample must be an exact algebraic object and necessarily lives on the measure-zero locus that this literature sets aside; Appendix H explains how the two coexist.

Tools. Three further bodies of work are used technically rather than thematically. The quotient identity for Schur complements, which is what licenses treating a level as a set rather than a sequence, is due to Haynsworth and to Crabtree and Haynsworth [Hay68, CH69]; it appears here as Lemma 2.4. Which zero patterns are realizable by an SPD matrix at all is the positive-definite completion problem of Grone, Johnson, Sá and Wolkowicz [GJSW84], with clique trees [BP93] as the structure behind the chordal case; this is the background against which our notion of realizability should be read, since a blocking family refutes the conjecture only if some SPD matrix realizes it. The conditional-independence axioms for Gaussians catalogued by Lnenicka and Matus [LM07] contain our Nesting Lemma 3.14 as the weak-transitivity axiom. Finally, the infeasibility certificates of Section 3 are an application of Stiemke's transposition theorem [Sti15], a refinement of Gordan's [Gor73].

1.8. Organization. Section 2 sets notation and states conjectures. Section 3 proves core lemmas. Section 4 proves positive results including PEO-insufficiency witness. Section 5 proves Pairing Lemma, derives M_{2m}, and states A^*. Section 7 states open problems and established results. Appendix gives computational protocol. Figures illustrate the forest path formula, the refill mechanism, the strip induction, and the pairing structure of M_{2m}.

1.9. Why it matters. These results close a foundational gap between theory and practice felt in several communities. For numerical linear algebra and sparse direct solvers, symbolic factorization allocates G^+ from the graph alone assuming no cancellation; we show the assumption is safe for correctness (over-allocation) yet false existentially: $G^L_\sigma = G^+_\sigma$ cannot be guaranteed for all SPD matrices, the smallest known failure occurring at order 6 (K_6, via M_6; order five is open). Order 4 already obstructs the weaker first-Schur version (A^*: 0/24 stage-1 clean), while its 8/24 clean factors show that refill separates the two questions.

Human

For graph theory and combinatorial matrix theory, we map the boundary of when a chordal completion can be realized without algebraic accident: forests are strongly clean (every ordering clean), 2-path strips, fans, one-triangle adaptive, and edge-disjoint triangles with $\omega \leq 3$ are clean, while K_6 fails. Beyond order four, every positive class has clique number at most 3, while the failures we construct are complete graphs; the 4-vertex pairing involution forced on the curve $A^2 + (1 - B^2)A - 1 = 0$ explains how these fail, and where the boundary lies in between is open.

For statistics, graphical models, and causal inference, a cancellation $(A/A_S)_{jk} = 0$ is a failure of faithfulness. We re-derive, in factorization language, Malouche and Rajaratnam's result that forests are faithful in the strong form every ordering clean (proved under a different name fifteen years ago, never cited in the numerical literature that assumes it). We exhibit measure-zero unfaithful M_{2m} on K_{2m} with $A^2+(1-B^2)A-1 = 0$, and explain why unfaithfulness has measure zero (faithfulness is generic) yet coexists with existential failure.

For theoretical computer science, the covering problem over slots and the integer Stiemke certificates $y^\top T \geq 0$ used to refute blocking families provide a transposition-theorem view of when a chordal completion can be realized without algebraic accident.

Statement of AI Use

Human

This paper was developed through collaboration between researchers and Muse Spark (AI model) via the meta.ai chat interface. The model helped explore ideas, develop candidate arguments, and draft material. Researchers guided the work, checked and corrected the mathematics, and take ownership for the final manuscript. Following [Sch25], margin markers identify human-drafted (**Human**) and AI-assisted material (**AI**) separately.

Concurrent Work

Human

We are not aware of any concurrent work at the time of publication.

2. Setting, vocabulary, and conjectures

Human

This section fixes every notion used in Sections 3–5. It is meant to be read once and consulted thereafter; Table 1 collects the notation. Three points are worth flagging before the formal definitions. First, we work with two graphs at once: a *symbolic* graph predicted from the sparsity pattern alone, and the *numeric* pattern of the actual Schur complement. The whole subject is the gap between them. Second, elimination is indexed by *sets* rather than sequences, which is legitimate by the quotient property (Lemma 2.4). Third, a *slot* is a formal assertion that some entry vanishes; it becomes true or false only relative to a matrix, and that distinction drives the entire method.

2.1. SPD matrices and Schur complements of sets.

Definition 2.1 (SPD)**.** A symmetric $n \times n$ real matrix A is *symmetric positive definite* (SPD), written $A \succ 0$, if $x^\top Ax > 0$ for every $x \neq 0$. Throughout, $V = \{0, 1, \ldots, n-1\}$ indexes the rows and columns of A, and we freely call the elements of V *vertices*.

Definition 2.2 (Schur complement of a set)**.** Let A be symmetric and let $S \subseteq V$ be *any* subset

Human

with $A_{SS} \succ 0$; put $R = V \setminus S$ and write $A = \begin{pmatrix} A_{SS} & A_{SR} \\ A_{RS} & A_{RR} \end{pmatrix}$ after a symmetric permutation. The *Schur complement of S in A* is the matrix on R

$$A/A_S = A_{RR} - A_{RS}(A_{SS})^{-1}A_{SR}.$$

For a single vertex, $A/A_{\{v\}} = B - rr^\top/a$ when $A = \begin{pmatrix} a & r^\top \\ r & B \end{pmatrix}$ with $a = A_{vv}$. For an ordering $\sigma = (v_1, \dots, v_n)$ we abbreviate $S^{(0)} = A$ and $S^{(j)} = A/A_{\{v_1,\dots,v_j\}}$.

Proposition 2.3 (Basic properties)**.** *If $A \succ 0$ and $S \subseteq V$ then $A_{SS} \succ 0$, so A/A_S is defined, and $A/A_S \succ 0$. Moreover A/A_S depends only on the* set *S, not on any ordering of it.*

Proof. $A_{SS} \succ 0$ because it is a principal submatrix of A. For y supported on R, one has $y^\top(A/A_S)y = \min\{x^\top Ax : x|_R = y\} > 0$, so $A/A_S \succ 0$. Order-independence is immediate from the closed formula of Definition 2.2, which mentions no ordering of S; the compatibility with step-by-step elimination is Lemma 2.4. □

Lemma 2.4 (Quotient property, transitivity of Schur complements)**.** *Let $A \succ 0$, $S \subset V$ and $v \in V \setminus S$, and set $M = A/A_S$. Then*

$$M/M_{\{v\}} = A/A_{S\cup\{v\}}$$

as matrices on $V \setminus (S \cup \{v\})$. That is, eliminating S and then v gives the same result as eliminating $S \cup \{v\}$ at once.

Proof. Write A partitioned, with $R = V \setminus (S \cup \{v\})$, as

$$A = \begin{pmatrix} A_{SS} & A_{Sv} & A_{SR} \\ A_{vS} & a_{vv} & a_{vR} \\ A_{RS} & a_{Rv} & A_{RR} \end{pmatrix}.$$

Then $A/A_S = \begin{pmatrix} b & c \\ c^\top & A_{RR} - A_{RS}A_{SS}^{-1}A_{SR} \end{pmatrix}$ where $b = a_{vv} - A_{vS}A_{SS}^{-1}A_{Sv}$ and $c = a_{vR} - A_{vS}A_{SS}^{-1}A_{SR}$. Taking the Schur complement of $\{v\}$ in M yields $A_{RR} - A_{RS}A_{SS}^{-1}A_{SR} - cb^{-1}c^\top$, which is $A/A_{S\cup\{v\}}$ by block inversion of $A_{S\cup\{v\},S\cup\{v\}}$. See Haynsworth [Hay68] and Crabtree–Haynsworth [CH69] for the general quotient formula. □

Remark 2.5 (Levels and stages)**.** Because of Proposition 2.3 and Lemma 2.4 we may speak of a *level*: a set $S \subseteq V$ of already-eliminated vertices, with A/A_S the matrix currently being factored and $R = V \setminus S$ the *survivors*. The *stage* is the number $|S|$ of completed eliminations, so an ordering passes through levels $\emptyset = S_0 \subset S_1 \subset \cdots \subset S_n = V$ with $|S_j| = j$. "Level" is thus a set and "stage" is a counter; a statement "at level S" is a statement about the single matrix A/A_S, whereas a statement "at stage j" refers to whichever level of size j the ordering has reached.

2.2. Pivots, the elimination game, and fill graphs.

Definition 2.6 (Pivot)**.** Let S be a level and $p \in R = V \setminus S$ a survivor, and put $M = A/A_S$. To *pivot at p* is to pass from level S to level $S \cup \{p\}$, that is, to replace M by

$$M/M_{\{p\}}, \qquad (M/M_{\{p\}})_{jk} = M_{jk} - \frac{M_{jp}M_{kp}}{M_{pp}} \quad (j,k \in R \setminus \{p\}),$$

a symmetric rank-one update of the surviving block. We call $D = M_{pp} > 0$ the *pivot value*, the

Human

vector $(M_{kp})_{k\in R\setminus\{p\}}$ the *pivot column*, and the two factors M_{jp}, M_{kp} appearing in the update of the pair (j,k) the *legs* of that update (the legs of the route $j-p-k$). An ordering σ is thus exactly a schedule of n pivots, and the Cholesky factor is assembled from the pivot columns (Definition 2.23).

Definition 2.7 (Pattern graph, elimination game, G_S)**.** The *pattern graph* G_A has vertex set V and an edge $\{i,j\}$, $i\neq j$, exactly when $A_{ij}\neq 0$. The *elimination game* on G_A is the following purely combinatorial process. Fix an ordering $\sigma=(v_1,\dots,v_n)$ of V and put $H_0=G_A$. At step i let

$$F_i=\{\{x,y\}: x\neq y,\ x,y\in N_{H_{i-1}}(v_i),\ \{x,y\}\notin E(H_{i-1})\}$$

be the *fill edges created by* v_i, and let H_i be H_{i-1} together with F_i, restricted to the surviving vertices $\{v_{i+1},\dots,v_n\}$. Neighborhoods are always taken in the *current* graph H_{i-1}, not in G_A; this is what makes fill beget fill.

For a level $S\subseteq V$ we define G_S, a graph on the survivors $V\setminus S$, as follows: run the game along any ordering of V whose first $|S|$ entries enumerate S, and stop after $|S|$ steps. Thus $G_S=H_{|S|}$ for such an ordering, and in particular

$$G_\emptyset=H_0=G_A,\qquad G_{\{v_1,\dots,v_i\}}=H_i.$$

Remark 2.8. The notation in the definition is justified because G_S depends only on the *set* S, not on the order in which its vertices were eliminated, nor on how the ordering continues after them; this is immediate from the intrinsic description

$$\{j,k\}\in G_S \iff G_A \text{ contains a } j\text{–}k \text{ path all of whose interior vertices lie in } S \qquad (j,k\in V\setminus S), \tag{1}$$

which is part of the fill-path theorem of Rose, Tarjan and Lueker [RTL76] and is recorded as Proposition 2.10 below. Two consequences of (1) are used constantly: G_S contains the subgraph of G_A induced on $V\setminus S$ (take the one-edge path, so eliminating vertices never destroys an edge between survivors), and G_S grows with S, in the sense that if $S\subseteq T$ then every edge of G_S with both ends outside T is an edge of G_T.

Definition 2.9 (Fill graph G_σ^+)**.** The *fill graph* of σ is the graph on the *full* vertex set V

$$G_\sigma^+=\big(V,\ E(G_A)\cup F_1\cup\cdots\cup F_n\big):$$

original edges together with every fill edge ever created, and no edge or vertex is ever removed. The deletions in Definition 2.7 serve only to determine *which* fill edges arise; G_σ^+ records them cumulatively. In particular G_σ^+ is *not* the terminal graph of the game: that graph is G_V, which has no vertices at all.

Proposition 2.10 (Fill-path theorem; two indexings agree)**.** *Let σ be an ordering and $j\neq k$. Put $S=\{u:\sigma^{-1}(u)<\min(\sigma^{-1}(j),\sigma^{-1}(k))\}$, the set of vertices preceding both j and k. Then the following are equivalent:*

1. $\{j,k\}\in G_\sigma^+$;
2. $\{j,k\}\in G_S$;
3. *G_A contains a j–k path all of whose interior vertices lie in S.*

Proof. This is the fill-path theorem of Rose, Tarjan and Lueker [RTL76]; see also [FG65]. The equivalence of (1) and (2) is the observation that the membership of $\{j,k\}$ is decided by the time the earlier of j,k is pivoted and is not changed afterwards, since later steps only add edges among later vertices. □

Human

Remark 2.11 (Sanity checks on G_σ^+)**.** $G_\sigma^+ \supseteq G_A$, so G_σ^+ is never empty for $n \geq 2$ with G_A connected; G_σ^+ is chordal and σ is a PEO of it (Definition 2.14); and G_σ^+ is exactly the symbolic pattern of $L + L^\top$ (Definition 2.23). The last point is why G_σ^+ must live on all of V: the equality $G_\sigma^L = G_\sigma^+$ asserted by Conjecture 2.27 compares two graphs on the vertex set V.

Definition 2.12 (Numeric pattern; predicted-nonzero positions)**.** Let P_S denote the *numeric* pattern of A/A_S: the graph on $V \setminus S$ with an edge $\{j,k\}$ iff $(A/A_S)_{jk} \neq 0$. Let G_σ^L denote the pattern of the actual Cholesky factor L under σ. Always

$$P_S \subseteq G_S, \qquad G_\sigma^L \subseteq G_\sigma^+,$$

with equality in the first exactly when no cancellation has yet occurred at level S. A position $\{j,k\}$ is *predicted nonzero* for σ if $\{j,k\} \in G_\sigma^+$, equivalently $\{j,k\} \in G_S$ for the S of Proposition 2.10. An *accidental cancellation* is a predicted-nonzero position whose numeric value is zero at the stage where it is committed to L.

Example 2.13 (Fill from a path)**.** Let G_A be the path $0-1-2$, so $E(G_A) = \{\{0,1\},\{1,2\}\}$. If σ pivots 1 first then $F_1 = \{\{0,2\}\}$, so $\{0,2\} \in G_{\{1\}}$ and $\{0,2\} \in G_\sigma^+$, although $\{0,2\} \notin G_A$: it is a fill edge. Numerically $(A/A_{\{1\}})_{02} = -A_{01}A_{12}/A_{11} \neq 0$, so the fill edge is nonzero at creation (Lemma 3.12); at larger levels it may still cancel. Symbolic fill is an upper bound for numeric fill, never a lower one.

AI

2.3. Graph vocabulary.

Definition 2.14 (Graph notions used throughout)**.** Let G be a finite simple graph. $\omega(G)$ is its *clique number*, the size of a largest clique. A vertex v is *simplicial* if $N(v)$ is a clique, and *pendant* if $|N(v)| = 1$. G is *chordal* if it has no induced cycle of length at least 4. A *cut vertex* is a vertex whose removal disconnects the component containing it, a *block* is a maximal connected subgraph without a cut vertex, and the *bipartite block-cut tree* is the tree whose nodes are the blocks and the cut vertices, with a block adjacent to each cut vertex it contains; its leaves are always blocks. G is a *clique sum* of G_1 and G_2 along K if $G = G_1 \cup G_2$ and $G_1 \cap G_2 = K$ is a clique.

Remark 2.15. By [Dir61] G is chordal if and only if every minimal separator of G is a clique; equivalently [FG65] G admits a *perfect elimination ordering* (PEO), an ordering in which the later neighbors of each vertex form a clique.

Definition 2.16 (Irreducible matrix)**.** A symmetric matrix A is *irreducible* if there is no partition $V = V_1 \sqcup V_2$ into nonempty parts with $A_{ij} = 0$ for all $i \in V_1$, $j \in V_2$; equivalently, A is not similar by a symmetric permutation to a block-diagonal matrix with more than one block. Irreducibility is the standing hypothesis of Conjectures 2.26 and 2.27.

Proposition 2.17. *A symmetric matrix A is irreducible if and only if G_A is connected.*

Proof. If G_A is disconnected, take V_1 a union of components and V_2 its complement: all crossing entries vanish, so A is reducible. Conversely a partition with $A_{ij} = 0$ across it exhibits no edges of G_A between V_1 and V_2, so G_A is disconnected. □

Remark 2.18 (Why irreducibility is harmless)**.** If A is block diagonal then a clean ordering for each block concatenates to a clean ordering for A, and G_σ^+ is the disjoint union of the blocks' fill graphs; so nothing is lost by assuming G_A connected.

2.4. Cancellations, apexes, and what a pivot can destroy.

Definition 2.19 (Cancellation graph, zeroing, apex, covering, surviving)**.** Let S be a level and

AI

$p \in V \setminus S$ a survivor. Put

$$C_p(S) = \{\{j,k\} : j,k \in V \setminus (S \cup \{p\}),\ j \neq k,\ (A/A_S)_{pp}(A/A_S)_{jk} = (A/A_S)_{pj}(A/A_S)_{pk}\},$$

the *cancellation graph of p at level S*, and write $C_p := C_p(\emptyset)$. Thus $C_p(S)$ is exactly the set of pairs that the pivot at p destroys: by Definition 2.6,

$$\{j,k\} \in C_p(S) \iff (A/A_{S\cup\{p\}})_{jk} = 0,$$

and we then say that p *zeroes the pair* $\{j,k\}$ *at level* S, that p is the *apex* of that cancellation, and that $j - p - k$ is the *route*. We say $C_p(S)$ *covers* a vertex q if some pair of $C_p(S)$ contains q, and that q *survives* the pivot at p otherwise; equivalently, q survives if $(A/A_{S\cup\{p\}})_{qk} \neq 0$ for every $k \in V \setminus (S \cup \{p,q\})$. A survivor p is a *clean pivot at level* S if $C_p(S) \cap E(G_S) = \emptyset$, i.e. it destroys no predicted-nonzero pair; for a triangle T we say p is a *clean apex for* T if $C_p(S)$ contains no pair of T. The unqualified phrase *clean apex* means $C_p(S) = \emptyset$.

Remark 2.20 (What $C_p(S)$ is for)**.** Every result in Section 3 is a constraint on the sets $C_p(S)$: the Exchange and Triangle Lemmas say they cannot be too large inside a triangle; the Max-Edge and Dense Safety Lemmas exhibit pivots whose cancellation graph misses a prescribed vertex; Fact 1 says that a universal pivot destroys every pre-existing zero, so that after it the zero set is *exactly* $C_p(S)$. Choosing a clean ordering means walking through levels always pivoting at a vertex whose $C_p(S)$ avoids the predicted-nonzero pairs it would commit.

Definition 2.21 (Universal vertex)**.** With $M = A/A_S$ and $R = V \setminus S$, a survivor $v \in R$ is *universal in* M if $M_{vj} \neq 0$ for every $j \in R \setminus \{v\}$, i.e. its row of M has no zero among the survivors.

Remark 2.22. If v is universal then eliminating v makes all survivors pairwise adjacent symbolically and, by Lemma 3.5, destroys every pre-existing zero incident to the surviving block: after the pivot the zero set is exactly $C_v(S)$. Universality is the hypothesis under which "the only zeros are the ones this pivot creates".

2.5. Clean orderings, factor entries, and strategies.

Definition 2.23 (Factor entry)**.** Let $\sigma = (v_1, \ldots, v_n)$, $S_{j-1} = \{v_1, \ldots, v_{j-1}\}$, $M = S^{(j-1)} = A/A_{S_{j-1}}$, $p = v_j$ and $D_j = M_{pp}$. Then for every k after p in σ,

$$L_{kp} = M_{kp}/\sqrt{D_j} = (A/A_{S_{j-1}})_{kp}/\sqrt{D_j},$$

so the p-th column of L is the scaled pivot column and $L_{kp} \neq 0$ iff $(A/A_{S_{j-1}})_{kp} \neq 0$. Consequently, if j precedes k and S is the set of vertices preceding j, then $L_{kj} \neq 0$ iff $(A/A_S)_{jk} \neq 0$: the pair $\{j,k\}$ is *committed to* L exactly when the earlier of j,k is pivoted, and G_σ^L is the pattern of $L + L^\top$.

Definition 2.24 (Clean ordering)**.** An ordering σ is *clean* for A if for every $\{j,k\} \in G_\sigma^+$, writing $S = \{u : \sigma^{-1}(u) < \min(\sigma^{-1}(j), \sigma^{-1}(k))\}$, we have $(A/A_S)_{jk} \neq 0$. Equivalently, by Definition 2.23, $G_\sigma^L = G_\sigma^+$: no predicted nonzero cancels in the factor. This is the *factor* notion; it is strictly weaker than demanding $(A/A_S)_{jk} \neq 0$ at *every* level S with $\{j,k\} \in G_S$, because a zero created at one level can be refilled before the pair is committed (Section 5).

Definition 2.25 (Strategies: fixed, adaptive, peeling)**.** A *fixed ordering* for a pattern G is a single permutation σ, chosen from G alone, to be used for every SPD matrix with $G_A = G$. An *adaptive strategy* chooses each pivot as a function of the current matrix A/A_S; it therefore produces, for each SPD A with $G_A = G$, an ordering $\sigma(A)$. We say G *admits an adaptive clean ordering* if some adaptive strategy yields a clean ordering for every SPD A with $G_A = G$. This asks no more than a

clean ordering for each matrix: if every SPD A with $G_A = G$ has a clean ordering, then G admits an adaptive clean ordering, and the converse is immediate. Indeed, whether the pivots still to come can be completed cleanly from a level S depends only on S, on G and on the current matrix A/A_S, by Lemma 2.4 and Proposition 2.10; so the strategy that pivots a vertex beginning such a completion produces a clean ordering for every A that has one. A fixed ordering is a genuinely stronger demand: Remark 4.9 shows that the triangle admits an adaptive clean ordering but no fixed σ that is clean for all matrices with that pattern. To *peel* a simplicial survivor v is to pivot at it; since $N(v)$ is a clique, no fill is created, and if v is pendant then no pair of survivors is updated at all, so peeling a pendant vertex can neither create nor destroy a zero. The two named moves *END* and *SECOND* used for strips in Section 4 are, respectively, peeling the end vertex of a strip and pivoting its neighbor; they are defined precisely in Definition 4.5.

Conjecture 2.26 (First-Schur Conjecture, "Conjecture 2" of Section 1.3)**.** *For every irreducible SPD matrix A there exists an ordering σ such that the first Schur complement $S^{(1)} = A/A_{\{\sigma_1\}}$ has no cancellation at a predicted-nonzero position.*

Conjecture 2.27 (Fill-Graph Conjecture, "Conjecture 1" of Section 1.3)**.** *For every irreducible SPD matrix A there exists an ordering σ with $G_\sigma^L = G_\sigma^+$, that is, a clean ordering.*

2.6. Slots, blocking families, and realizability.

Definition 2.28 (Slot; active)**.** A *slot* is a formal triple (S, j, k) with $S \subseteq V$ a level and $j, k \in V \setminus S$, $j \neq k$. It is a *demand*, not a fact: it asserts the equation $(A/A_S)_{jk} = 0$. Given a particular matrix A, the slot is *active for A* if that equation actually holds. Thus slots are combinatorial objects quantified over, and activity is the only notion that refers to numbers; a family of slots is a system of equations whose solvability is the question.

Definition 2.29 (Spoiling, blocking, realizable, feasible)**.** A slot (S, j, k) *spoils* an ordering σ if S is exactly the set of predecessors of the earlier of j, k in σ and $\{j, k\} \in G_S$; that is, the position is predicted nonzero and is committed to L precisely at level S, so activity of the slot makes σ unclean. A family H of slots is *blocking* for a pattern G if every permutation of V is spoiled by some slot of H. H is *realizable* for G if there is an SPD A with $G_A = G$ for which every slot of H is active. A family of slots at a single fixed level S is *feasible* if the additive system $Tt^{(S)} = 0$ of Definition 2.32 that encodes it admits a strictly positive solution $t^{(S)} > 0$, and *infeasible* otherwise. Feasibility is the linear surrogate for realizability at one level: by Lemma 2.33, an SPD A that makes every slot $(S \cup \{v\}, j, k)$ of the family active, with both legs $(A/A_S)_{vj}$ and $(A/A_S)_{vk}$ nonzero, makes it feasible (take $t^{(S)}$ to be the log-magnitudes of A/A_S), so infeasibility refutes realizability by such matrices. Without nonzero legs Lemma 2.33 supplies no equation.

Remark 2.30 (The covering reformulation)**.** Conjecture 2.27 fails for a pattern G exactly when some blocking family is realizable for G: a matrix has no clean ordering iff its active slots spoil every permutation. Section 5 exploits one direction: to refute the conjecture for G one realizes a blocking family. The converse direction, refuting every blocking family, is what a positive result at a given order requires.

Example 2.31 (Spoiling and blocking)**.** The slot $(\emptyset, 0, 1)$ spoils every ordering whose *first* vertex is 0 or 1 (provided $\{0, 1\} \in G_A$), because then $S = \emptyset$ is exactly the predecessor set of the earlier of $0, 1$; it does not require the ordering to begin with the pair $\{0, 1\}$.

For a triangle on $\{0, 1, 2\}$ the three apex slots $(\{0\}, 1, 2)$, $(\{1\}, 0, 2)$, $(\{2\}, 0, 1)$ form a blocking family: whichever vertex comes first, its apex slot spoils the ordering, since after that pivot the opposite edge is predicted nonzero ($\{1, 2\} \in G_{\{0\}}$, and so on). The family is not realizable, be-

AI

cause by Lemma 3.2 at most one apex route of a triangle can cancel. Contrast the level-$\emptyset$ triple $\{(\emptyset,0,1),(\emptyset,0,2),(\emptyset,1,2)\}$, which is unrealizable for a trivial reason of a different kind: it asserts $A_{01}=A_{02}=A_{12}=0$, which contradicts G_A containing the triangle. Lemma 3.2 says nothing about level $\emptyset$.

2.7. The additive parametrization.

Definition 2.32 (Partial correlation, log-magnitude, legs)**.** Let S be a level and $j,k\in V\setminus S$ distinct with $(A/A_S)_{jk}\neq 0$. The *partial correlation* is

$$r_{jk}^{(S)}=\frac{(A/A_S)_{jk}}{\sqrt{(A/A_S)_{jj}(A/A_S)_{kk}}},$$

and $|r_{jk}^{(S)}|<1$ because the 2×2 principal minor $(A/A_S)_{jj}(A/A_S)_{kk}-(A/A_S)_{jk}^2$ of the SPD matrix A/A_S is positive. Put $t_{jk}^{(S)}=-\log|r_{jk}^{(S)}|\in(0,\infty)$ for such pairs, with the convention $t_{jk}^{(S)}=+\infty$ when $(A/A_S)_{jk}=0$; in particular $\log 0$ is never taken, and $t_{jk}^{(S)}>0$ is exactly strict definiteness on the pair $\{j,k\}$.

Lemma 2.33 (Cancellation is additive in t)**.** *Let S be a level, v a survivor, and j,k survivors distinct from v whose* legs $(A/A_S)_{vj}$ *and* $(A/A_S)_{vk}$ *are both nonzero (this is what "legs nonzero" means: the two entries of the pivot column at v used in the update of the pair $\{j,k\}$, in the sense of Definition 2.6). Then*

$$(A/A_{S\cup\{v\}})_{jk}=0\iff r_{jk}^{(S)}=r_{vj}^{(S)}\,r_{vk}^{(S)},$$

and in that case

$$t_{jk}^{(S)}=t_{vj}^{(S)}+t_{vk}^{(S)}.$$

Conversely the additive relation together with the sign condition $\operatorname{sgn} r_{jk}^{(S)}=\operatorname{sgn} r_{vj}^{(S)}\operatorname{sgn} r_{vk}^{(S)}$ *implies the cancellation.*

Proof. Divide the update

$$(A/A_{S\cup\{v\}})_{jk}=(A/A_S)_{jk}-\frac{(A/A_S)_{vj}(A/A_S)_{vk}}{(A/A_S)_{vv}}$$

by $\sqrt{(A/A_S)_{jj}(A/A_S)_{kk}}$, which is positive because A/A_S is SPD. The first term becomes $r_{jk}^{(S)}$ by Definition 2.32. For the second, observe that each leg carries a factor $\sqrt{(A/A_S)_{vv}}$ in its own normalisation, so that

$$r_{vj}^{(S)}r_{vk}^{(S)}=\frac{(A/A_S)_{vj}}{\sqrt{(A/A_S)_{vv}(A/A_S)_{jj}}}\cdot\frac{(A/A_S)_{vk}}{\sqrt{(A/A_S)_{vv}(A/A_S)_{kk}}}=\frac{(A/A_S)_{vj}(A/A_S)_{vk}}{(A/A_S)_{vv}\sqrt{(A/A_S)_{jj}(A/A_S)_{kk}}},$$

which is exactly the normalised correction term: the single power of $(A/A_S)_{vv}$ appearing in the update is absorbed by the two half-powers contributed by the legs, so no normalisation of the diagonal is needed. Hence the normalised update equals $r_{jk}^{(S)}-r_{vj}^{(S)}r_{vk}^{(S)}$, and since the divisor is nonzero the first equivalence follows. Taking $-\log|\cdot|$ of $r_{jk}^{(S)}=r_{vj}^{(S)}r_{vk}^{(S)}$ and using Definition 2.32 gives the additive relation. For the converse, the additive relation determines the magnitudes only, so it yields the cancellation precisely when the signs agree; that is the stated condition on $\operatorname{sgn} r$, a linear condition over GF(2) independent of the magnitudes. □

AI

Remark 2.34 (How the additive system is used)**.** A family of slots at a level S imposes, by Lemma 2.33, a system of linear equations $Tt^{(S)} = 0$ in the unknowns $t_{jk}^{(S)} > 0$, one row per slot, on every realizing matrix whose legs are nonzero. Only the forward (magnitude) implication is used, so the sign conditions are separated off and handled as a parity question; this can only make our refutations stronger, since we refute a system implied by realizability with nonzero legs. By Stiemke's alternative (Section 3), infeasibility of $Tt = 0,\ t > 0$ is certified by a single rational vector y with $y^\top T \geq 0$ and $y^\top T \neq 0$, which is what our computations output.

Table 1: Notation

V, n	vertex set $\{0, \ldots, n-1\}$, order of A
S, R	level (eliminated set); survivors $R = V \setminus S$
stage	$\lvert S\rvert$; $S^{(j)} = A/A_{\{v_1,\ldots,v_j\}}$ (Remark 2.5)
A/A_S	Schur complement of the set S; SPD, order-independent
pivot at p	pass from A/A_S to $A/A_{S\cup\{p\}}$ (Definition 2.6)
pivot value, legs	$D = M_{pp}$; the entries M_{jp}, M_{kp} updating the pair $\{j,k\}$
G_A	pattern graph of A
G_S	symbolic elimination-game graph on $V \setminus S$ (Definition 2.7)
G_σ^+	fill graph on all of V: original $\cup$ all fill edges (Definition 2.9)
G_V	terminal graph of the game: empty, and *not* G_σ^+
P_S	numeric pattern of A/A_S; $P_S \subseteq G_S$
G_σ^L	pattern of the actual factor $L + L^\top$; $G_\sigma^L \subseteq G_\sigma^+$
predicted nonzero	$\{j,k\} \in G_\sigma^+$, i.e. $\{j,k\} \in G_S$ at the committing level
$r_{jk}^{(S)}$, $t_{jk}^{(S)}$	partial correlation ($\lvert r\rvert < 1$); $-\log\lvert r\rvert > 0$, $+\infty$ if the entry is 0
$C_p(S)$	cancellation graph: pairs destroyed by the pivot at p
zeroes, apex, route	$\{j,k\} \in C_p(S)$; p is the apex; route $j-p-k$
covers, survives	$C_p(S)$ covers q if some pair of it contains q; else q survives
clean pivot / clean apex	$C_p(S) \cap E(G_S) = \emptyset$ / $C_p(S) = \emptyset$; "clean apex for T" is relative to a triangle T
universal	$M_{vj} \neq 0$ for all survivors $j \neq v$
slot (S,j,k)	formal demand $(A/A_S)_{jk} = 0$; *active* for A if it holds
spoils, blocking	spoils σ at its committing level; blocking = spoils every σ
realizable, feasible	$\exists$ SPD A with $G_A = G$ making H active; $Tt^{(S)} = 0$ has $t^{(S)} > 0$
clean, adaptive	$G_\sigma^L = G_\sigma^+$; pivots chosen from the current matrix (Definition 2.25)
peel, END, SECOND	pivot a simplicial (resp. strip-end, strip-second) vertex
$\omega(G)$, simplicial	clique number; $N(v)$ is a clique
chordal, PEO	no induced cycle ≥ 4; $\iff$ has a perfect elimination ordering [Dir61, FG65]
block-cut tree	bipartite tree of blocks and cut vertices; leaves are blocks

3. Core lemmas

3.1. Roadmap and Stiemke.

Human

Our infeasibility arguments all rest on a single linear-algebraic alternative. Let T be a real matrix. Stiemke's lemma [Sti15, Gor73] states that exactly one of the following holds: either there is a strictly positive vector $t > 0$ with $Tt = 0$, or there is a vector y such that $y^\top T \geq 0$ and $y^\top T \neq 0$. We use the alternative in the second direction. To show that an additive system $Tt = 0$ has no strictly positive solution, it suffices to exhibit one such y, and because T has integer entries

Human

in all of our applications, y may be taken with integer entries as well. Each y is therefore an exact rational certificate that can be checked by hand, with no appeal to floating-point computation. The lemmas of this section fall into four families, which we describe in turn. The first family derives local obstructions from the positivity of 2×2 principal minors. The Exchange Lemma 3.1 shows that two cancellation routes through the same pair of vertices cannot both occur. The Triangle Lemma 3.2 applies this to a triangle and concludes that at most one of its three apex routes can cancel, so at least two of its vertices cancel no pair of the triangle. The Max-Edge Lemma 3.3 turns the same inequality into a pivot selection rule: the pair of largest log-magnitude supplies a pivot whose partner survives. The second family consists of maximality arguments that produce safe pivots at a given level. Dense Safety 3.6 carries the Max-Edge rule to an arbitrary level: when every pair of survivors is nonzero and $\{p, q\}$ has the largest log-magnitude, pivoting p cancels no pair containing q. The Universal Successor Lemma 3.8 shows that a vertex whose weakest neighbour is a universal vertex p is itself universal after p is eliminated. Together they let a strategy keep choosing universal pivots. The third family governs how zeros persist, propagate, and are destroyed as the level grows. The Zero-leg Lemma 3.4 shows that when one leg of a route vanishes the corresponding entry is unchanged by the elimination, so a zero already present is carried forward. The Prefix Lemma 3.10 runs this backwards: a zero at a predicted-nonzero position must originate at an apex, which yields Corollary 3.11, that only an edge lying in a triangle of the fill graph can ever cancel. In the opposite direction, Fact 1 3.5 shows that eliminating a universal vertex destroys every pre-existing zero among the survivors, because the update contributes a nonzero product; this is the refill mechanism in its simplest form. The Refill Lemma 3.9 is its structural counterpart under the hypothesis $\omega(G_A) \le 3$: a cancelled pair lying in a second triangle forces the two apexes to be non-adjacent, since otherwise the four vertices would form a K_4, and the second apex then restores the entry. Finally, Fresh fill 3.12 shows that a fill edge is nonzero at the moment it is created, though it may cancel at a later level, and the Nesting Lemma 3.14 shows that a pair cannot be zero at two consecutive levels without one of its legs vanishing. The fourth family establishes infeasibility of the additive system and the monotonicity that makes exhaustive search sound. A cancellation relation reads $t_{jk} = t_{ji} + t_{ki}$ with all terms positive, so each relation strictly decreases the log-magnitude along the arcs of the digraph of Definition 3.15. The General Cycle Lemma 3.16 concludes that a directed cycle is impossible, since it would give $t_x > \cdots > t_x$. The 4-cycle Lemma 3.17 is the instance of this used in the pairing classification of Lemma 5.5. The Monotone Lemma 3.18 states that unrealizability is inherited by supersets of a slot family, which is what allows the search to restrict attention to minimal blocking families. Underlying all of these is the quotient property of Schur complements, Lemma 2.4, which is what licenses the staged computations: eliminating a level and then one further vertex agrees with eliminating the enlarged level in one step. In summary, the dependencies run Exchange → Triangle → Max-Edge → Dense Safety and Universal Successor; Zero-leg → Prefix → Fresh fill and Nesting; General Cycle → 4-cycle → Pairing Lemma; and the Monotone Lemma closes the search.

Lemma 3.1 (Exchange Lemma)**.** *Let $A \succ 0$, u, j, k distinct, $A_{uk} \neq 0$. If $A_{uu}A_{jk} = A_{uj}A_{uk}$ then $A_{jj}A_{uk} \neq A_{ju}A_{jk}$.*

Proof. Suppose for contradiction that

$$A_{jj}A_{uk} = A_{ju}A_{jk}. \tag{2}$$

If $A_{jk} \neq 0$, then $A_{jk} = A_{uj}A_{uk}/A_{uu}$. Insert this value into (2) and simplify, we get $A_{uu}A_{jj} = A_{uj}^2$. This contradicts the corresponding 2×2 minor has determinant > 0. If $A_{jk} = 0$, then since $A_{uk} \neq 0$, $A_{uj} = 0$. But then (2) implies $A_{jj}A_{uk} = 0$. This is a contradiction because $A_{uk} \neq 0$ by assumption

Human

and $A_{jj} \neq 0$ because it is a diagonal entry of an SPD matrix. □

Lemma 3.2 (Triangle Lemma)**.** *Let $\{j, k, u\}$ be a triangle of G_A whose three off-diagonal entries are all nonzero. Then at most one of its three apex routes cancels, and consequently at least two of j, k, u cancel no pair of the triangle.*

Proof. The pairs of the triangle are $\{j, k\}$, $\{u, k\}$ and $\{u, j\}$, and a vertex can cancel only the pair opposite to it, since by Definition 2.19 the pair cancelled by a pivot consists of two other survivors. There are therefore exactly three apex routes to consider: u cancelling (j, k), j cancelling (u, k), and k cancelling (u, j).

Suppose u cancels (j, k), that is, $A_{uu}A_{jk} = A_{uj}A_{uk}$. Applying Lemma 3.1 to the triple (u, j, k), whose hypothesis $A_{uk} \neq 0$ holds by assumption, we obtain $A_{jj}A_{uk} \neq A_{ju}A_{jk}$, which says precisely that j does not cancel (u, k). Applying the same lemma to the triple (u, k, j), whose hypothesis $A_{uj} \neq 0$ also holds, we obtain $A_{kk}A_{uj} \neq A_{ku}A_{kj}$, so k does not cancel (u, j). Hence if one apex route cancels the other two do not, which is the first assertion. For the second, the vertex whose route cancels is unique, so each of the remaining two cancels no pair of the triangle. □

Lemma 3.3 (Max-Edge Lemma)**.** *Let $A \succ 0$ have all off-diagonal entries nonzero, and let (p, q) maximize $t_{pq}^{(\emptyset)}$ over all nonzero pairs. Then no pair of C_p contains q; consequently q survives the elimination of p.*

Proof. Write $t_{\max} = t_{pq}^{(\emptyset)}$ and suppose, for contradiction, that some pair of C_p contains q, say $(q, k) \in C_p$ with $k \notin \{p, q\}$. By Lemma 2.33 a cancellation is additive in the log-magnitudes, so

$$t_{qk}^{(\emptyset)} = t_{pq}^{(\emptyset)} + t_{pk}^{(\emptyset)}.$$

Every off-diagonal entry of A is nonzero, so $t_{pk}^{(\emptyset)}$ is finite and strictly positive by Definition 2.32, whence $t_{qk}^{(\emptyset)} > t_{\max}$. But $\{q, k\}$ is itself a nonzero pair, so $t_{qk}^{(\emptyset)} \leq t_{\max}$ by the maximality of (p, q). This contradiction shows that no pair of C_p contains q.

For the consequence, let k be any survivor with $\{q, k\}$ predicted nonzero after p is pivoted. By Definition 2.19 we have $(A/A_{\{p\}})_{qk} = 0$ if and only if $(q, k) \in C_p$, and we have just excluded that. Hence every such entry remains nonzero, which is what it means for q to survive the elimination of p. □

Lemma 3.4 (Zero-leg equivalence)**.** *Let S be a level and let $j, k, v \in V \setminus S$ be distinct. If $(A/A_S)_{jv} = 0$ then $(A/A_{S\cup\{v\}})_{jk} = (A/A_S)_{jk}$. Consequently, writing $a = (S, j, v)$, $b = (S, j, k)$ and $c = (S \cup \{v\}, j, k)$: if a is active, then b is active if and only if c is.*

Proof. By Definition 2.6, pivoting at v gives

$$(A/A_{S\cup\{v\}})_{jk} = (A/A_S)_{jk} - \frac{(A/A_S)_{jv}(A/A_S)_{kv}}{(A/A_S)_{vv}},$$

where the pivot value $(A/A_S)_{vv}$ is positive because A/A_S is positive definite. The hypothesis $(A/A_S)_{jv} = 0$ makes the numerator of the correction term vanish, so the correction term is 0 and the two entries agree.

For the consequence, suppose a is active, that is $(A/A_S)_{jv} = 0$. Then b and c assert that the same number is zero, namely $(A/A_S)_{jk} = (A/A_{S\cup\{v\}})_{jk}$, so one of them is active exactly when the other is. □

Lemma 3.5 (Fact 1: a universal pivot destroys every pre-existing zero)**.** *Let S be a level and let v be universal in $M = A/A_S$ (Definition 2.21). If j, k are survivors distinct from v with $M_{jk} = 0$,*

Human

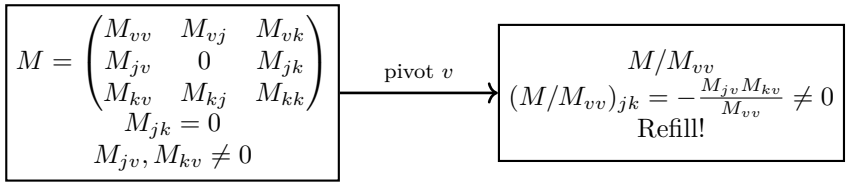
then

$$(M/M_{vv})_{jk} = -\frac{M_{jv}M_{kv}}{M_{vv}} \neq 0.$$

Consequently the set of zero pairs after pivoting at v is exactly $C_v(S)$.

Proof. By Definition 2.6, $(M/M_{vv})_{jk} = M_{jk} - M_{jv}M_{kv}/M_{vv}$. The first term vanishes by hypothesis, and both legs M_{jv}, M_{kv} are nonzero because v is universal, so the update is nonzero. Hence no pre-existing zero survives the pivot, and the only zeros present afterwards are those the pivot itself creates, which by Definition 2.19 are exactly the pairs of $C_v(S)$. Figure 1 shows the mechanism: a zero entry is overwritten by the rank-one update, while the pivot simultaneously creates the zeros of $C_v(S)$. □

AI



Lemma 3.5: universal v destroys pre-existing zeros, same computation as Lemma 3.12 fresh fill nonzero. In strip State1, s_1 refills (s_2, s_3): $(A/A_{S\cup\{s_1\}})_{s_2s_3} = 0 - M_{s_1s_2}M_{s_1s_3}/D \neq 0$.

Figure 1: Refill phenomenon Lemma 3.5: $(M/M_{vv})_{jk} = -M_{jv}M_{kv}/M_{vv} \neq 0$ nonzero even if $M_{jk} = 0$ and legs nonzero. A universal vertex destroys pre-existing zeros, just as fresh fill is nonzero at creation. This is what clears the exceptional zero in State 1 of the strip induction via END. It does *not* apply to the separator pair of a clique sum, where the offending pivot creates the zero rather than inheriting it (Question 7.2).

Lemma 3.6 (Dense Safety)**.** *Let $M = A/A_S$ be such that every pair of survivors $R = V \setminus S$ is nonzero in M, and let $\{p, q\}$ maximize $t_{pq}^{(S)}$ over the pairs with $M_{pq} \neq 0$. Then $C_p(S)$ does not cover q.*

Proof. Write $t_{\max} = t_{pq}^{(S)}$ and suppose that $C_p(S)$ covers q, so that $(q, k) \in C_p(S)$ for some survivor $k \notin \{p, q\}$. By Lemma 2.33, $t_{qk}^{(S)} = t_{pq}^{(S)} + t_{pk}^{(S)}$, and $t_{pk}^{(S)} > 0$ by Definition 2.32 because $M_{pk} \neq 0$. Hence $t_{qk}^{(S)} > t_{\max}$. On the other hand $M_{qk} \neq 0$ by hypothesis, so $\{q, k\}$ is one of the pairs over which the maximum was taken and $t_{qk}^{(S)} \leq t_{\max}$. The two inequalities are incompatible, so $C_p(S)$ does not cover q. □

Remark 3.7. Lemma 3.6 is applied below with p universal in M, which is how it arises in the strip and fan arguments, but universality plays no role in the proof: the density hypothesis on R is what is used.

Lemma 3.8 (Universal Successor)**.** *Let $M = A/A_S$ with survivors R, let p be universal in M, and let $q \in R \setminus \{p\}$ be such that p is a weakest neighbour of q, that is $t_{pq}^{(S)} = \max\{t_{qx}^{(S)} : M_{qx} \neq 0\}$. Then q is universal in M/M_{pp}.*

Proof. Let $k \in R \setminus \{p, q\}$; we must show $(M/M_{pp})_{qk} \neq 0$. If $M_{qk} = 0$, then since p is universal in M, Lemma 3.5 gives $(M/M_{pp})_{qk} = -M_{qp}M_{kp}/M_{pp} \neq 0$. If instead $M_{qk} \neq 0$, then $t_{qk}^{(S)}$ is finite, and $(M/M_{pp})_{qk} = 0$ would be a cancellation, so Lemma 2.33 would give $t_{qk}^{(S)} = t_{pq}^{(S)} + t_{pk}^{(S)} > t_{pq}^{(S)}$, using

AI

$t_{pk}^{(S)} > 0$. That contradicts the maximality of $t_{pq}^{(S)}$ among the $t_{qx}^{(S)}$. In both cases $(M/M_{pp})_{qk} \neq 0$, so q is universal in M/M_{pp}. □

Lemma 3.9 (Refill Lemma)**.** *Assume $\omega(G_A) \leq 3$. Let $\{j,k,u\}$ and $\{j,k,w\}$ be two distinct triangles of G_A sharing the edge $\{j,k\}$, so that $A_{jw} \neq 0$ and $A_{kw} \neq 0$, and suppose that u cancels (j,k). Then $A_{uw} = 0$; moreover, writing $M = A/A_{\{u\}}$, we have $M_{jw} = A_{jw} \neq 0$ and $M_{kw} = A_{kw} \neq 0$, and*

$$(M/M_{ww})_{jk} = -\frac{A_{jw}A_{kw}}{M_{ww}} \neq 0.$$

That is, pivoting at w restores the entry that u destroyed.

Proof. Suppose $A_{uw} \neq 0$. The six pairs among u,j,k,w would then all be nonzero: $\{j,k\}$, $\{u,j\}$ and $\{u,k\}$ because $\{j,k,u\}$ is a triangle, $\{j,w\}$ and $\{k,w\}$ because $\{j,k,w\}$ is a triangle, and $\{u,w\}$ by supposition. Thus $\{u,j,k,w\}$ would be a clique of G_A on four vertices, giving $\omega(G_A) \geq 4$ and contradicting the hypothesis. Hence $A_{uw} = 0$.

Now pivot at u. By Definition 2.6, $M_{jw} = A_{jw} - A_{ju}A_{wu}/A_{uu} = A_{jw}$, since $A_{wu} = A_{uw} = 0$, and likewise $M_{kw} = A_{kw}$; both are nonzero by hypothesis. Since u cancels (j,k) we have $M_{jk} = 0$. Pivoting at w therefore gives

$$(M/M_{ww})_{jk} = M_{jk} - \frac{M_{jw}M_{kw}}{M_{ww}} = -\frac{A_{jw}A_{kw}}{M_{ww}},$$

which is nonzero because the two legs are nonzero and $M_{ww} > 0$ by positive definiteness of M. □

Lemma 3.10 (Prefix Lemma)**.** *Let S be a level and suppose $(A/A_S)_{jk} = 0$ for a pair with $\{j,k\} \in G_S$. Then there exists $v \in S$ with $\{j,v\},\{k,v\} \in G_{S\setminus\{v\}}$.*

Proof. We argue by induction on $|S|$.

If $S = \emptyset$ the hypothesis is vacuous: $\{j,k\} \in G_\emptyset = G_A$ means $A_{jk} \neq 0$, whereas $(A/A_\emptyset)_{jk} = A_{jk} = 0$, so there is nothing to prove.

Let $|S| \geq 1$, write $S = S' \cup \{v\}$ where v is the vertex of S eliminated last, and recall from Lemma 2.4 that this decomposition may be made for any choice of last vertex. There are two cases.

If $\{j,v\} \in G_{S'}$ and $\{k,v\} \in G_{S'}$, then v is the required witness, because $S \setminus \{v\} = S'$.

Otherwise one of the two pairs is absent from $G_{S'}$; by the symmetry of j and k we may assume $\{j,v\} \notin G_{S'}$. Since the numeric pattern is contained in the symbolic one (Definition 2.12), this forces $(A/A_{S'})_{jv} = 0$, so Lemma 3.4 gives $(A/A_S)_{jk} = (A/A_{S'})_{jk}$, and the zero was therefore already present at level S'. We claim moreover that $\{j,k\} \in G_{S'}$. Indeed, if $\{j,k\} \notin G_{S'}$ then $\{j,k\}$ is a fill edge created by v, which by the definition of the elimination game requires both $\{j,v\}$ and $\{k,v\}$ to lie in $G_{S'}$, contrary to the case assumption. The inductive hypothesis applied at level S' now yields a vertex $v' \in S'$ with $\{j,v'\},\{k,v'\} \in G_{S'\setminus\{v'\}}$, and $v' \in S$, which completes the induction. □

Corollary 3.11. *If $(A/A_S)_{jk} = 0$ with $\{j,k\} \in G_S$, then $\{j,k\}$ lies in a triangle of the fill graph. In particular, only an edge that lies in a triangle of the fill graph can ever cancel.*

Proof. Let $v \in S$ be the vertex supplied by Lemma 3.10, so that $\{j,v\}$ and $\{k,v\}$ are edges of $G_{S\setminus\{v\}}$ while $\{j,k\}$ is an edge of G_S. Fix an ordering σ that enumerates $S \setminus \{v\}$ first, then v, then the survivors in any order. Every edge of $G_{S\setminus\{v\}}$ and of G_S is an edge of G_σ^+, because the fill graph accumulates the edges of G_A together with all fill created along the game and never deletes one (Definition 2.9). Hence $\{j,k\}$, $\{j,v\}$ and $\{k,v\}$ are all edges of G_σ^+, so $\{j,k,v\}$ is a triangle of G_σ^+ containing $\{j,k\}$. □

AI

Lemma 3.12 (Fresh fill is nonzero when created)**.** *Let S be a level, let $v \in V \setminus S$, and let $j, k \in V \setminus (S \cup \{v\})$ be distinct with $\{j, v\}, \{k, v\} \in G_S$ but $\{j, k\} \notin G_S$, so that $\{j, k\}$ is a fill edge created by v. Then $(A/A_S)_{jk} = 0$, and*

$$(A/A_{S\cup\{v\}})_{jk} = -\frac{(A/A_S)_{jv}(A/A_S)_{kv}}{(A/A_S)_{vv}}.$$

If in addition both legs are nonzero, that is $(A/A_S)_{jv} \neq 0$ and $(A/A_S)_{kv} \neq 0$, then this entry is nonzero: fresh fill is nonzero at the moment of its creation. It may of course cancel at a later level.

Proof. Since the numeric pattern is contained in the symbolic one (Definition 2.12), the hypothesis $\{j, k\} \notin G_S$ gives $(A/A_S)_{jk} = 0$. Pivoting at v and using Definition 2.6,

$$(A/A_{S\cup\{v\}})_{jk} = (A/A_S)_{jk} - \frac{(A/A_S)_{jv}(A/A_S)_{kv}}{(A/A_S)_{vv}} = -\frac{(A/A_S)_{jv}(A/A_S)_{kv}}{(A/A_S)_{vv}},$$

which is the displayed formula. If both legs are nonzero then the numerator is nonzero, and $(A/A_S)_{vv} > 0$ because A/A_S is positive definite, so the quotient is nonzero. □

Corollary 3.13 (Closure form of fresh fill)**.** *With S, v, j, k as in Lemma 3.12, if the slot $(S \cup \{v\}, j, k)$ is active then at least one of the slots (S, j, v) and (S, k, v) is active.*

Proof. By the displayed formula of Lemma 3.12, the entry $(A/A_{S\cup\{v\}})_{jk}$ vanishes only if the product $(A/A_S)_{jv}(A/A_S)_{kv}$ vanishes, and hence only if one of its factors does. □

Lemma 3.14 (Nesting Lemma)**.** *Let S be a level and let $i, j, k \in V \setminus S$ be distinct. If the slots (S, j, k) and $(S\cup\{i\}, j, k)$ are both active, then at least one of the slots (S, j, i) and (S, k, i) is active.*

Proof. By Definition 2.6,

$$(A/A_{S\cup\{i\}})_{jk} = (A/A_S)_{jk} - \frac{(A/A_S)_{ji}(A/A_S)_{ki}}{(A/A_S)_{ii}}.$$

The left-hand side is zero because $(S\cup\{i\}, j, k)$ is active, and the first term on the right is zero because (S, j, k) is active. Hence $(A/A_S)_{ji}(A/A_S)_{ki}/(A/A_S)_{ii} = 0$, and since the pivot value $(A/A_S)_{ii}$ is positive the product $(A/A_S)_{ji}(A/A_S)_{ki}$ vanishes. A product of real numbers vanishes only if one factor does, so $(A/A_S)_{ji} = 0$ or $(A/A_S)_{ki} = 0$, which says that (S, j, i) or (S, k, i) is active. □

Definition 3.15 (Arc digraph of a slot family)**.** Let S be a level, let $A \succ 0$, and let H be a family of slots at the levels $S \cup \{i\}$ for $i \in V \setminus S$, whose imposed relations $t_{jk}^{(S)} = t_{ji}^{(S)} + t_{ki}^{(S)}$ are those supplied by Lemma 2.33. The *arc digraph* $D(H)$ has as its vertices the pairs $\{j, k\}$ of survivors with $(A/A_S)_{jk} \neq 0$, and for each imposed relation of H it has the two arcs

$$\{j, k\} \to \{j, i\} \qquad \text{and} \qquad \{j, k\} \to \{k, i\},$$

included only when the target pair is itself nonzero in A/A_S; if a target pair is zero then $t^{(S)} = +\infty$ there and no arc is added. Note that $D(H)$ depends on A as well as on H, since which pairs are nonzero in A/A_S is a numeric matter and not determined by the pattern alone.

Lemma 3.16 (General Cycle Lemma)**.** *Fix a level S and let H be a family of slots at levels $S \cup \{i\}$, $i \in V \setminus S$, whose imposed relations are $t_{jk}^{(S)} = t_{ji}^{(S)} + t_{ki}^{(S)}$ in the sense of Lemma 2.33. If the associated arc digraph $D(H)$ of Definition 3.15 contains a directed cycle, then H is infeasible: no positive solution $t^{(S)} > 0$ exists.*

Proof. Each imposed relation $t_{jk}^{(S)} = t_{ji}^{(S)} + t_{ki}^{(S)}$ has both summands strictly positive (Definition 2.32), so $t_{jk}^{(S)} > t_{ji}^{(S)}$ and $t_{jk}^{(S)} > t_{ki}^{(S)}$: every arc of $D(H)$ strictly decreases the value of $t^{(S)}$. A directed cycle

would therefore give $t_x^{(S)} > \cdots > t_x^{(S)}$, which is impossible. Hence no positive solution exists. □

The following is the four-vertex instance of Lemma 3.16, and it is the step invoked in the proof of the Pairing Lemma 5.5.

Lemma 3.17 (4-cycle Lemma)**.** *Let a, b, c, d be four distinct survivors at a level S. Then the four relations*

$$t_{ab}^{(S)} = t_{ac}^{(S)} + t_{bc}^{(S)}, \quad t_{bc}^{(S)} = t_{bd}^{(S)} + t_{cd}^{(S)}, \quad t_{cd}^{(S)} = t_{ca}^{(S)} + t_{da}^{(S)}, \quad t_{da}^{(S)} = t_{db}^{(S)} + t_{ab}^{(S)}$$

cannot hold simultaneously; that is, the corresponding family of four slots is infeasible.

Proof. The four relations contribute the arcs $\{a, b\} \to \{b, c\} \to \{c, d\} \to \{d, a\} \to \{a, b\}$ of Definition 3.15, a directed cycle, so Lemma 3.16 applies. Concretely, $t_{ab}^{(S)} > t_{bc}^{(S)} > t_{cd}^{(S)} > t_{da}^{(S)} > t_{ab}^{(S)}$, a contradiction. □

Lemma 3.18 (Monotone Lemma)**.** *Fix a pattern G. If H is unrealizable for G, then every superset $H' \supseteq H$ is unrealizable for G. Equivalently, in contrapositive form: if H' is realizable for G then so is H.*

Proof. We prove the contrapositive. Suppose H' is realizable for G, and let $A \succ 0$ with $G_A = G$ be a matrix witnessing this, so that every slot of H' is active for A. Since $H \subseteq H'$, every slot of H is in particular a slot of H' and is therefore active for A as well. The same matrix A thus witnesses that H is realizable for G, the pattern being the same on both sides. Hence H' realizable for G implies H realizable for G, which is what was required. □

4. Positive results

Theorem 4.1 (Forests, strong form)**.** *If G_A is a forest, then every ordering of V is clean for A.*

Proof. For $j, k \notin S$, the Schur complement has the standard almost-principal-minor form

$$(A/A_S)_{jk} = \frac{\det A_{S\cup\{j\},\, S\cup\{k\}}}{\det A_{S,S}},$$

where $A_{I,J}$ denotes the submatrix with rows I and columns J; $A_{S,S}$ is principal. In particular the expression is manifestly independent of the elimination order on S.

Assume G_A is a forest, and fix $j, k \notin S$ with $\{j, k\} \in G_S$ (symbolic). Any two vertices of G_A are joined by *at most* one simple path: none if they lie in different components, and exactly one otherwise. By Proposition 2.10 the hypothesis $\{j, k\} \in G_S$ supplies such a path, so the j–k path exists and is unique; we write $\mathrm{path}(j, k)$ for its edge set and I for its interior, and $I \subseteq S$. A term of $\det A_{S\cup\{j\},S\cup\{k\}}$ is $\pm\prod_u A_{u,\pi(u)}$ for a bijection $\pi : S \cup \{j\} \to S \cup \{k\}$, and it is nonzero only if $\{u, \pi(u)\} \in E(G_A)$ or $u = \pi(u)$ for every u. Regard π as a partial permutation of V: it is defined on $S \cup \{j\}$ and takes values in $S \cup \{k\}$, so the two indices that fail to match are j (a source, not in the codomain) and k (a sink, not in the domain). Decomposing π accordingly, it consists of one directed path from j to k through vertices of S, together with fixed points and cycles on the remaining indices of S. The path contributes $\prod_e A_e$ over the edges of a j–k walk in G_A with interior in S; in a forest the only such walk without repeated vertices is $\mathrm{path}(j, k)$. The fixed points and cycles involve only indices of $S \setminus I$ and are matched among themselves, and their total contribution is precisely $\det A_{S\setminus I,S\setminus I}$. Note that this includes 2-cycles $A_{uv}A_{vu} = A_{uv}^2$ along genuine forest edges: such terms are legitimate and are exactly what the determinant $\det A_{S\setminus I,S\setminus I}$ collects, which is why

no claim is made that off-path indices cannot be moved. Hence

$$\det A_{S\cup\{j\},S\cup\{k\}} = \pm\Big(\prod_{e\in\mathrm{path}(j,k)} A_e\Big)\,\det A_{S\setminus I,S\setminus I},$$

and therefore

$$(A/A_S)_{jk} = \pm\frac{\big(\prod_{e\in\mathrm{path}(j,k)} A_e\big)\,\det A_{S\setminus I}}{\det A_S}.$$

This is the Jones–West path formula [JW05], see Lemma 1 there for the precise expansion; both $\det A_{S\setminus I}$ and $\det A_S$ are principal minors of the SPD matrix A, hence >0, and every $A_e \neq 0$ by the forest pattern. Thus $(A/A_S)_{jk} \neq 0$. Vertices of S off the path are accounted for automatically via $\det A_{S\setminus I}$, which collects all their internal matchings, and they only affect pivots, never the numerator product $\prod_{e\in\mathrm{path}(j,k)} A_e$, which is manifestly order-independent; Figure 2 illustrates the expansion on a small forest. No accidental zero ever occurs, irrespective of ordering [MR11, JW05]. □

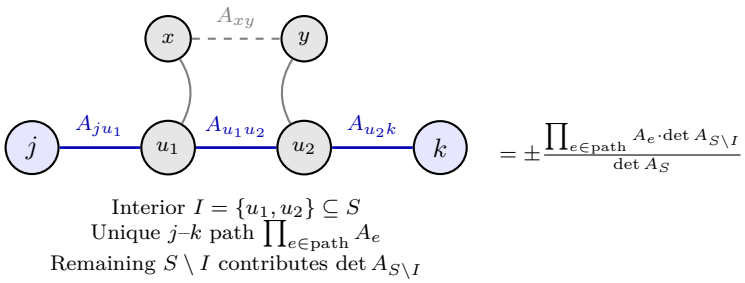


Figure 2: Forest path formula: j–k with interior S, $I \subseteq S$. Unique path in forest gives product $\prod A_e$. Every nonzero term routes j to k along that path, remaining indices permute among $S \setminus I$, contributing $\det A_{S\setminus I}$ collecting off-path matchings A_{xy}^2. Vertices of S off path accounted via that determinant, only pivots affected.

Remark 4.2. In the statement of the theorem, A is not assumed to be irreducible. If A is irreducible, G_A is a forest if and only if G_A is a tree. Malouche and Rajaratnam's covariance-faithfulness theorem is the tree case; the forest statement follows block-wise. The corresponding statement for concentration graphs, rather than covariance graphs, is due to Becker, Geiger and Meek [BGM05].

Definition 4.3 (2-path strip)**.** For $k \geq 3$, the 2-path strip on vertices $v_1, \ldots, v_k$ is the graph P whose edge set is exactly

$$E(P) = \{\{v_i, v_{i+1}\} : 1 \leq i \leq k-1\} \cup \{\{v_i, v_{i+2}\} : 1 \leq i \leq k-2\},$$

the first family called *boundary edges* and the second *diagonal edges*; its triangles are $T_i = \{v_i, v_{i+1}, v_{i+2}\}$ for $1 \leq i \leq k-2$. We say G_A is a 2-path strip if $G_A = P$ for some such labelling; in particular all listed edges are original pattern edges, not fill.

Definition 4.4 (Strip state)**.** A *strip state* is a pair (S, P) where $S \subseteq V$ is a level (Remark 2.5) and P is the graph that G_S induces on the survivors $V \setminus S$, assumed to be a 2-path strip on $k = |V \setminus S|$ vertices. Note that P is taken in the symbolic graph G_S, not in G_A: after a SECOND move the strip uses a fill edge. We write its left end as $s_1 = v_1, s_2 = v_2, s_3 = v_3, s_4 = v_4$; this names four particular survivors and carries no assumption that $k = 4$ (the right end is symmetric, and we treat only the left). For a 2-path strip this P is itself a 2-path strip, except after a SECOND move it uses the fill edge (s_1, s_4), which is not an edge of G_A but is an edge of G_S by Lemma 3.12. The

AI

state is

- *State 0* if every boundary and diagonal edge of P (including the fill edge (s_1, s_4) when present) is nonzero in A/A_S;
- *State 1* if exactly one pair is zero in A/A_S, namely (s_2, s_3), and every other boundary and diagonal edge of P is nonzero in A/A_S.

Definition 4.5 (Moves on a strip state)**.** Let (S, P) be a strip state with $k \geq 4$ survivors. Both moves below are pivots in the sense of Definition 2.6, and both are the named moves forward-declared in Definition 2.25:

- *END* pivots s_1. Since $N(s_1) = \{s_2, s_3\}$ is an edge of P, *no fill is created*, and *the only pair updated is* (s_2, s_3).
- *SECOND* pivots s_2. Since $N(s_2) = \{s_1, s_3, s_4\}$, *exactly one fill edge* (s_1, s_4) *is created*; the pairs updated are (s_1, s_3), (s_1, s_4), (s_3, s_4), and the survivors $V \setminus (S \cup \{s_2\})$ again carry a 2-path strip, namely $v_1, v_3, v_4, \ldots, v_k$ in that order: the old diagonal v_1v_3 becomes a boundary edge, the new fill edge v_1v_4 becomes a diagonal, and no other pair is affected. Its left end is v_1, v_3, v_4, together with v_5 when $k \geq 5$; for $k = 4$ the resulting strip is the triangle $\{v_1, v_3, v_4\}$, which is the base case of the induction.

Remark 4.6 (PEO-insufficiency witness)**.** There exists a 2-path of 5 triangles where 23 of 400 adversarial SPD realizations have zero clean simplicial vertices, so a pure PEO strategy that never creates fill fails. Hence the induction must sometimes use SECOND to create fill. That sampling run is not included in the ancillary material; the script `peo_insufficiency.py` (Appendix E) instead checks one such realization in exact arithmetic: neither simplicial vertex is a clean pivot, all 96 perfect elimination orderings of the pattern are unclean, and 3228 of the 5040 orderings, each creating fill, are clean, as Theorem 4.8(i) guarantees for strips.

Definition 4.7. Let (S, P) be a strip state. A continuation of (S, P) is an ordering $\tau = (p_1, \ldots, p_k)$ of the survivors $V \setminus S$, and we write $S_0 = S, S_i = S \cup \{p_1, \ldots, p_i\}$. The continuation τ is clean if for every $i = 1, \ldots, k$ and every survivor $x \in V \setminus S_i$ with $\{p_i, x\} \in G_{S_{i-1}}$, we have $(A/A_{S_{i-1}})_{p_i x} \neq 0$. We say (S, P) admits a clean continuation if some continuation of it is clean.

Theorem 4.8 (Strips, fans, one-triangle, edge-disjoint triangles)**.** *Conjecture 2.27 holds, and hence (Definition 2.25) an adaptive clean ordering exists, for (i) 2-path strips and dominating-vertex fans, (ii) chordal graphs with exactly one triangle, and (iii) chordal graphs with $\omega \leq 3$ and pairwise edge-disjoint triangles. For none of these classes does a fixed ordering suffice (Remark 4.9).*

Proof of (i), strips and fans. We prove by induction on $k = |V \setminus S|$ that a strip state (S, P) admits a clean continuation and that States 0 and 1 are preserved. For $k = 3$ there are two cases, according to the state. In State 0 all three pairs of the triangle $\{s_1, s_2, s_3\}$ are nonzero, so Lemma 3.2 applies: at most one apex route cancels, and at least two of the three vertices zero no pair of the triangle. Pivot such a clean apex; the pivot is clean because its two entries among the survivors are nonzero, and the surviving pair is nonzero (otherwise it would lie in the cancellation graph of that apex), so the final 2×2 Schur complement is clean. In State 1 the pair (s_2, s_3) is zero while (s_1, s_2) and (s_1, s_3) are nonzero, so Lemma 3.2 does not apply and the argument just given is unavailable. Pivot s_1 instead: it is clean because both of its entries among the survivors are nonzero, and it restores (s_2, s_3) by the computation displayed in the State 1 case below, leaving a 2×2 Schur complement whose off-diagonal entry is nonzero. Both cases are needed, because a State 1 strip on three vertices does arise: a SECOND move at $k = 4$ can leave the exceptional zero at the new (s_2, s_3).

Inductive step: let (S, P) be a strip state with left end $s_1 = v_1, s_2 = v_2, s_3 = v_3, s_4 = v_4$ (right

AI

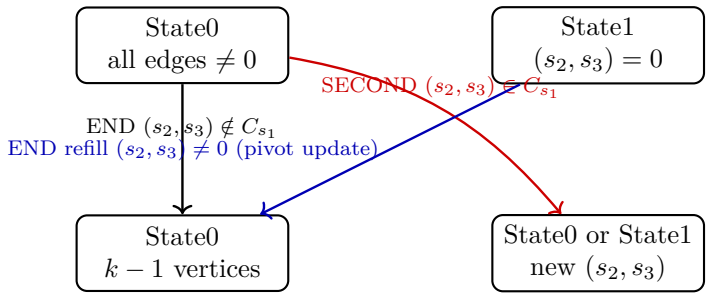
end symmetric), $k \geq 4$. Distinguish:

- *State 0 and* $(s_2, s_3) \notin C_{s_1}(S)$*:* END at s_1. Since $N(s_1) = \{s_2, s_3\}$ is an edge, no fill is created and the only pair updated is (s_2, s_3), which is nonzero by hypothesis $(s_2, s_3) \notin C_{s_1}(S)$. The pivot reads $(s_1, s_2), (s_1, s_3)$, both nonzero in State 0, hence clean. Result is State 0 on $k - 1$ vertices.
- *State 0 and* $(s_2, s_3) \in C_{s_1}(S)$*:* SECOND at s_2. Note (s_2, s_3) is nonzero at the time s_2 is pivoted, because s_1 has not been eliminated yet — this subtle point ensures the pivot itself is clean. The pairs updated are (s_1, s_3), (s_1, s_4) (fill), (s_3, s_4). Lemma 3.12 gives $(s_1, s_4) = -(A/A_S)_{s_2,s_1}(A/A_S)_{s_2,s_4}/(A/A_S)_{s_2s_2} \neq 0$. Since s_1 zeroes (s_2, s_3) by assumption, Lemma 3.2 in triangle $\{s_1, s_2, s_3\}$ implies s_2 cannot zero (s_1, s_3); hence $(s_1, s_3) \neq 0$ after s_2. Only (s_3, s_4) may become zero, which is exactly the new (s_2, s_3) in the remaining strip with vertices $s_1, s_3, s_4, s_5, \ldots$. Thus we move to State 0 if (s_3, s_4) stays nonzero, otherwise to State 1 with zero at new (s_2, s_3). In either case the pivot s_2 is clean because $(s_2, s_1), (s_2, s_3), (s_2, s_4)$ are all nonzero in State 0 before s_1 is gone.
- *State 1:* END at s_1. The exceptional zero is at (s_2, s_3), not at (s_1, s_2). Removing s_1 does *not* remove that pair; if we pivoted s_2 next we would commit a zero to L and be dirty. Instead END refills it:

$$(A/A_{S\cup\{s_1\}})_{s_2s_3} = 0 - \frac{(A/A_S)_{s_1s_2}(A/A_S)_{s_1s_3}}{(A/A_S)_{s_1s_1}} \neq 0,$$

  because both legs are nonzero in State 1 by definition and $(A/A_S)_{s_1s_1} > 0$ by positive definiteness. This is the pivot update of Definition 2.6 applied to a pair that is already zero, the same computation that appears in Lemma 3.5 and in Lemma 3.12. Note that Lemma 3.5 is not being invoked: s_1 need not be universal in A/A_S, since a strip with $k \geq 4$ survivors has survivors not adjacent to s_1. What makes the two legs nonzero here is the strip state, not domination of the survivors by s_1. It is the refill phenomenon of Section 1.3 doing constructive work. The pivot s_1 reads $(s_1, s_2), (s_1, s_3)$, both nonzero, hence clean. After END the state is State 0 on $k - 1$ vertices.

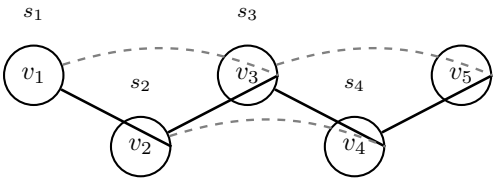
The table is exhaustive, State 0 and State 1 are preserved, and k drops by one per move, terminating at the $k = 3$ base case. Figure 4 shows the two moves on a strip of five triangles, and Figure 3 collects the three cases above as a transition diagram.



State	Condition	Move → Result
State0	$(s_2, s_3) \notin C_{s_1}(S)$	END → State0 $k-1$
State0	$(s_2, s_3) \in C_{s_1}(S)$	SECOND → State0 or State1 new (s_2, s_3)
State1	—	END → State0 $k-1$ refill

Figure 3: Strip state machine: $k \geq 4$ left end s_1, s_2, s_3, s_4. END simplicial no fill only pair updated (s_2, s_3); SECOND creates fill (s_1, s_4) updates $(s_1, s_3), (s_1, s_4), (s_3, s_4)$, $V \setminus (S \cup \{s_2\})$ again strip with s_1, s_3, s_4, s_5 as new $s_1 \dots s_4$. Table exhaustive. Subtle point (s_2, s_3) nonzero at time s_2 pivoted because s_1 not yet eliminated. The State 1 → State 0 END transition is the pivot update of Definition 2.6, not an application of Lemma 3.5 (Fact 1): s_1 need not be universal.

(a) State0 before, s_1, s_2, s_3, s_4



(b) END vs SECOND

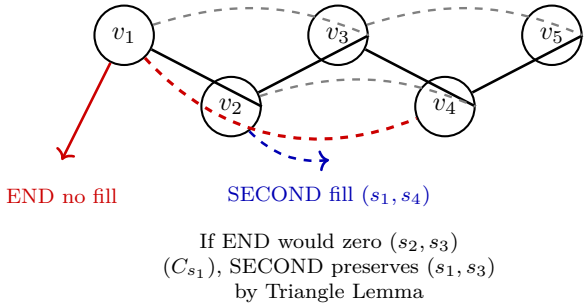


Figure 4: 2-path strip 5 triangles $v_1 \dots v_7$, $T_i = (v_i, v_{i+1}, v_{i+2})$. Boundary solid thick 1pt, diagonal dashed 0.9pt. (a) State0 before with s_1, s_2, s_3, s_4. (b) END simplicial no fill vs SECOND one fill (s_1, s_4) curved thick dashed red leaves 2-path. If END would zero (s_2, s_3) (C_{s_1}), SECOND preserves (s_1, s_3) by Triangle Lemma. Larger canvas, State0/1 illustrated before/after.

Fans (dominating-vertex fan): G has a center c adjacent to all vertices $v_1, \dots, v_k$ of a path, plus boundary edges $v_i v_{i+1}$ (no diagonals initially). Then $N(v_1) = \{c, v_2\}$ and $N(v_2) = \{c, v_1, v_3\}$ in G_A, because the strip diagonals $v_i v_{i+2}$ are not present, and c–v_2 is an edge, so v_1 is simplicial. In general v_1 is simplicial with clique $\{c, v_2\}$, so END at v_1 creates no fill and updates only (c, v_2). Eliminating v_1 leaves a fan on $k-1$ path vertices. Eliminating v_2 next has $N(v_2) = \{c, v_1, v_3\}$ (or $\{c, v_3\}$ if v_1 gone) and creates exactly one fill edge (v_1, v_3) when v_1 still present, which becomes a diagonal that will be present as fill in subsequent states. Thus diagonals do appear as fill once peeling starts, which is why the fan argument is not verbatim the strip's but is analogous.

We maintain fan states: (S, P) where P is G_S on $V \setminus S$, which is a fan possibly with one extra diagonal (s_1, s_3) from a previous SECOND. Invariant: all edges cv_i and all path edges $v_i v_{i+1}$ (and any created diagonal) are nonzero, except possibly one exceptional pair. Define State 0: all cv_i

and $v_i v_{i+1}$ (and existing diagonal) nonzero; State 1: exactly one pair zero, namely (c, s_2), others nonzero. Then:

- END at $s_1 = v_1$: $N(s_1) = \{c, s_2\}$ is an edge, so no fill, only pair updated is (c, s_2). If State 0 and $(c, s_2) \notin C_{s_1}(S)$, we stay State 0 on $k-1$ vertices; if $(c, s_2) \in C_{s_1}(S)$, we must pivot s_2 instead (next case). Pivot s_1 is clean because $(s_1, c), (s_1, s_2)$ nonzero.
- SECOND at s_2: $N(s_2) = \{c, s_1, s_3\}$ (up to s_4 if diagonal present), creates fill (s_1, s_3). Pairs updated are $(c, s_1), (c, s_3), (s_1, s_3)$. The fill edge (s_1, s_3) is nonzero at the moment of its creation by Lemma 3.12, its two legs (s_2, s_1) and (s_2, s_3) being nonzero. If s_1 zeroed (c, s_2), Lemma 3.2 in triangle $\{c, s_1, s_2\}$ gives s_2 cannot zero (c, s_1), so (c, s_1) stays nonzero. Only (c, s_3) may become the new exceptional (c, s_2) for the remaining fan. Pivot s_2 is clean because $(s_2, c), (s_2, s_1), (s_2, s_3)$ are nonzero at that time (s_1 not yet eliminated for the (c, s_2) check).
- State 1 (zero at (c, s_2)): END at s_1 refills it, since

$$(A/A_{S\cup\{s_1\}})_{c,s_2} = -\frac{(A/A_S)_{s_1,c}\,(A/A_S)_{s_1,s_2}}{(A/A_S)_{s_1,s_1}} \neq 0$$

  because both legs are nonzero in State 1 and $(A/A_S)_{s_1,s_1} > 0$. As in the strip case this is the pivot update of Definition 2.6 and not an application of Lemma 3.5: in a fan with at least three path vertices s_1 is not universal. Note that s_1 is adjacent to both c and s_2, which is what makes the pair (c, s_2) one of the pairs this pivot updates. So State 1 $\to$ State 0 on $k-1$ vertices.

The induction terminates when a single path vertex remains. That configuration is always in State 0: State 1 has its exceptional zero at (c, s_2) and therefore needs two path vertices, so it cannot occur with one. The unique surviving pair (c, v) is then nonzero and either pivot is clean. If c is eliminated last, there is no surviving pair and no off-diagonal L entry in its column, so the step is vacuously clean (not because $C_c = \emptyset$, which would be a non-sequitur: universality says the row is nonzero, not that c zeroes no pair). Induction on k gives a clean ordering. Thus fans admit an adaptive clean ordering.

□

Proof of (ii), one-triangle. First we prove the positive claim for all chordal graphs with exactly one triangle. Let G be chordal with exactly one triangle T. Every simplicial vertex $v \notin T$ has exactly one neighbour: if v had two neighbours x and y, they would be adjacent to each other because $N(v)$ is a clique, so $\{v, x, y\}$ would be a triangle, and it would be distinct from T because $v \notin T$, contradicting the hypothesis that T is the only triangle. Hence eliminating v creates no fill and updates no off-diagonal pair: $(A/A_S)_{jk}$ is unchanged for $j, k \neq v$, only the diagonal $(A/A_S)_{vv}$ disappears. Hence such a v cannot create a zero and can be peeled cleanly irrespective of numerics. Moreover, as long as G properly contains T there is a simplicial vertex outside T available to peel: a chordal graph with exactly one triangle consists of T with trees attached, since a cycle other than T would have a chord and so create a second triangle, and a leaf of such a tree is simplicial and outside T. Peeling these vertices one at a time therefore leaves exactly T, a triangle, where Lemma 3.2 supplies a clean apex (at most one of the three vertices is dirty, so at least two are clean). Thus an adaptive clean ordering exists for every SPD matrix with that pattern.

□

Proof of (iii), edge-disjoint triangles. Let G be chordal with $\omega \leq 3$ and pairwise edge-disjoint triangles. Use the bipartite block-cut tree (blocks vs cut vertices) whose leaves are always blocks.

Consider a leaf block B in this tree. If B is an edge $\{c, x\}$ where c is the cut vertex, then x

AI

is simplicial with $N(x) = \{c\}$; eliminating x creates no fill and updates no pair, so it is clean and leaves a graph in the same class.

If B is a triangle $\{c, a, b\}$ where c is the cut vertex (or B is isolated triangle when G itself is a triangle), then a, b are simplicial and $N(a) = \{c, b\}$, $N(b) = \{c, a\}$ are edges. By Lemma 3.2 at most one of the three vertices of B is dirty (zeroes a pair of B), so at least one of the two simplicial vertices a, b is clean — pigeonhole. Choose a clean apex among a, b, say a. Then a zeroes no pair of its triangle, its neighbourhood is exactly the opposite edge $\{c, b\}$ because $\omega \leq 3$ and edge-disjointness, so eliminating a creates no fill and zeroes no pair of B. The remaining graph is still chordal with $\omega \leq 3$ and edge-disjoint triangles, with $|V|$ reduced by one (the triangle block $\{c, a, b\}$ becomes the edge block $\{c, b\}$, so the number of blocks is unchanged). Induction on $|V|$ (equivalently $|E|$) closes. □

Remark 4.9. In the proof of (iii), the naive block tree that connects blocks directly fails for a star of three bridges sharing a central cut vertex: the three bridge-blocks are pairwise adjacent through the hub, so they do not form a tree; the bipartite incidence tree fixes this.

For none of the classes (i)–(iii) does a fixed ordering suffice, so the clean orderings of Theorem 4.8 must depend on the matrix (Definition 2.25). The simplest witness is the triangle K_3, which is a 2-path strip, a fan, a chordal graph with exactly one triangle, and a chordal graph with $\omega \leq 3$ whose triangles are (trivially) edge-disjoint. For a vertex a of K_3 with opposite pair $\{b, c\}$, let $T^{(a)}$ have unit diagonal, $T^{(a)}_{ab} = T^{(a)}_{ac} = 1/2$ and $T^{(a)}_{bc} = 1/4$. It is SPD, with leading principal minors $1, 3/4, 9/16$ in the order a, b, c, and $(T^{(a)}/T^{(a)}_{aa})_{bc} = 1/4 - (1/2)(1/2) = 0$. Given an ordering σ, let a be its first vertex. The pair $\{b, c\}$ is an edge, hence predicted nonzero, and it is committed at level $\{a\}$, where it vanishes for $T^{(a)}$. So no fixed σ is clean for all three matrices.

The phenomenon is not special to complete patterns. Take the pattern $E = \{03, 04, 14, 23, 34\}$, vertices $0, \ldots, 4$, edges as listed, unique triangle $\{0, 3, 4\}$, vertex 1 adjacent only to 4, vertex 2 adjacent only to 3. This graph is chordal, has exactly one triangle and no dominating vertex. We exhibit explicit 5×5 SPD matrices with that zero pattern, chosen according to the ordering, to show no fixed ordering works.

For $u = 0$ (first among $\{0, 3, 4\}$ is 0, cancel $(3, 4)$):

$$A^{(0)} = \begin{pmatrix} 3 & 0 & 0 & 1 & 1 \\ 0 & 3 & 0 & 0 & 1 \\ 0 & 0 & 3 & 1 & 0 \\ 1 & 0 & 1 & 3 & 1/3 \\ 1 & 1 & 0 & 1/3 & 3 \end{pmatrix},$$

Leading principal minors $3, 9, 27, 63, 147 > 0$ hence SPD, $(A^{(0)}/A^{(0)}_{00})_{34} = 1/3 - 1 \cdot 1/3 = 0$.

For $u = 3$ (cancel $(0, 4)$): need $A_{33}A_{04} = A_{03}A_{34}$. Choose $A_{33} = 2$, $A_{04} = 0.5$, $A_{03} = 1$, $A_{34} = 1$ gives $2 \cdot 0.5 = 1$:

$$A^{(3)} = \begin{pmatrix} 3 & 0 & 0 & 1 & 0.5 \\ 0 & 3 & 0 & 0 & 1 \\ 0 & 0 & 3 & 1 & 0 \\ 1 & 0 & 1 & 2 & 1 \\ 0.5 & 1 & 0 & 1 & 3 \end{pmatrix},$$

Leading minors $3, 9, 27, 36, 297/4 > 0$ hence SPD (not diagonally dominant, so we quote minors), $(A^{(3)}/A^{(3)}_{33})_{04} = 0.5 - 1 \cdot 1/2 = 0$.

For $u = 4$ (cancel $(0,3)$): need $A_{44}A_{03} = A_{04}A_{34}$. Choose $A_{44} = 2$, $A_{03} = 0.5$, $A_{04} = 1$, $A_{34} = 1$ gives $2 \cdot 0.5 = 1$:

$$A^{(4)} = \begin{pmatrix} 3 & 0 & 0 & 0.5 & 1 \\ 0 & 3 & 0 & 0 & 1 \\ 0 & 0 & 3 & 1 & 0 \\ 0.5 & 0 & 1 & 3 & 1 \\ 1 & 1 & 0 & 1 & 2 \end{pmatrix},$$

Leading minors $3, 9, 27, 279/4, 297/4 > 0$ hence SPD, $(A^{(4)}/A^{(4)}_{44})_{03} = 0.5 - 1 \cdot 1/2 = 0$.

Each matrix has exactly pattern E, exactly one triangle and no extra edges.

Now fix a permutation σ and let u be the first vertex of $\{0,3,4\}$ in σ. Pivoting u can zero the opposite pair of the triangle, but that pair is committed only when the earlier of its two ends is pivoted, so the zero must be present at the level S of the vertices preceding that end. Two points need care. First, the pendant vertices 1 and 2 may precede u, and pivoting a pendant changes the diagonal entry at its unique neighbour: eliminating 2 replaces A_{33} by $A_{33} - A^2_{23}/A_{22}$, and eliminating 1 replaces A_{44} by $A_{44} - A^2_{14}/A_{11}$, while no off-diagonal entry is affected, since a pendant has a single neighbour. Second, a pendant pivoted after its neighbour u is no longer pendant: eliminating u joins it to both ends of the zeroed pair, and pivoting it then updates that pair. The witness is therefore chosen by whether the pendant at u lies in S, as follows.

If $u = 0$, take $A^{(0)}$. No pendant is adjacent to 0, and the entries $A_{00}, A_{03}, A_{04}, A_{34}$ that enter the condition $A_{00}A_{34} = A_{03}A_{04}$ are untouched by pivoting 1 or 2; hence the zero at $(3,4)$ appears when 0 is pivoted, whichever pendants precede it. It persists until the pair is committed, since 1 and 2 remain pendant after 0 is eliminated and pivoting them changes only the diagonal entries at 4 and 3.

If $u = 3$, the zeroed pair is $\{0,4\}$, and S consists of 3, possibly 1 and possibly 2. Whether $1 \in S$ does not matter: while 4 survives, 1 is adjacent only to 4, so pivoting it changes only the diagonal entry at 4. Take $A^{(3)}$ as displayed when $2 \notin S$, that is, when 2 does not precede both 0 and 4; then $(A/A_S)_{04} = (A/A_{\{3\}})_{04} = 0$. Take $A^{(3)}$ with $A_{33} = 7/3$ in place of 2 when $2 \in S$. The modified matrix is SPD, with leading principal minors $3, 9, 27, 45, 195/2$, and the compensation is exact: $7/3 - A^2_{23}/A_{22} = 7/3 - 1/3 = 2$, so $(A/A_S)_{04} = (A/A_{\{2,3\}})_{04} = 0$. Symmetrically, if $u = 4$ the zeroed pair is $\{0,3\}$: take $A^{(4)}$ as displayed when 1 does not precede both 0 and 3, and with $A_{44} = 7/3$ in place of 2 when it does; the minors are then $3, 9, 27, 279/4, 195/2$ and $(A/A_{\{1,4\}})_{03} = 0$.

In every case the pair zeroed by u, an edge of E and hence predicted nonzero, is zero at the level where it is committed, so σ is not clean for that matrix. Hence no fixed σ is clean for every SPD matrix with pattern E, whereas the peeling argument above supplies an adaptive clean ordering for each individual matrix. Both adjustments are needed. For the displayed $A^{(3)}$ one computes $(A/A_{\{2,3\}})_{04} = -1/10 \neq 0$. So $\sigma = (2,3,0,1,4)$ escapes $A^{(3)}$ because pivoting 2 first lowers the diagonal at 3; and $\sigma = (3,2,0,1,4)$ escapes it although pivoting 3 first does create the zero at $(0,4)$, because pivoting 2 refills it before 0 commits the pair. In fact σ is clean for $A^{(3)}$. This is the refill phenomenon of Section 1.3, and it is why the witness is chosen by when the zeroed pair is committed rather than by the position of u. The script `fixed_ordering_witnesses.py` (Appendix F) checks the three matrices for K_3 against all 6 orderings and this choice of witness for E against all 120.

5. Even counterexamples and order four

AI

Theorem 5.1 (Conjecture 2 fails at order 4, A^*)**.** *Let*

$$A^* = \begin{pmatrix} 4 & -2 & -2 & -1 \\ -2 & 4 & 1 & 2 \\ -2 & 1 & 4 & 2 \\ -1 & 2 & 2 & 4 \end{pmatrix},$$

$\operatorname{spec}(A^*) = \{9, 3, 3, 1\}$*, SPD. Then* 0 *of* 24 *permutations have clean first Schur complement, while* 8 *of* 24 *have clean full Cholesky factor. Hence Conjecture 2.26 is false. However,* A^* *is not a counterexample to Conjecture 2.27: the refill phenomenon rescues eight of its orderings.*

Proof. Positive definiteness and the spectrum. The leading principal minors of A^* are 4, 12, 36 and 81, all positive, so $A^* \succ 0$. Its characteristic polynomial factors as

$$\det(xI - A^*) = x^4 - 16x^3 + 78x^2 - 144x + 81 = (x-9)(x-3)^2(x-1),$$

so $\operatorname{spec}(A^*) = \{9, 3, 3, 1\}$.

Every off-diagonal entry of A^* is nonzero, so $G_{A^*} = K_4$, and by Proposition 2.10 every pair of distinct vertices is a predicted-nonzero position at every level. Cleanliness therefore amounts to the statement that no committed entry vanishes, and no fill-path bookkeeping is required.

Stage one: every ordering fails. The first Schur complement depends only on the first pivot, so there are four matrices to inspect rather than 24. Pair the vertices as $\{0, 3\}$ and $\{1, 2\}$, write q for the partner of the first pivot p, and let $\{j, k\}$ be the remaining pair. In each of the four cases the first Schur complement has a single zero, located at (j, k). For $p = 0$, for instance,

$$A^*/A^*_{\{0\}} = \begin{pmatrix} 3 & 0 & 3/2 \\ 0 & 3 & 3/2 \\ 3/2 & 3/2 & 15/4 \end{pmatrix}$$

on the survivors $1, 2, 3$, whose $(1, 2)$ entry is $1 - (-2)(-2)/4 = 0$; the three remaining cases are the same computation after relabelling, with the zero at $(1, 2)$ for $p = 3$ and at $(0, 3)$ for $p = 1$ and $p = 2$. Since (j, k) is a predicted-nonzero position, each of the 24 orderings meets a cancellation in its first Schur complement, and Conjecture 2.26 fails for A^*.

Stage two: exactly eight orderings are clean. Fix the first pivot p and let q, j, k be as above. The only vanishing pair of $M = A^*/A^*_{\{p\}}$ is (j, k), so $M_{qj} \neq 0 \neq M_{qk}$ and q is universal in M in the sense of Definition 2.21. Lemma 3.5 applies to q and shows that pivoting it destroys the zero; for $p = 0$ and $q = 3$,

$$(M/M_{33})_{12} = 0 - \frac{(3/2)(3/2)}{15/4} = -\frac{3}{5} \neq 0.$$

The pivot at q is itself clean, because its committed entries M_{qj} and M_{qk} are nonzero, and after it the two remaining survivors are joined by a nonzero entry, so the whole factor is clean. Conversely, if the second pivot is j or k instead of q, then its committed column contains the entry (j, k), which is zero, and the ordering is dirty. An ordering is therefore clean precisely when its second pivot is the partner of its first, which leaves $4 \cdot 1 \cdot 2 = 8$ clean orderings, namely

$$(0,3,1,2),\ (0,3,2,1),\ (1,2,0,3),\ (1,2,3,0),\ (2,1,0,3),\ (2,1,3,0),\ (3,0,1,2),\ (3,0,2,1).$$

Every computation above is exact rational arithmetic. Hence A^* refutes Conjecture 2.26 while

AI satisfying Conjecture 2.27, and the mechanism separating the two is refill. □

Human **Theorem 5.2** (Conjecture 1 holds for $n \leq 4$)**.** *Conjecture 2.27 holds for $n \leq 4$.*

Proof. The argument is elementary and uses no computation: only the pivot update of Definition 2.6, the cancellation graph of Definition 2.19, and Lemmas 3.2, 3.3, 3.9 and 3.12. Since A is irreducible, G_A is connected, so it suffices to treat the connected patterns on at most four vertices. We use two observations repeatedly.

(O1) Pivoting a vertex of surviving degree one is clean and creates no fill. Let p be a survivor at level S whose only neighbor in G_S is w. The pairs updated by the pivot at p are those $\{j,k\}$ with $(A/A_S)_{jp} \neq 0 \neq (A/A_S)_{kp}$, which forces $j = k = w$; there are none. So no entry among the survivors changes, no fill is created, and the column committed to L consists of the single entry $(A/A_S)_{pw} \neq 0$.

(O2) Three survivors forming a triangle admit a clean pivot. Let the survivors be $\{j,k,u\}$ with all three pairs nonzero. By Lemma 3.2 at most one of the three apex routes cancels, so some vertex, say u, zeroes no pair of the triangle. Pivoting u commits (u,j) and (u,k), both nonzero, and leaves $(j,k) \neq 0$ because $(j,k) \notin C_u(S)$; the final 2×2 Schur complement is therefore clean as well.

For $n = 1$ there is nothing to prove, and for $n = 2$ connectedness gives $A_{01} \neq 0$, so the unique off-diagonal entry of L is nonzero. The connected patterns on three vertices are the path and the triangle: the path is handled by peeling a leaf with (O1) and then applying the case $n = 2$, and the triangle is (O2). Up to isomorphism there are six connected patterns on four vertices, which we treat in turn.

(a) The path P_4 and the star $K_{1,3}$. Both are trees, so each has a leaf; peel it by (O1) and induct, the remaining graph being a tree on three vertices.

(b) The paw, a triangle $\{0,1,2\}$ with a pendant 3 attached to 0. Pivot the pendant 3 first. By (O1) the step is clean and alters no entry among $\{0,1,2\}$, so the survivors still form a triangle with all three pairs nonzero, and (O2) finishes.

(c) $K_4 - e$, with vertices $\{1,2,3,4\}$ and non-edge $\{1,4\}$, so that the triangles $\{1,2,3\}$ and $\{2,3,4\}$ share the edge $\{2,3\}$. The vertex 1 is simplicial, since $N(1) = \{2,3\}$ is an edge; pivoting it creates no fill and commits the nonzero entries $(1,2)$ and $(1,3)$, so the step is clean whatever the numerics. If $(2,3) \notin C_1$, the survivors $\{2,3,4\}$ form a triangle with all pairs nonzero and (O2) finishes. If instead $(2,3) \in C_1$, then Lemma 3.9 applies to the two triangles sharing $\{2,3\}$: as $\omega(G_A) = 3$ we have $A_{14} = 0$, so pivoting 1 leaves $(2,4)$ and $(3,4)$ at their original nonzero values while $(2,3)$ becomes zero. Pivot 4 next. Its committed entries $(4,2)$ and $(4,3)$ are nonzero, so the step is clean, and the update restores

$$(A/A_{\{1,4\}})_{23} = -\frac{(A/A_{\{1\}})_{24}(A/A_{\{1\}})_{34}}{(A/A_{\{1\}})_{44}} \neq 0,$$

so the final 2×2 Schur complement is clean.

(d) The cycle C_4, say $0-1-2-3-0$. Pivot 0. Its neighbourhood $\{1,3\}$ is not an edge, so the pivot creates the fill pair $(1,3)$, which is nonzero by Lemma 3.12 because both legs $(0,1)$ and $(0,3)$ are original nonzero entries; those two legs are also exactly the committed column, so the step is clean. The survivors $\{1,2,3\}$ now form a triangle with all three pairs nonzero, and (O2) finishes.

(e) The complete graph K_4. Let (p,q) maximise $t_{pq}^{(\emptyset)}$ over all pairs, and let $\{k,l\}$ be the two remaining vertices. By Lemma 3.3 no pair of C_p contains q, so neither (q,k) nor (q,l) is cancelled by p; and since A has no zero off-diagonal entry, the zeros present after pivoting p are by Definition 2.19

Human

exactly the pairs of C_p. Hence at most one pair among the survivors is zero, namely possibly (k,l). Pivoting p is clean, because its committed column $(p,q),(p,k),(p,l)$ consists of original nonzero entries. If $(k,l) \notin C_p$, the survivors form a triangle with all pairs nonzero and (O2) finishes. If $(k,l) \in C_p$, pivot q next: its committed entries (q,k) and (q,l) are nonzero by the preceding sentence, so the step is clean, and the update restores

$$(A/A_{\{p,q\}})_{kl} = 0 - \frac{(A/A_{\{p\}})_{qk}(A/A_{\{p\}})_{ql}}{(A/A_{\{p\}})_{qq}} \neq 0,$$

leaving a clean final 2×2 Schur complement.

In every case we have produced an ordering all of whose committed entries are nonzero, which is cleanliness in the sense of Definition 2.24. □

Remark 5.3 (The argument is special to order four)**.** Case (e) is genuinely an order-four argument. With three survivors the only pair avoiding q is $\{k,l\}$, so C_p can leave at most one zero behind. From five vertices onwards C_p may leave several, the case analysis branches, and an exhaustive search over blocking families would be needed in its place; we do not carry one out here.

AI

5.1. Pairing Lemma and forced magnitudes.

Definition 5.4. A *permutation* π of a finite set W is an *involution* if $\pi^2 = \mathrm{id}$, and it is *fixed-point-free* if $\pi(x) \neq x$ for every $x \in W$. Fixed-point-free involutions of W correspond bijectively to perfect matchings of W, the matching pairing each x with $\pi(x)$; in particular such an involution exists if and only if $|W|$ is even.

Lemma 5.5 (Pairing Lemma)**.** *Fix a level S and a set W of four surviving vertices. There are $4\binom{3}{2} = 12$ candidate slots $(u,\{j,k\})$ with $u \in W$ and $\{j,k\} \subseteq W \setminus \{u\}$, one for each choice of apex together with the pair it cancels, and $\binom{4}{2} = 6$ variables $t^{(S)}_{jk}$, one for each pair inside W. Then:*

1. *every feasible set of these slots has at most four elements;*
2. *a feasible set of four slots that uses all four vertices of W as apexes is a fixed-point-free involution of W: the four vertices split into two pairs, and each vertex cancels the pair opposite its own;*
3. *all three such configurations, one for each of the three partitions of W into two pairs, are feasible.*

Proof. Throughout, a slot (u,p) with $p = \{j,k\}$ imposes the relation $t^{(S)}_{jk} = t^{(S)}_{uj} + t^{(S)}_{uk}$ of Lemma 2.33, and feasibility of a set of slots means that the resulting relations admit a solution with all six variables strictly positive.

(1) Size at most four. Map a slot (u,p) to the ordered pair (u,d), where $d = W \setminus (\{u\} \cup p)$ is the unique vertex of W lying neither in p nor at the apex. Suppose two slots of a feasible set, say (u,p) and (v,q), have the same second coordinate d. Since $u \neq v$ and neither u nor v equals d, the pair $p = W \setminus \{u,d\}$ contains v and the pair $q = W \setminus \{v,d\}$ contains u. Writing w for the fourth vertex of W, this gives $p = \{v,w\}$ and $q = \{u,w\}$, so the two slots impose $t^{(S)}_{vw} = t^{(S)}_{uv} + t^{(S)}_{uw}$ and $t^{(S)}_{uw} = t^{(S)}_{uv} + t^{(S)}_{vw}$. Adding them gives $t^{(S)}_{uv} = 0$, contradicting strict positivity; this is the additive form of Lemma 3.1. Hence the second coordinates of a feasible set are distinct, the set injects into W, and its size is at most $|W| = 4$.

(2) Four slots using all four apexes. Let H be such a set. By the injectivity just proved its second coordinates are four distinct vertices, so they exhaust W; by hypothesis its first coordinates exhaust W as well. Writing $\pi(u)$ for the second coordinate of the slot with apex u, we therefore

AI

obtain a permutation π of W, and π has no fixed point because $d \neq u$ by construction. A fixed-point-free permutation of a four-element set is either a product of two disjoint transpositions, of which there are three, or a four-cycle, of which there are six.

Suppose π is a four-cycle. Relabelling W as $\{0,1,2,3\}$ we may assume $\pi = (0\,1\,2\,3)$, so that each vertex u cancels the pair $W \setminus \{u, \pi(u)\}$, and the four imposed relations are

$$t_{23}^{(S)} = t_{02}^{(S)} + t_{03}^{(S)}, \qquad t_{03}^{(S)} = t_{01}^{(S)} + t_{13}^{(S)}, \qquad t_{01}^{(S)} = t_{02}^{(S)} + t_{12}^{(S)}, \qquad t_{12}^{(S)} = t_{13}^{(S)} + t_{23}^{(S)}.$$

Every variable is strictly positive, so each relation makes its left side strictly larger than either summand on its right. Reading the four relations in the order above gives

$$t_{12}^{(S)} > t_{23}^{(S)} > t_{03}^{(S)} > t_{01}^{(S)} > t_{12}^{(S)},$$

which is impossible. This is the four-cycle instance of the General Cycle Lemma 3.16, recorded as Lemma 3.17. Hence π is a product of two disjoint transpositions, which is precisely the assertion that W splits into two pairs with each vertex cancelling the opposite pair.

(3) The three involutions are feasible. By symmetry it suffices to treat the pairing $\{0,1\},\{2,3\}$, whose four slots impose

$$t_{23}^{(S)} = t_{02}^{(S)} + t_{03}^{(S)}, \qquad t_{23}^{(S)} = t_{12}^{(S)} + t_{13}^{(S)}, \qquad t_{01}^{(S)} = t_{02}^{(S)} + t_{12}^{(S)}, \qquad t_{01}^{(S)} = t_{03}^{(S)} + t_{13}^{(S)}.$$

Given any $\alpha, \beta > 0$, set $t_{02}^{(S)} = t_{13}^{(S)} = \alpha$, $t_{03}^{(S)} = t_{12}^{(S)} = \beta$ and $t_{01}^{(S)} = t_{23}^{(S)} = \alpha + \beta$. All six values are positive and all four relations hold, so the configuration is feasible. (Corollary 5.6 shows that these are the only solutions.)

Finally, the hypothesis in (2) that all four vertices occur as apexes cannot be dropped: a feasible set of four slots may repeat an apex. For instance the four slots $(0,\{1,2\})$, $(0,\{1,3\})$, $(2,\{0,3\})$, $(2,\{1,3\})$ use only 0 and 2 as apexes, and they are feasible, since for arbitrary $t_{01}^{(S)}, t_{02}^{(S)}, t_{23}^{(S)} > 0$ the assignments $t_{03}^{(S)} = t_{02}^{(S)} + t_{23}^{(S)}$, $t_{12}^{(S)} = t_{01}^{(S)} + t_{02}^{(S)}$ and $t_{13}^{(S)} = t_{01}^{(S)} + t_{02}^{(S)} + t_{23}^{(S)}$ satisfy all four relations. Such a configuration is of no use in blocking: a level at which some vertex never serves as a pivot leaves that vertex available, so it cannot spoil every ordering. This is why Theorem 5.8 needs the configurations of (2) rather than merely maximal feasible ones. □

Corollary 5.6 (Magnitudes forced)**.** *Let S be a level, let W be a set of four survivors, and let H be one of the three feasible configurations of four slots using all four vertices of W as apexes, supplied by Lemma 5.5. Relabel W as $\{0,1,2,3\}$ so that the two pairs of the associated involution are $\{0,1\}$ and $\{2,3\}$. Then the relations imposed by H determine the six log-magnitudes inside W up to two positive parameters: with $\alpha := t_{02}^{(S)}$ and $\beta := t_{03}^{(S)}$,*

$$t_{02}^{(S)} = t_{13}^{(S)} = \alpha, \qquad t_{03}^{(S)} = t_{12}^{(S)} = \beta, \qquad t_{01}^{(S)} = t_{23}^{(S)} = \alpha + \beta,$$

and conversely every pair $\alpha, \beta > 0$ arises. Equivalently, setting $A = e^{-\alpha}$ and $B = e^{-\beta}$, the two within-pair correlations have magnitude AB and the four cross-pair correlations form the block $\begin{pmatrix} A & B \\ B & A \end{pmatrix}$.

Proof. We first record why this is a consequence of Lemma 5.5 and not an independent computation. That lemma classifies the configurations available at a four-vertex level: a feasible set of slots that has the maximum size 4 and uses all four vertices of W as pivots is a fixed-point-free involution of W, so it partitions W into two pairs and each vertex cancels the pair opposite its own. There are exactly three such configurations, one for each partition of a four-element set into two pairs, and

the symmetric group on W permutes those three partitions transitively; relabelling W therefore allows us to assume that the pairs are $\{0,1\}$ and $\{2,3\}$. In other words, Lemma 5.5 leaves no freedom in *which* relations are imposed, and the only thing left to do is to solve them.

The configuration consists of the four slots in which 0 and 1 each cancel $\{2,3\}$ and 2 and 3 each cancel $\{0,1\}$. By Lemma 2.33 every cancellation is additive in the log-magnitudes, so these four slots impose exactly

$$t_{23}^{(S)} = t_{02}^{(S)} + t_{03}^{(S)}, \quad t_{23}^{(S)} = t_{12}^{(S)} + t_{13}^{(S)}, \quad t_{01}^{(S)} = t_{02}^{(S)} + t_{12}^{(S)}, \quad t_{01}^{(S)} = t_{03}^{(S)} + t_{13}^{(S)}. \tag{3}$$

Comparing the first two relations gives $t_{02}^{(S)} + t_{03}^{(S)} = t_{12}^{(S)} + t_{13}^{(S)}$, and comparing the last two gives $t_{02}^{(S)} + t_{12}^{(S)} = t_{03}^{(S)} + t_{13}^{(S)}$. Subtracting the second identity from the first yields $t_{03}^{(S)} - t_{12}^{(S)} = t_{12}^{(S)} - t_{03}^{(S)}$, hence $t_{03}^{(S)} = t_{12}^{(S)}$; substituting this back into the first identity yields $t_{02}^{(S)} = t_{13}^{(S)}$. Writing α and β for the two common values, the first and third relations of (3) give $t_{01}^{(S)} = t_{23}^{(S)} = \alpha + \beta$. Conversely, any $\alpha, \beta > 0$ satisfies all four relations, so the solution set is the open quadrant and the system is consistent with strict positivity; this is exactly the feasibility already asserted by Lemma 5.5.

Finally, $t_{jk}^{(S)} = -\log |r_{jk}^{(S)}|$ by Definition 2.32, so exponentiating gives $|r_{02}^{(S)}| = |r_{13}^{(S)}| = A$ and $|r_{03}^{(S)}| = |r_{12}^{(S)}| = B$ for the four cross-pair correlations, and $|r_{01}^{(S)}| = |r_{23}^{(S)}| = e^{-(\alpha+\beta)} = AB$ for the two within-pair correlations. Indexing the two pairs in the order $0,1$ and $2,3$ puts the cross-pair values in the block $\begin{pmatrix} A & B \\ B & A \end{pmatrix}$, as claimed. □

Remark 5.7 (Why the order must be even, and why odd orders resist)**.** A fixed-point-free involution exists only on a set of even cardinality, so the pairing of Lemma 5.5 is available at a level with four survivors precisely because 4 is even. Propagating the pairing structure to the whole vertex set therefore requires n even, which is what M_{2m} realizes; this is the source of the parity restriction in Theorem 5.8.

Odd orders $n \geq 7$ remain open, and it is worth recording exactly how little of the method survives there. The classification of Lemma 5.5 needs $|W| = 4$: its injectivity step fails at $|W| = 5$, where feasible sets using all five pivots do exist, a 4-pairing together with any slot for the fifth pivot being feasible in all six cases. Parity therefore does not propagate, and odd orders enjoy no structural protection of the kind that forces the construction to be even. Two computations delimit the position further. At $n = 5$ a feasible family has at most one slot on each triple $\{u\} \cup p$, since two slots on one triple force a log-magnitude to vanish (add their relations, as in the proof of Lemma 5.5(1)); so it has at most $\binom{5}{3} = 10$ slots. The bound is attained: for the points $0, 1, \ldots, 4$ of a line, the ten slots in which the middle point of a triple cancels the other two are realized by the SPD matrix with entries $2^{-|j-k|}$. A scheme leaving one survivor at every vertex needs at least two slots at each vertex, since one slot covers only two of the other four vertices; it therefore has exactly two at each vertex and exactly one on each of the ten triples, and none of the 288 configurations of that kind is feasible (Appendix G), so no such scheme exists at order five. Among the $3^{10} = 59049$ three-magnitude colorings the minimum total number of survivors is 8, and none leaves exactly one survivor at every vertex. At $n = 7$ a pruned depth-first search of 280 million nodes together with canonical augmentation (92 423 orbits at level 7, growth factor ≈ 4.7) failed to exhaust the space, so brute force did not settle the question; that search is not included in the ancillary material. We note in passing that a counting heuristic distinguishes the parities — a scheme leaving one survivor at every vertex imposes $n\lceil (n-2)/2 \rceil$ relations on $\binom{n}{2}$ unknowns, which is underdetermined for even n and exactly square for odd n, so that a *generic* square system would force every $t_{jk}^{(\emptyset)} = 0$ and hence $|r^{(\emptyset)}| = 1$, contradicting strict definiteness — but we attach no weight to it: the hypothesis

AI

of genericity fails precisely on the singular locus where M_{2m} and every other counterexample of this paper live.

5.2. M_{2m} family.

Theorem 5.8 (Conjecture 1 fails at every even $n \geq 6$)**.** *Let $m \geq 2$ and let $n = 2m$. Partition $V = \{0, 1, \ldots, 2m-1\}$ into the m pairs $P_k = \{2k, 2k+1\}$, $0 \leq k \leq m-1$. Let A, B satisfy*

$$A^2 + (1 - B^2)A - 1 = 0, \qquad 0 < B < A < 1. \tag{4}$$

Let M_{2m} be the symmetric matrix with unit diagonal whose off-diagonal entries are

$$(M_{2m})_{xy} = \begin{cases} AB, & \{x, y\} = P_k \text{ for some } k \quad \text{(within a pair)}, \\ A, & x, y \text{ in different pairs and } x \equiv y \pmod 2, \\ B, & x, y \text{ in different pairs and } x \not\equiv y \pmod 2. \end{cases}$$

Then $M_{2m} \succ 0$, with eigenvalues

$$\lambda_1 = 1 + AB + (A + B)(m - 1), \qquad \lambda_2 = 1 - AB + (A - B)(m - 1),$$

$$\lambda_3 = (1 - A)(1 - B) \text{ with multiplicity } m - 1, \qquad \lambda_4 = (1 - A)(1 + B) \text{ with multiplicity } m - 1,$$

all four positive by (4). Moreover, for every $m \geq 3$ the matrix M_{2m} has no clean ordering. Hence Conjecture 2.27 is false at every even order $n \geq 6$.

Remark 5.9 (The parameter curve is a one-parameter family)**.** The hypothesis (4) is neither vacuous nor more restrictive than the curve itself. Fix $B \in (0, 1)$ and put $f(A) = A^2 + (1 - B^2)A - 1$. Then $f(0) = -1 < 0$ and f is strictly increasing on $A > 0$, while $f(1) = 1 - B^2 > 0$; hence f has exactly one root in $(0, 1)$, namely

$$\alpha(B) = \frac{-(1 - B^2) + \sqrt{(1 - B^2)^2 + 4}}{2} \in (0, 1).$$

Here $\alpha(B)$ denotes the root of f regarded as a function of B, not a product. Moreover $\alpha(B) > B$ automatically. Indeed f is strictly increasing on $A > 0$ and vanishes at $\alpha(B)$, so for $x > 0$ we have $x < \alpha(B)$ if and only if $f(x) < 0$; taking $x = B$ gives

$$f(B) = B^2 + (1 - B^2)B - 1 = -(B - 1)^2(B + 1) < 0 \qquad \text{for all } B \in (0, 1),$$

whence $B < \alpha(B)$. So the admissible parameters are exactly the pairs $(\alpha(B), B)$ with $0 < B < 1$, and the inequalities $0 < B < A < 1$ of (4) are consequences of the equation rather than extra assumptions. The endpoints are excluded for good reason: $B = 0$ would annihilate the opposite-parity entries, so that $G_{M_{2m}}$ would no longer be complete, while $B \to 1$ forces $A \to 1$ and $\lambda_3 = (1 - A)(1 - B) \to 0$.

Consequently Theorem 5.8 produces, at each even order $n = 2m \geq 6$, a one-parameter family of counterexamples lying on a measure-zero algebraic curve, not a single matrix. The instance $B = \frac{1}{2}$, for which $A = (-3 + \sqrt{73})/8 \approx 0.693$ and $AB \approx 0.3465$, is the one used for the explicit verification described in Appendices A and H; a concrete instance is needed there only because exact arithmetic must be carried out in a specific field, here $\mathbb{Q}(\sqrt{73})$. Whether the curve accounts for *every* counterexample at even order is a separate question, and an open one; we state it as Question 7.1.

Proof. Throughout we use the two facts recorded in (4): that A lies strictly between B and 1,

and that A is the root of $A^2 + (1 - B^2)A - 1 = 0$ in $(0, 1)$. The first gives $\lambda_2 > 0$ for every m, since $A - B > 0$, and the remaining three eigenvalues are visibly positive; the second will be used only once, in step 5, where it is forced upon us rather than assumed. The diagonal of M_{2m} is the identity, so every pivot value at stage 1 equals 1 and the Schur update of Definition 2.6 reads $S^{(1)}_{jk} = M_{jk} - M_{p_0 j} M_{p_0 k}$ for a first pivot p_0.

Fix any first pivot. Since relabelling pairs is a symmetry of M_{2m}, and since the two vertices of a pair play symmetric roles, we may assume the first pivot is $p_0 = 2k_0$ for some k_0; write $q_0 = 2k_0 + 1$ for its *partner*. Let $m \geq 3$. Figure 5 shows the pair structure and the two stages of the argument for $m = 3$.

Step 1: the first pivot zeroes a perfect matching on the other pairs, for every choice of parameters. Let $k \neq k_0$ and consider the within-pair position $(2k, 2k+1)$. Here $M_{2k,2k+1} = AB$, while $M_{p_0,2k} = A$ because p_0 and $2k$ have equal parity, and $M_{p_0,2k+1} = B$ because p_0 and $2k + 1$ have opposite parity. Hence

$$S^{(1)}_{2k,2k+1} = AB - A \cdot B = 0.$$

This cancellation is an identity in A and B: it holds for every choice of the two parameters, and no relation between them is required. Write

$$Z_{p_0} = \{\{2k, 2k + 1\} : k \neq k_0\}$$

for the resulting set of zero pairs, a perfect matching on the $m - 1$ pairs other than P_{k_0}.

Step 2: the partner q_0 meets none of these zeros. For $k \neq k_0$ we have $M_{q_0,2k} = B$ and $M_{q_0,2k+1} = A$, while $M_{q_0 p_0} = AB$, so

$$S^{(1)}_{q_0,2k} = B - AB \cdot A = B(1 - A^2) \neq 0, \qquad S^{(1)}_{q_0,2k+1} = A - AB \cdot B = A(1 - B^2) \neq 0,$$

both nonzero because $0 < A < 1$ and $0 < B < 1$. Hence no zero created by the first pivot is incident to q_0: writing Z_{p_0} for the graph on $V \setminus \{p_0\}$ whose edges are the cancelled pairs, q_0 is an isolated vertex of Z_{p_0}, while every other survivor meets exactly one edge of it.

Step 3: no cross-pair position cancels. Let $k \neq \ell$ with $k, \ell \neq k_0$. There are three cases, according to parities.

- Both even, $p = 2k$ and $q = 2\ell$: then $M_{pq} = A$ and $M_{p_0 p} = M_{p_0 q} = A$, so $S^{(1)}_{pq} = A - A^2 = A(1 - A) \neq 0$.
- Mixed, $p = 2k$ and $q = 2\ell + 1$: then $M_{pq} = B$, $M_{p_0 p} = A$ and $M_{p_0 q} = B$, so $S^{(1)}_{pq} = B - AB = B(1 - A) \neq 0$.
- Both odd, $p = 2k + 1$ and $q = 2\ell + 1$: then $M_{pq} = A$ and $M_{p_0 p} = M_{p_0 q} = B$, so $S^{(1)}_{pq} = A - B^2$. Since $0 < B < A$ and $B < 1$, we have $A - B^2 > B - B^2 = B(1 - B) > 0$, so this entry too is nonzero.

Together with step 2 this shows that Z_{p_0} is exactly the set of zeros of $S^{(1)}$.

Step 4: the second pivot is forced to be q_0. Suppose the second pivot is some $p_1 \neq q_0$, say $p_1 \in P_k$ with $k \neq k_0$, and let p_1' be its partner, so that $\{p_1, p_1'\} \in Z_{p_0}$ by step 1. Because $G_A = K_{2m}$ is complete, this pair is predicted nonzero, and it is committed to L at the moment p_1 is pivoted, that is at level $S = \{p_0\}$, where its value is 0. Hence any such ordering already fails. Only $p_1 = q_0$ avoids an immediate failure, and by step 2 it does avoid one, since q_0 meets no zero of $S^{(1)}$.

It is worth being explicit about why the zeros of step 1 are not themselves failures of Conjecture 2.27. Under the factor definition of cleanliness a pair $\{j, k\}$ is committed only at the level

preceding the earlier of j and k, so a zero in $S^{(1)}$ costs nothing unless the second pivot lies in that pair; an intervening elimination may restore the entry, which is the refill phenomenon of Section 2. This is exactly why M_4 has eight clean orderings while M_{2m} has none for $m \geq 3$.

Step 5: after the forced second pivot, every remaining vertex meets a zero — and this is where the parameter equation earns its place. Let $k, \ell \neq k_0$ with $k \neq \ell$. By steps 2 and 3,

$$S^{(1)}_{2k,2\ell+1} = B(1-A), \quad S^{(1)}_{q_0,2k} = B(1-A^2), \quad S^{(1)}_{q_0,2\ell+1} = A(1-B^2), \quad S^{(1)}_{q_0q_0} = 1 - A^2B^2,$$

the last because $M_{q_0p_0} = AB$ and the diagonal is 1. Pivoting at q_0 therefore gives

$$S^{(2)}_{2k,2\ell+1} = S^{(1)}_{2k,2\ell+1} - \frac{S^{(1)}_{q_0,2k}\, S^{(1)}_{q_0,2\ell+1}}{S^{(1)}_{q_0q_0}} = B(1-A) - \frac{AB(1-A^2)(1-B^2)}{1-A^2B^2}.$$

We claim this vanishes precisely on the curve (4). Setting it to zero and clearing the denominator gives $B(1-A)(1-A^2B^2) = AB(1-A^2)(1-B^2)$; cancelling $B \neq 0$ and then $1-A \neq 0$, and using $1-A^2 = (1-A)(1+A)$, this is equivalent to

$$1 - A^2B^2 = A(1+A)(1-B^2),$$

that is, to $1 = A^2B^2 + A(1-B^2) + A^2(1-B^2)$. Since $A^2B^2 + A^2(1-B^2) = A^2$, the right-hand side is $A^2 + A(1-B^2)$, so the condition is exactly $A^2 + (1-B^2)A - 1 = 0$. Thus $S^{(2)}_{2k,2\ell+1} = 0$ if and only if (4) holds: the parameters are forced onto the curve, not chosen on it, and with $B = 1/2$ the root in $(0,1)$ is $A = (-3+\sqrt{73})/8$.

Consequently the zero set after the two forced pivots contains

$$Z^{(2)} = \{\{2k, 2\ell+1\} : k \neq \ell,\ k, \ell \neq k_0\},$$

in which every surviving vertex has degree exactly $m-2$: an even survivor $2k$ is paired with $2\ell+1$ for each of the $m-2$ indices $\ell \neq k, k_0$, and symmetrically for an odd survivor. For $m \geq 3$ this degree is at least 1, so whichever vertex p_2 is pivoted third, some pair $\{p_2, x\}$ with x a survivor is zero, is predicted nonzero because G_A is complete, and is committed to L at that stage. Hence the third pivot fails.

Collecting the three possibilities: a first pivot may be any vertex, after which either the second pivot is not the partner q_0 and fails at stage 2 by step 4, or it is q_0 and the third pivot fails at stage 3 by step 5. No ordering of M_{2m} is therefore clean when $m \geq 3$, which proves the theorem. □

AI

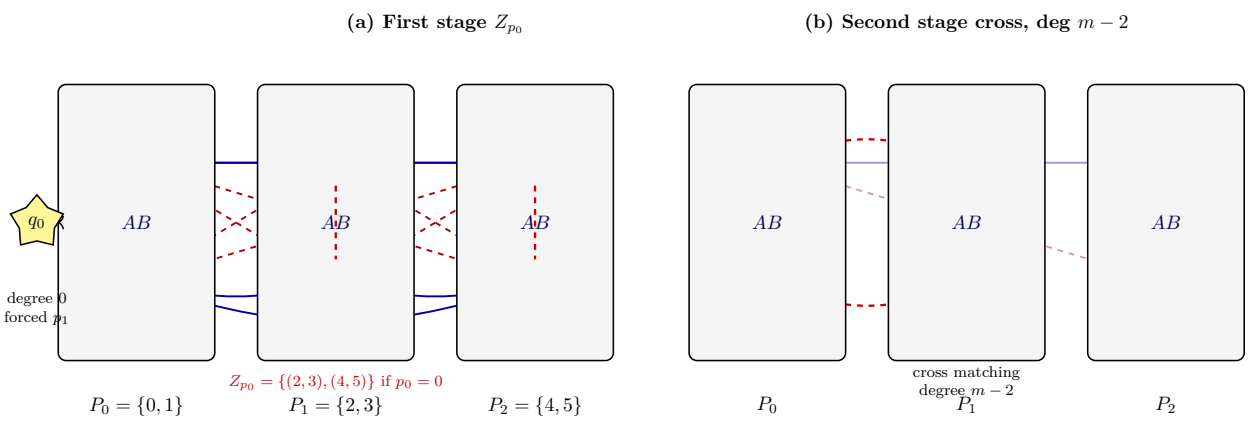


Figure 5: Pairing structure of M_{2m} for $m = 3$. Pair-boxes $P_k = \{2k, 2k+1\}$ carry the within-pair entry AB; cross-pair blocks are $\left(\begin{smallmatrix} A & B \\ B & A \end{smallmatrix}\right)$. (a) First stage: Z_{p_0} is a perfect matching on the other $m-1$ pairs (degree $m-1$); the partner q_0 has degree 0, forcing $p_1 = q_0$. (b) Second stage: the cross matching has degree $m-2 \geq 1$ for $m \geq 3$.

Human

Remark 5.10. The matrix A^* in Theorem 5.1 has the shape of the M_{2m} family at order four. Normalized to unit diagonal, $|r_{03}| = |r_{12}| = 1/4$ and all four cross entries have modulus $1/2$. Hence, after relabeling $(0, 3, 1, 2)$ and conjugating by the diagonal sign matrix $D = \mathrm{diag}(-1, 1, 1, 1)$ — which preserves positive definiteness and leaves every $t_{jk}^{(S)} = -\log|r_{jk}^{(S)}|$ unchanged, so it preserves the entire cancellation structure — A^* becomes the matrix M_4 of Theorem 5.8 with $m = 2$, $A = B = 1/2$, $AB = 1/4$. This parameter point is off the curve, since $A^2+(1-B^2)A-1 = -3/8 \neq 0$, but the curve is not what makes A^* fail only the First-Schur Conjecture. For $m = 2$ and any $0 < A, B < 1$ the matrix M_4 is positive definite, with eigenvalues $1 + AB + A + B$, $(1 + A)(1 - B)$, $(1-A)(1-B)$ and $(1-A)(1+B)$. Step 1 of the proof of Theorem 5.8, which holds for all parameters, shows that every first pivot zeroes the within-pair entry of the other pair, so Conjecture 2.26 fails, while Conjecture 2.27 holds by Theorem 5.2. The curve enters only in step 5, which needs $m \geq 3$.

6. Consequences for solver authors

Human

Two questions a reader implementing sparse Cholesky will ask deserve explicit answers. *Must a solver test for these cancellations?* No: symbolic factorization allocates for G_σ^+, and by Definition 2.12 the numeric pattern is contained in the symbolic one, so a cancellation can only leave a reserved entry holding an exact zero. The factorization and the solve remain correct, at the price of a slightly over-allocated structure, and none of our counterexamples makes a standard code return a wrong answer; a solver that wanted to reclaim that space would have to test numerically during the factorization, and Theorem 5.8 shows that no ordering can guarantee those tests all fail. *Is there a benefit to knowing that a clean ordering exists?* Not for allocation, which is driven by G_σ^+ either way, but for diagnosis: an exact zero at a predicted-nonzero position is an artefact of the ordering when the pattern lies in one of the classes of Theorem 4.8, whereas for K_6 it may be intrinsic to the matrix (Theorem 5.8). In the statistical reading of Section 1.4 this is the difference between an unfaithfulness a reordering can remove and one it cannot. We claim no performance consequence.

7. Status and open problems

Human

It is worth collecting in one place what the preceding sections establish and what they leave open, since a paper that pairs an impossibility result with a list of positive classes makes the two easy to conflate.

The existential question is settled, and settled negatively. Conjecture 2.26 already fails at order four: the matrix A^* of Theorem 5.1, with spectrum $\{9, 3, 3, 1\}$, has no ordering whose first Schur complement is cancellation-free, while 8 of its 24 orderings nevertheless deliver a clean factor. The gap between those two counts is the refill phenomenon, and isolating it is precisely what allows the stronger Conjecture 2.27 to survive at order four. Conjecture 2.27 itself fails at every even order $n \geq 6$, by the family M_{2m} of Theorem 5.8, so the smallest known pattern on which cleanliness genuinely fails is the complete graph K_6; whether some pattern of order five fails, K_5 in particular, is open.

The counterexample is derived rather than exhibited. This is the part of the argument we would emphasize, because it is what distinguishes the result from a lucky search. We did not find a matrix and then verify it. The Pairing Lemma 5.5 classifies the zero-configurations available at a level with four surviving vertices, and shows that a feasible family of maximal size using all four survivors as apexes must be a fixed-point-free involution. For counterexamples built this way, evenness of the order is then a consequence rather than a hypothesis; the magnitudes AB within pairs and the blocks $\begin{pmatrix} A & B \\ B & A \end{pmatrix}$ across them are consequences; and the curve $A^2 + (1 - B^2)A - 1 = 0$ is a consequence. By Remark 5.9 what emerges at each even order is a one-parameter family on a measure-zero algebraic curve, not an isolated matrix, and the inequalities $0 < B < A < 1$ follow from the equation instead of being imposed alongside it. How far that rigidity extends is the subject of Question 7.1 below.

The positive side is a boundary rather than a miscellany. Conjecture 2.27 holds for every pattern of order at most four (Theorem 5.2), by an elementary case analysis that uses only the pivot update, the cancellation graph and four lemmas. Beyond that, Theorem 4.1 gives forests in the strong form that *every* ordering is clean, which recovers in factorization language the covariance-faithfulness theorem of Malouche and Rajaratnam [MR11], and Theorem 4.8 adds 2-path strips, dominating-vertex fans, chordal graphs with exactly one triangle, and chordal graphs of clique number at most three whose triangles are pairwise edge-disjoint. Each of these classes is adaptive of necessity: the triangle lies in all of them, and no single fixed ordering is clean for every matrix with that pattern (Remark 4.9), so the quantifier order in Conjecture 2.27 is a feature of the problem and not of its phrasing. All of these classes have clique number at most three, while the failures we know are complete graphs; where the boundary lies between the two is open.

The apparatus is reusable, and is the part most likely to outlast the specific results. Cleanliness is recast as a covering problem over slots: a slot $(S, \{j, k\})$ asserts $(A/A_S)_{jk} = 0$, a blocking family is one that meets every ordering, and Conjecture 2.27 fails for a pattern exactly when some blocking family is realizable (Remark 2.30). The additive parametrization turns the multiplicative cancellation condition into a linear system, infeasibility of which is certified by an integer Stiemke witness, and the pattern-level lemmas — Prefix, fresh-fill closure, zero-leg, Nesting, Exchange and Monotone — are what make families refutable without reference to any particular matrix.

No theorem in this paper depends on a computation. Every result above is proved by hand, in

Human

exact arithmetic where arithmetic is needed at all. The ancillary scripts of Appendix A recompute asserted values independently, and Appendix H is explicit that both the order-four slot search and the four-vertex feasibility counts corroborate rather than constitute the proofs they accompany. The computations reported in Remark 5.7 and the order-five search below concern cases that remain open, and together with the sampling count of Remark 4.6 they are the places where this reading needs care: they are not steps in any argument, nothing established above depends on them, and the order-five and order-seven searches and that sampling run are not included in the ancillary material.

What is not settled. The overall shape of the results is that the positive side is governed by the scarcity of triangles and by the availability of a simplicial or universal pivot, whereas the negative side, so far, uses complete patterns. Three frontiers remain open — order five, the connected patterns of order six other than K_6, and all odd orders from seven upwards. The first two are finite questions that we leave undecided; the third is not, and Remark 5.7 explains why the parity argument that settles the even orders gives nothing there. To these we add the structural question of how much the even-order construction really forces.

How far order five has been pushed. The order-five question is finite, and we have gone some way into it without settling it. Using the covering reformulation of Remark 2.30 together with the pattern rules of Section 3, a depth-first search over partial blocking families closes twenty of the twenty-one connected patterns of order five: for each of them every branch ends in a refutation, by a pattern rule or, where the rules do not suffice, by an integer Stiemke certificate checked in exact arithmetic. The complete graph K_5 is the exception. There the search does not terminate, with branches reaching families of some twenty slots without closing, and the natural alternative — enumerating the minimal blocking families of K_5 and refuting each — is worse, since a direct enumeration passes 2×10^7 of them without exhausting the list. Because the order-five statement would rest on a single unresolved pattern, we record this as an observation rather than a theorem. The search is not included in the ancillary material.

Six questions are left open: order five; the connected patterns of order six other than K_6, of which $K_6 - e$ is the densest; all odd orders $n \geq 7$; general partial 2-trees; outerplanar graphs in general; and whether the M_{2m} curve accounts for every even-order counterexample. Two of them we state formally: the rigidity question just mentioned, and a scheduling question which arose while proving Theorem 4.8(i) and which would extend it.

Question 7.1 (Does the curve exhaust the even-order counterexamples?)**.** Is every counterexample to Conjecture 2.27 at even order $n \geq 6$ a point of the M_{2m} curve (4), up to permutation and diagonal scaling?

Theorem 5.8 gives no information here, because it is existential: it produces counterexamples and shows that their parameters are forced once the pairing shape is assumed, but it never quantifies over all SPD matrices of a given order. The forcing in Lemma 5.5 is conditional in the same way. It applies to a level with exactly four survivors carrying a feasible family of maximal size that uses all four of them as apexes, and only three of the fifteen maximal feasible families at such a level meet that last condition; at five survivors the classification fails outright, since feasible families using all five apexes exist there (Remark 5.7). A counterexample resting on a different mechanism is therefore not excluded by anything we prove, and we state Question 7.1 as a question rather than as a conjecture because we have no evidence either way.

Human

An affirmative answer would be strong, and the reason is worth recording. Every off-diagonal entry of M_{2m} is nonzero, and permutation and diagonal scaling preserve this, so every counterexample on the curve has a complete pattern. An affirmative answer at $n = 6$ would therefore show that every connected pattern of order six other than K_6 admits a clean ordering for every SPD matrix of that pattern, settling order six, one of the two finite frontiers above. Question 7.1 is in that sense at least as hard as the whole of order six, and a counterexample to it would be at least as interesting as a proof: it would mean that the mechanism isolated by the Pairing Lemma is not the only route to failure.

Question 7.2 (Clean strategies ending at a prescribed edge)**.** Does every 2-path strip, and every dominating-vertex fan, admit an adaptive clean strategy whose last two pivots are the two ends of a prescribed boundary edge?

An affirmative answer would give Conjecture 1 for clique sums along cliques of size at most two, and hence for strips and fans glued along an edge or a vertex.[1] The obstruction is the final triangle. Running the strip induction from the left end leaves $\{v_{k-2}, v_{k-1}, v_k\}$ as the last three survivors, and Lemma 3.2 supplies only *some* clean apex among them; finishing with $K = \{v_{k-1}, v_k\}$ requires the apex to be v_{k-2}. If v_{k-2} is the dirty vertex of that triangle, pivoting it commits a zero at the K-pair. No refill argument repairs this, because refill (Lemma 3.5) needs the zero to pre-exist the pivot that destroys it, whereas here the offending pivot is itself the apex that creates it; and reversing the direction of the induction merely relocates the final triangle. A positive answer would also bear on the question of outerplanar graphs in the list above.

8. Conclusion

Human

The existential question has a negative answer in general and a positive one on a class we can now delimit. Conjecture 2.26 is false already at order four (Theorem 5.1): the matrix A^* admits no ordering whose first Schur complement is cancellation-free, yet 8 of its 24 orderings still yield a clean factor, which is the refill phenomenon in its simplest form. Conjecture 2.27 holds for $n \leq 4$ by an elementary case analysis with no computation (Theorem 5.2), and fails at every even order $n \geq 6$ through the family M_{2m} (Theorem 5.8). That family is derived rather than exhibited: classifying the zero-configurations available at a level with four surviving vertices forces a fixed-point-free involution, hence even order, and forces the magnitudes within and across pairs, so the curve $A^2 + (1 - B^2)A - 1 = 0$ on which this family lies is imposed by the classification rather than chosen. The forcing is a rigidity statement about counterexamples of this shape; it does not assert that every counterexample lies on the curve (Question 7.1). Every point of that curve with $0 < B < 1$ yields a counterexample, and $B = 1/2$ is the instance we verify explicitly.

On the positive side, forests are clean in the strong sense that every ordering is clean (Theorem 4.1), and Conjecture 2.27 also holds for 2-path strips, for dominating-vertex fans, for chordal graphs with exactly one triangle, and for chordal graphs of clique number at most three whose triangles are pairwise edge-disjoint (Theorem 4.8); on each class of Theorem 4.8 the clean ordering

[1]Sketch. Let $G = G_1 \cup G_2$ with $G_1 \cap G_2 = K$ a clique, $|K| \leq 2$; suppose G_2 admits an adaptive clean strategy (a) eliminating K last, and G_1 an adaptive clean strategy (b). Pivot $G_2 \setminus K$ by the initial segment of (a), then $G_1 \cup K$ by (b). Since no vertex of K is pivoted in phase 1, no pair inside K is committed there, and because K separates, $A[G_2]$ is a principal submatrix with pattern G_2 on which phase 1 is exactly (a); as K comes last in (a), the pair inside K is still nonzero after phase 1. By the fill-path theorem no fill crosses the separator, so the Schur complement handed to phase 2 is $A[G_1]$ altered only in the $K \times K$ block and still has pattern G_1; since (b) is adaptive it is valid for every SPD matrix of that pattern, so phase 2 is clean. Only hypothesis (a) is at issue, and it is exactly Question 7.2.

Human

must adapt to the matrix, since no fixed ordering suffices even for the triangle (Remark 4.9).

Two gaps remain, and they are of different kinds. Order five we leave open: the question there is finite, pattern by pattern, but we do not settle it. Odd orders $n \geq 7$ are open for a structural reason rather than a computational one, since the pairing classification that drives the even construction requires exactly four survivors and its injectivity step fails at five (Remark 5.7); the square-count identity is evidence about where counterexamples could live rather than an obstruction to their existence, because the counterexamples we construct lie on a singular locus that such counting does not detect.

For practice the conclusion is reassuring in the direction that matters. The no-cancellation assumption of symbolic factorization remains safe for allocation, since over-allocation never threatens correctness, and none of our counterexamples causes a standard code to return a wrong answer. What is not guaranteed is the existential statement: at every even order from six upwards there are SPD matrices for which no ordering avoids an exact zero at a position the symbolic phase predicts nonzero. The ancillary scripts listed in Appendix A reproduce the numerical claims they cover, and Appendix H reports the independent recomputation and says which computations are not included.

Acknowledgments

The author would like to thank Ricky Simanjuntak for discussions at various stages of this project and his feedback on the final draft. The authors would also like to thank Paul Bendich and Patrick Tam for so kindly reviewing this work and offering very thoughtful suggestions and edits which have greatly improved the content of this draft.

Appendix A. Computational protocol

Human

All computations reported here were carried out in Python 3.11.9. The ancillary scripts need only the standard library, with exact integer or rational arithmetic, except `enum-pairing-4vert.py`, which uses NumPy and SciPy's `linprog` (its counts are reproduced without a solver by `slot_counts.py`). Pseudorandom draws, which enter only the sampling experiment of Appendix H, were seeded with 42, 123 and 2026. Where a linear program is solved — in the order-four slot search of Appendix H and in the feasibility counts reported there — HiGHS [HH18] serves only as a search device for a candidate certificate, and correctness never rests on it. By Stiemke's alternative, infeasibility of $Tt = 0$, $t > 0$ is witnessed by an integer vector y with $y^\top T \geq 0$ and $y^\top T \neq 0$; since T has integer entries such a witness is checked by integer arithmetic alone, so no floating-point computation enters any proof.

Six scripts are provided as ancillary files. `verify_cholesky.py` recomputes the worked instances of Theorems 5.1 and 5.8 from their statements alone. `enum-pairing-4vert.py` enumerates the $2^{12} = 4096$ slot subsets at a four-vertex level and finds 182 of them feasible, the computation behind the three all-pivot configurations of Lemma 5.5. `three-mag-59049.py` sweeps the $3^{10} = 59049$ three-magnitude colourings at order five, establishing the minimum survivor count of 8 reported in Remark 5.7. `peo_insufficiency.py` runs all 5040 orderings of an SPD realization of the 2-path of 5 triangles, the witness of Remark 4.6. `fixed_ordering_witnesses.py` checks the witnesses of Remark 4.9: the three realizations of the triangle and the five realizations of the pattern E defeat every fixed ordering. `slot_counts.py` decides feasibility exactly, by Fourier–Motzkin elimination in

Human

rational arithmetic, and reproduces the four-vertex counts of Lemma 5.5, the order-four slot search of Appendix H, and the order-five slot bounds of Remark 5.7. For the instance of Theorem 5.8 used throughout, $B = 1/2$ and $A = (-3+\sqrt{73})/8$, so that $B(1-A) = 11/16 - \sqrt{73}/16 \approx 0.1534997659$.

Appendix B. Source of `verify_cholesky.py`

AI

```
#!/usr/bin/env python3
"""
verify_cholesky.py -- independent verification of the two counterexamples in
"Accidental Cancellations in Cholesky Factorization"  (Nam Trang).

Part A (Theorem 5.1).  The 4x4 matrix

        A* = [[ 4, -2, -2, -1],
              [-2,  4,  1,  2],
              [-2,  1,  4,  2],
              [-1,  2,  2,  4]]

is SPD with spectrum {9,3,3,1} and pattern graph K_4.  None of its 24
elimination orderings has a cancellation-free FIRST Schur complement, so the
First-Schur Conjecture (Conjecture 2) is false; but 8 of the 24 orderings give
a clean Cholesky factor, so A* is NOT a counterexample to the Fill-Graph
Conjecture (Conjecture 1).  The gap between the two is the refill phenomenon.

Part B (Theorem 5.8).  With B = 1/2 and A = (-3 + sqrt(73))/8, the positive
root of the curve A^2 + (1 - B^2) A - 1 = 0, the 6x6 matrix

        M_6 = I_6 + AB*P + A*Q + B*R,
        P = I_3 (x) [[0,1],[1,0]],
        Q = (J_3 - I_3) (x) I_2,
        R = (J_3 - I_3) (x) [[0,1],[1,0]]

(vertices paired {0,1},{2,3},{4,5}; within-pair entry AB; each cross-pair block
[[A,B],[B,A]]) is SPD with pattern K_6, and NONE of its 720 orderings is clean.
That refutes the Fill-Graph Conjecture (Conjecture 1) at order 6, the smallest
order at which it fails.  Part B also re-checks the steps of the proof, not
just its conclusion: the forced zero matching after the first pivot, the forced
second pivot, and the covering of all survivors after it.

DEPENDENCIES: none.  Python standard library only (fractions, itertools, sys);
no NumPy, no SymPy, nothing to install.  Works on Python 3.7+.

ARITHMETIC: everything is exact.  Part A needs only rationals; Part B needs the
quadratic field Q(sqrt(73)), since A is irrational.  Both are handled by one
class Q: a number is a pair of Fractions standing for a + b*sqrt(73), with
exact equality, exact division, and an exact sign test that compares a^2 with
73*b^2 instead of evaluating a square root.  No decision anywhere in this
script is taken in floating point; floats appear only in printed decimals.
This matters: the claims are that certain entries are EXACTLY zero while their
neighbours are not, which no float computation can certify.

USAGE
    python3 verify_cholesky.py                # both parts (about 1 second)
    python3 verify_cholesky.py --only astar   # Part A only
    python3 verify_cholesky.py --only m6      # Part B only
    python3 verify_cholesky.py --all          # + every stage of all 24
                                              #   orderings of A*
    python3 verify_cholesky.py --show-stages  # + the forced opening of M_6
    python3 verify_cholesky.py --m8           # + all 40320 orderings of M_8
                                              #   (about 40 seconds)
    python3 verify_cholesky.py --help
```

AI

```
Exit status 0 if every check passes, 1 otherwise.
"""
```

AI

```
import sys
from fractions import Fraction as F
from itertools import permutations

if "--help" in sys.argv or "-h" in sys.argv:
    print(__doc__)
    sys.exit(0)

D = 73                      # the quadratic field is Q(sqrt(D))
_ok = [True]
_count = [0]


def check(cond, msg):
    _count[0] += 1
    print(("  [ok]   " if cond else "  [FAIL] ") + msg)
    if not cond:
        _ok[0] = False


# ---------------------------------------------------------------------------
# Exact arithmetic in Q(sqrt(D)); rationals are the b == 0 case.
# ---------------------------------------------------------------------------

class Q(object):
    """a + b*sqrt(D) with a, b rational.  Exact."""
    __slots__ = ("a", "b")

    def __init__(self, a=0, b=0):
        self.a = F(a)
        self.b = F(b)

    def __add__(s, o):
        o = _q(o)
        return Q(s.a + o.a, s.b + o.b)

    __radd__ = __add__

    def __neg__(s):
        return Q(-s.a, -s.b)

    def __sub__(s, o):
        return s + (-_q(o))

    def __rsub__(s, o):
        return _q(o) + (-s)

    def __mul__(s, o):
        o = _q(o)
        return Q(s.a * o.a + D * s.b * o.b, s.a * o.b + s.b * o.a)

    __rmul__ = __mul__
```

AI

```
    def inv(s):
        den = s.a * s.a - D * s.b * s.b
        if den == 0:
            raise ZeroDivisionError("division by zero in Q(sqrt(%d))" % D)
```

```
        return Q(s.a / den, -s.b / den)

    def __truediv__(s, o):
        return s * _q(o).inv()

    def __rtruediv__(s, o):
        return _q(o) * s.inv()

    def __eq__(s, o):
        o = _q(o)
        return s.a == o.a and s.b == o.b

    def __ne__(s, o):
        return not s.__eq__(o)

    def __hash__(s):
        return hash((s.a, s.b))

    def is_zero(s):
        return s.a == 0 and s.b == 0

    def sign(s):
        """Exact sign, by comparing a^2 with D*b^2.  No square roots."""
        if s.a == 0 and s.b == 0:
            return 0
        if s.b == 0:
            return 1 if s.a > 0 else -1
        if s.a == 0:
            return 1 if s.b > 0 else -1
        if s.a > 0 and s.b > 0:
            return 1
        if s.a < 0 and s.b < 0:
            return -1
        rational_part_bigger = s.a * s.a > D * s.b * s.b
        if s.a > 0:                                  # a > 0 > b
            return 1 if rational_part_bigger else -1
        return -1 if rational_part_bigger else 1     # a < 0 < b

    def __float__(s):
        return float(s.a) + float(s.b) * (D ** 0.5)

    def __repr__(s):
        if s.b == 0:
            return str(s.a)
        if s.a == 0:
            return "%s*sqrt(%d)" % (s.b, D)
        return "(%s + %s*sqrt(%d))" % (s.a, s.b, D)


def _q(x):
    return x if isinstance(x, Q) else Q(x)


ZERO, ONE = Q(0), Q(1)
```

```


# ---------------------------------------------------------------------
# Shared linear algebra.  Matrices are dicts of dicts keyed by vertex label,
# so that a Schur complement can simply forget the eliminated labels.
# ---------------------------------------------------------------------

def as_dict(M):
    n = len(M)
    return dict((i, dict((j, M[i][j]) for j in range(n))) for i in range(n))
```

AI

```python


def pivot(Mc, surv, p):
    """One elimination step: return (A/A_{S + p}) and the new survivor list."""
    rest = [v for v in surv if v != p]
    d = Mc[p][p]
    out = {}
    for j in rest:
        out[j] = dict((k, Mc[j][k] - Mc[j][p] * Mc[p][k] / d) for k in rest)
    return out, rest


def det(M):
    """Exact determinant by Gaussian elimination over Q(sqrt(D))."""
    M = [row[:] for row in M]
    n = len(M)
    d = ONE
    for c in range(n):
        p = None
        for r in range(c, n):
            if not M[r][c].is_zero():
                p = r
                break
        if p is None:
            return ZERO
        if p != c:
            M[c], M[p] = M[p], M[c]
            d = -d
        d = d * M[c][c]
        for r in range(c + 1, n):
            f = M[r][c] / M[c][c]
            if not f.is_zero():
                for k in range(c, n):
                    M[r][k] = M[r][k] - f * M[c][k]
    return d


def submatrix(M, idx):
    return [[M[i][j] for j in idx] for i in idx]
```

AI

```python


def cholesky_pivots(M):
    """The pivots D_j of the unpermuted factorization; all > 0 iff SPD."""
    n = len(M)
    Mc, surv, out = as_dict(M), list(range(n)), []
    for _ in range(n):
        out.append(Mc[surv[0]][surv[0]])
        if len(surv) > 1:
            Mc, surv = pivot(Mc, surv, surv[0])
    return out


def char_poly(M):
    """Coefficients of det(xI - M), highest degree first (Faddeev-LeVerrier)."""
    n = len(M)
    coeffs = [ONE]
    Mk = [[ZERO] * n for _ in range(n)]
    for k in range(1, n + 1):
        if k == 1:
            Mk = [[ONE if i == j else ZERO for j in range(n)] for i in range(n)]
        else:
            prod = []
            for i in range(n):
                row = []
```

```
                for j in range(n):
                    acc = ZERO
                    for t in range(n):
                        acc = acc + M[i][t] * Mk[t][j]
                    row.append(acc)
                prod.append(row)
            for i in range(n):
                prod[i][i] = prod[i][i] + coeffs[-1]
            Mk = prod
        tr = ZERO
        for i in range(n):
            for t in range(n):
                tr = tr + M[i][t] * Mk[t][i]
        coeffs.append(-tr / k)
    return coeffs


def poly_eval(coeffs, x):
    v = ZERO
    for c in coeffs:
        v = v * x + c
    return v


def zero_pairs(Mc, surv):
    out = []
    for ii, j in enumerate(surv):
        for k in surv[ii + 1:]:
            if Mc[j][k].is_zero():
                out.append((j, k))
    return out
```

```


def show(Mc, surv, indent="      "):
    """Print a Schur complement with its surviving labels."""
    cells = {}
    for i in surv:
        for j in surv:
            v = Mc[i][j]
            cells[(i, j)] = str(v.a) if v.b == 0 else "%.6f" % float(v)
    w = max(len(s) for s in cells.values()) if cells else 1
    print(indent + "     " + "  ".join(str(j).rjust(w) for j in surv))
    for i in surv:
        print(indent + str(i).rjust(3) + "  " +
              "  ".join(cells[(i, j)].rjust(w) for j in surv))


def clean(order, M, verbose=False):
    """Is `order` clean for M?

    An ordering is clean iff at every step every off-diagonal entry in the
    pivot's row (among survivors) is nonzero.  Those entries are exactly the
    column of L committed at that step (Definition 'factor entry'), and both
    matrices here have a COMPLETE pattern graph, so every one of them is a
    predicted-nonzero position and no separate fill-graph computation is
    needed.  Returns (is_clean, first_offence or None).
    """
    Mc, surv = as_dict(M), list(range(len(M)))
    for step, p in enumerate(order):
        if verbose:
            z = zero_pairs(Mc, surv)
            print("      stage %d: level {%s}, pivot %d; zeros among survivors: %s"
                  % (step, ", ".join(str(v) for v in order[:step]), p,
                     z if z else "none"))
```

```
            if len(M) <= 4:
                show(Mc, surv)
        for k in surv:
            if k != p and Mc[p][k].is_zero():
                if verbose:
                    print("        ^ position (%d,%d) is zero and is committed now"
                          " -> ordering is dirty" % (p, k))
                return False, (step, p, k)
        if len(surv) > 1:
            Mc, surv = pivot(Mc, surv, p)
    return True, None


# ---------------------------------------------------------------------------
# Part A: A*, the order-4 counterexample to the First-Schur Conjecture.
# ---------------------------------------------------------------------------

ASTAR = [[Q(x) for x in row] for row in
         [[4, -2, -2, -1],
          [-2, 4, 1, 2],
          [-2, 1, 4, 2],
          [-1, 2, 2, 4]]]
```

```


def part_a(dump_all=False):
    n = 4
    print("=" * 74)
    print("PART A -- A* (order 4):  Conjecture 2 is false, Conjecture 1 survives")
    print("=" * 74)

    print("\nA1. A* is symmetric positive definite")
    check(all(ASTAR[i][j] == ASTAR[j][i] for i in range(n) for j in range(n)),
          "A* is symmetric")
    minors = [det(submatrix(ASTAR, list(range(m)))) for m in range(1, n + 1)]
    check(all(m.sign() > 0 for m in minors),
          "leading principal minors all > 0: " + ", ".join(str(m) for m in minors))

    print("\nA2. The spectrum is {9,3,3,1}")
    cp = char_poly(ASTAR)
    print("       det(xI - A*) = " +
          " ".join(("+ " if c.sign() >= 0 else "- ") + str(abs(c.a)) + "x^" + str(n - i)
                   for i, c in enumerate(cp)))
    for lam, mult in [(9, 1), (3, 2), (1, 1)]:
        check(poly_eval(cp, Q(lam)).is_zero(),
              "x = %d is a root (claimed multiplicity %d)" % (lam, mult))
    check(cp[1] == Q(-16), "trace = 16 = 9+3+3+1")
    check(det(ASTAR) == Q(81), "det = 81 = 9*3*3*1")

    print("\nA3. The pattern graph is K_4, so every position is predicted nonzero")
    check(all(not ASTAR[i][j].is_zero() for i in range(n) for j in range(n) if i != j),
          "every off-diagonal entry of A* is nonzero")

    print("\nA4. Conjecture 2: every ordering cancels in the FIRST Schur complement")
    print("    The first Schur complement depends only on the first pivot, so the")
    print("    24 orderings give just four distinct matrices A*/A*_{p}:")
    firsts = {}
    for p in range(n):
        S, rest = pivot(as_dict(ASTAR), list(range(n)), p)
        z = zero_pairs(S, rest)
        firsts[p] = z
        print("\n    first pivot p = %d:   A*/A*_{%d}  on survivors %s" % (p, p, rest))
        show(S, rest)
        check(len(z) > 0,
              "has a zero at a predicted-nonzero position: %s" % (z,))
```

```
    stage1 = sum(1 for o in permutations(range(n)) if not firsts[o[0]])
    print()
    check(stage1 == 0, "orderings clean at stage 1: %d of 24" % stage1)

    print("\nA5. Conjecture 1: the FACTOR is nevertheless clean for 8 of the 24")
    good = [o for o in permutations(range(n)) if clean(o, ASTAR)[0]]
    print("       clean orderings: " +
          ", ".join("".join(str(v) for v in o) for o in good))
    check(len(good) == 8, "clean factor orderings: %d of 24" % len(good))
    partner = {0: 3, 3: 0, 1: 2, 2: 1}          # the pairing {0,3}, {1,2}
    check(all(o[1] == partner[o[0]] for o in good),
          "every clean ordering pivots the pairing partner second "
          "(pairing {0,3},{1,2}) -- this is the refill mechanism")

    print("\nA6. The refill mechanism, side by side")
    print("\n    (0,3,1,2) is clean.  Pivoting 0 zeroes (1,2), but 3 is pivoted next")
    print("    and refills it before anything commits it:")
```

```
    clean((0, 3, 1, 2), ASTAR, verbose=True)
    print("\n    (0,1,2,3) is dirty.  Pivoting 0 zeroes (1,2) and 1 comes next, so")
    print("    (1,2) is committed to L while still zero:")
    clean((0, 1, 2, 3), ASTAR, verbose=True)

    if dump_all:
        print("\nA7. --all: every stage of all 24 orderings")
        for o in permutations(range(n)):
            good_o, bad = clean(o, ASTAR)
            tag = "CLEAN" if good_o else ("dirty, first committed zero %s at stage %d"
                                          % (bad[1:], bad[0]))
            print("\n" + "-" * 66)
            print("    ordering %s: %s" % ("".join(str(v) for v in o), tag))
            print("-" * 66)
            clean(o, ASTAR, verbose=True)


# ---------------------------------------------------------------------------
# Part B: M_6, the order-6 counterexample to the Fill-Graph Conjecture.
# ---------------------------------------------------------------------------

B_PARAM = Q(F(1, 2))
A_PARAM = Q(F(-3, 8), F(1, 8))          # (-3 + sqrt(73))/8
AB_PARAM = A_PARAM * B_PARAM


def build(m):
    """M_{2m}: vertices 0..2m-1, paired {2k, 2k+1}."""
    n = 2 * m
    M = [[ZERO] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                M[i][j] = ONE
            elif i // 2 == j // 2:
                M[i][j] = AB_PARAM                  # within a pair
            elif i % 2 == j % 2:
                M[i][j] = A_PARAM                   # cross pair, same position
            else:
                M[i][j] = B_PARAM                   # cross pair, opposite
    return M


def part_b(show_stages=False, do_m8=False):
    m, n = 3, 6
```

AI

```
    M6 = build(m)
    print("\n" + "=" * 74)
    print("PART B -- M_6 (order 6):  Conjecture 1 is false")
    print("=" * 74)

    print("\nB1. The parameters lie on the forced curve")
    curve = A_PARAM * A_PARAM + (ONE - B_PARAM * B_PARAM) * A_PARAM - ONE
    check(curve.is_zero(), "A^2 + (1-B^2)A - 1 = 0 holds exactly")
    print("       A  = %s ~ %.10f" % (A_PARAM, float(A_PARAM)))
    print("       B  = %s" % B_PARAM)
    print("       AB = %s ~ %.10f" % (AB_PARAM, float(AB_PARAM)))

    print("\nB2. M_6 is symmetric, has pattern K_6, and is positive definite")
    check(all(M6[i][j] == M6[j][i] for i in range(n) for j in range(n)),
          "M_6 is symmetric")
    check(all(not M6[i][j].is_zero() for i in range(n) for j in range(n) if i != j),
          "every off-diagonal entry is nonzero: G = K_6, connected, so M_6 is irreducible")
    piv = cholesky_pivots(M6)
    check(all(p.sign() > 0 for p in piv),
          "all 6 Cholesky pivots are > 0 by exact sign test, so M_6 is SPD")
    print("       pivots ~ " + ", ".join("%.6f" % float(p) for p in piv))
    lams = [(ONE + AB_PARAM + (A_PARAM + B_PARAM) * (m - 1), 1, "1+AB+(A+B)(m-1)"),
            (ONE - AB_PARAM + (A_PARAM - B_PARAM) * (m - 1), 1, "1-AB+(A-B)(m-1)"),
            ((ONE - A_PARAM) * (ONE - B_PARAM), m - 1, "(1-A)(1-B)"),
            ((ONE - A_PARAM) * (ONE + B_PARAM), m - 1, "(1-A)(1+B)")]
    for lam, mult, name in lams:
        shifted = [[M6[i][j] - (lam if i == j else ZERO) for j in range(n)]
                   for i in range(n)]
        check(det(shifted).is_zero() and lam.sign() > 0,
              "lambda = %s ~ %.10f > 0 is an eigenvalue of multiplicity %d"
              % (name, float(lam), mult))
    tr = ZERO
    for i in range(n):
        tr = tr + M6[i][i]
    acc = ZERO
    for lam, mult, _ in lams:
        acc = acc + lam * mult
    check(acc == tr, "the eigenvalues sum to the trace (= 6)")
    prod = ONE
    for lam, mult, _ in lams:
        for _ in range(mult):
            prod = prod * lam
    check(prod == det(M6), "the eigenvalues multiply to det M_6")

    print("\nB3. Proof step 1: after any first pivot the zeros are exactly a perfect")
    print("    matching on the other pairs -- and proof step 2: the partner avoids them")
    for p0 in range(n):
        S, rest = pivot(as_dict(M6), list(range(n)), p0)
        z = zero_pairs(S, rest)
        expect = sorted((2 * k, 2 * k + 1) for k in range(m) if k != p0 // 2)
        q0 = p0 ^ 1
        check(sorted(z) == expect,
              "p0 = %d: zero set %s is the matching on the other pairs" % (p0, z))
        check(all(q0 not in pair for pair in z),
              "p0 = %d: partner q0 = %d meets no zero, so the second pivot is forced"
              % (p0, q0))
```

AI

```


    print("\nB4. Proof step 5: after the forced pivot, the zeros cover every survivor")
    for p0 in range(n):
        q0 = p0 ^ 1
        S, rest = pivot(as_dict(M6), list(range(n)), p0)
        S2, rest2 = pivot(S, rest, q0)
        z = zero_pairs(S2, rest2)
```

AI

```
        covered = set()
        for pair in z:
            covered.update(pair)
        check(covered == set(rest2),
              "p0 = %d, q0 = %d: zeros %s cover survivors %s, so the third pivot "
              "reads a zero" % (p0, q0, z, rest2))

    print("\nB5. The conclusion: no ordering of M_6 is clean")
    good = [o for o in permutations(range(n)) if clean(o, M6)[0]]
    check(len(good) == 0, "clean orderings: %d of 720" % len(good))
    hist = {}
    for o in permutations(range(n)):
        _, bad = clean(o, M6)
        hist[bad[0]] = hist.get(bad[0], 0) + 1
    print("       stage at which the first committed zero appears:")
    for st in sorted(hist):
        print("         stage %d: %d orderings" % (st, hist[st]))
    check(hist.get(2, 0) == n * 24,
          "exactly %d = 6 * 4! orderings reach stage 2 -- precisely those that take "
          "the forced partner second" % (n * 24,))

    if show_stages:
        print("\nB6. The forced opening, stage by stage (first pivot 0, so q0 = 1)")
        clean((0, 1, 2, 3, 4, 5), M6, verbose=True)
        print("\n    and if the forced second pivot is NOT taken (0 then 2):")
        clean((0, 2, 1, 3, 4, 5), M6, verbose=True)

    if do_m8:
        print("\nB7. --m8: the same at order 8 (40320 orderings; this takes a while)")
        M8 = build(4)
        p8 = cholesky_pivots(M8)
        check(all(p.sign() > 0 for p in p8), "M_8 is SPD (8 pivots > 0, exact)")
        g8 = 0
        for o in permutations(range(8)):
            if clean(o, M8)[0]:
                g8 += 1
        check(g8 == 0, "clean orderings of M_8: %d of 40320" % g8)
```

AI

```


# ---------------------------------------------------------------------------

def main():
    only = None
    if "--only" in sys.argv:
        i = sys.argv.index("--only")
        if i + 1 < len(sys.argv):
            only = sys.argv[i + 1].lower()
    print(__doc__.split("DEPENDENCIES")[0].strip())
    print()
    if only in (None, "astar", "a"):
        part_a(dump_all="--all" in sys.argv)
    if only in (None, "m6", "b", "m8"):
        part_b(show_stages="--show-stages" in sys.argv, do_m8="--m8" in sys.argv)

    print("\n" + "=" * 74)
    print("SUMMARY (%d checks)" % _count[0])
    print("=" * 74)
    if only in (None, "astar", "a"):
        print("  Conjecture 2 (First-Schur): FALSE at order 4.  Witness A*, 0 of 24")
        print("      orderings cancellation-free at stage 1.")
        print("  A* is NOT a counterexample to Conjecture 1: 8 of its 24 orderings")
        print("      give a clean factor, by refill.")
    if only in (None, "m6", "b", "m8"):
        print("  Conjecture 1 (Fill-Graph):   FALSE at order 6.  Witness M_6, 0 of 720")
```

```
        print("      orderings clean; the second pivot is forced and the third fails.")
    print()
    print("ALL CHECKS PASSED" if _ok[0] else "SOME CHECKS FAILED")
    return 0 if _ok[0] else 1


if __name__ == "__main__":
    sys.exit(main())
```

Appendix C. Source of enum-pairing-4vert.py

```
"""
enum_pairing_4vert.py -- Pairing Lemma (Lemma 5.5)
Fix level S, |W|=4, W={a,b,c,d}. Candidate slots 12, 6 variables t_{jk}.
Among 2^12=4096 subsets, 182 feasible, max feasible size 4,
exactly 3 achieve size 4 using all four pivots, each a fixed-point-free involution.

Protocol per Appendix:
Seeds 42,123,2026; Python 3.11.9, NumPy 1.26.4, SciPy 1.12 HiGHS
"""
import itertools
import numpy as np
from scipy.optimize import linprog

# W = 0,1,2,3
W = [0,1,2,3]
pairs = [(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]
pair_index = {p:i for i,p in enumerate(pairs)}
# candidate slots: (apex u, pair {j,k} subset of W\{u})
slots = []
for u in W:
    for (j,k) in pairs:
        if u not in (j,k) and j in W and k in W:
            # ensure j,k != u and distinct and both in W\{u}
            if j!=u and k!=u:
                # also need j,k in W\{u} automatically, but pair must be inside W\{u}
                # all pairs with u not in pair satisfy
                slots.append((u,(j,k)))
# slots should be 12
assert len(slots)==12, len(slots)

def build_T(subset_indices):
    """Build T matrix m x 6 where row = e_{jk} - e_{uj} - e_{uk}"""
    m = len(subset_indices)
    T = np.zeros((m,6))
    for r, idx in enumerate(subset_indices):
        u,(j,k) = slots[idx]
        # need indices for (j,k), (u,j), (u,k) with sorted pairs
        def srt(a,b): return (a,b) if a<b else (b,a)
        jk = srt(j,k)
        uj = srt(u,j)
        uk = srt(u,k)
        T[r, pair_index[jk]] += 1
        T[r, pair_index[uj]] -= 1
        T[r, pair_index[uk]] -= 1
    return T
```

```
def is_feasible(T):
    """Feasible if exists t>0 with T t =0. Equivalent to exists t>=1 with T t =0."""
```

```
    if T.shape[0]==0:
        return True
    m,n = T.shape
    # linprog: find t >=1 s.t. T t =0
    # variables t[0..5], minimize 0
    c = np.zeros(n)
    A_eq = T
    b_eq = np.zeros(m)
    bounds = [(1, None)]*n
    res = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')
    return res.success

feasible = []
max_size = 0
for mask in range(1<<12):
    idx = [i for i in range(12) if (mask>>i)&1]
    T = build_T(idx)
    if is_feasible(T):
        feasible.append((mask, idx, T))
        max_size = max(max_size, len(idx))

print(f"Total subsets 4096, feasible {len(feasible)}, max feasible size {max_size}")
# Count achieving max size 4 using all four pivots
max_sets = [f for f in feasible if len(f[1])==max_size]
print(f"Max size sets: {len(max_sets)}")
# filter using all four pivots
all_pivot = []
for mask, idx, T in max_sets:
    pivots = set(slots[i][0] for i in idx)
    if len(pivots)==4:
        all_pivot.append((mask, idx))
print(f"Max size using all 4 pivots: {len(all_pivot)}")
for mask, idx in all_pivot:
    print(" ", [(slots[i][0], slots[i][1]) for i in idx])
# Expected: 182 feasible, max size 4, exactly 3 achieve size 4 using all four pivots, each fixed-point-free
      involution
```

Appendix D. Source of three-mag-59049.py

```
"""
three-mag-59049.py -- 3^{10}=59049 three-magnitude colourings for n=5 unique-survivor
Tests Remark 5.7: minimum total survivors 8 and zero colourings with one survivor at every vertex.

Each of the 10 pairs gets one of three magnitudes alpha, beta, alpha+beta (colours 0, 1, 2),
with alpha, beta > 0 generic, i.e. linearly independent over Q. A magnitude is stored as its
coefficient vector over (alpha, beta), so the relation t_{jk} = t_{uj} + t_{uk} of Lemma 2.33
holds exactly when the coefficient vectors add up: exact integer arithmetic, no floating
point, and every magnitude is positive. For each apex u we mark the pairs (j,k) whose
relation holds; a vertex survives u if it lies in no marked pair.
"""
import itertools

pairs = [(0,1),(0,2),(0,3),(0,4),(1,2),(1,3),(1,4),(2,3),(2,4),(3,4)]
pair_index = {p:i for i,p in enumerate(pairs)}

# colour -> coefficients of the magnitude over (alpha, beta)
MAG = {0: (1, 0), 1: (0, 1), 2: (1, 1)}

def srt(a,b): return (a,b) if a<b else (b,a)

# For each apex u, the pairs it could zero: (j,k) with u not in (j,k)
```

AI

```
apex_to_pairs = {u: [(j,k) for (j,k) in pairs if u not in (j,k)] for u in range(5)}

def survivors_for_colouring(colours):
    """colours: 10 values in {0,1,2}. For each apex u, count the survivors of u."""
    mags = [MAG[c] for c in colours]
    total_survivors=0
    per_vertex_survivors={}
    all_one = True
    for u in range(5):
        covered=set()
        for (j,k) in apex_to_pairs[u]:
            jk = mags[pair_index[(j,k)]]
            uj = mags[pair_index[srt(u,j)]]
            uk = mags[pair_index[srt(u,k)]]
            # additive relation t_jk = t_uj + t_uk, exact for generic alpha, beta
            if jk == (uj[0] + uk[0], uj[1] + uk[1]):
                covered.add(j); covered.add(k)
        # survivors are vertices in V\{u} not covered
        survivors = [v for v in range(5) if v!=u and v not in covered]
        per_vertex_survivors[u]=len(survivors)
        total_survivors+=len(survivors)
        if len(survivors)!=1:
            all_one=False
    return total_survivors, per_vertex_survivors, all_one

min_total=None
count_one_survivor=0
enumerated=0
for colours in itertools.product(range(3), repeat=10):
    enumerated+=1
    tot, per, all_one = survivors_for_colouring(colours)
    if min_total is None or tot < min_total:
        min_total=tot
    if all_one:
        count_one_survivor+=1

print(f"Enumerated {enumerated} colourings")
print(f"Minimum total survivors: {min_total} (expected 8)")
print(f"Colourings with exactly one survivor at every vertex: {count_one_survivor} (expected 0)")
```

Appendix E. Source of `peo_insufficiency.py`

AI

```
"""
peo_insufficiency.py -- a 2-path of 5 triangles on which no fill-free ordering is clean
Tests Remark 4.6: an SPD realization with zero clean simplicial vertices, so that a pure
PEO strategy, which never creates fill, fails.

The pattern is the 2-path strip on 0..6 (edges {i,i+1} and {i,i+2}), whose triangles are
{i,i+1,i+2}, i = 0..4. Its simplicial vertices are the ends 0 and 6. The matrix below makes
each end zero the opposite edge of its triangle: A00*A12 = A01*A02 and A66*A45 = A64*A65
(Definition 2.19), so neither end is a clean pivot at the empty level. Arithmetic is exact
(fractions), so every zero test is exact. The script checks that the matrix is SPD with this
pattern, then runs all 7! orderings through the elimination game and the Schur updates of
Definition 2.6: an ordering is clean (Definition 2.24) if every position committed to L is
nonzero, and it is a perfect elimination ordering (PEO) if it creates no fill.
"""
import itertools
from fractions import Fraction as F

N = 7
h = F(1, 2)
```

AI

```
A = [[2, 1, 1, 0, 0, 0, 0],
     [1, 3, h, h, 0, 0, 0],
     [1, h, 3, h, h, 0, 0],
     [0, h, h, 3, h, h, 0],
     [0, 0, h, h, 3, h, 1],
     [0, 0, 0, h, h, 3, 1],
     [0, 0, 0, 0, 1, 1, 2]]
A = [[F(x) for x in row] for row in A]
STRIP = {frozenset((i, i + d)) for d in (1, 2) for i in range(N - d)}

def is_spd(M):
    """Symmetric, and every pivot of Gaussian elimination in natural order is positive."""
    if any(M[i][j] != M[j][i] for i in range(N) for j in range(N)):
        return False
    M = [row[:] for row in M]
    for p in range(N):
        if M[p][p] <= 0:
            return False
        for j in range(p + 1, N):
            f = M[j][p] / M[p][p]
            for k in range(p, N):
                M[j][k] -= f * M[p][k]
    return True
```

AI

```
def eliminate(order):
    """Run `order` through the elimination game and the Schur updates.
    Returns (clean, fill_free)."""
    M = {j: {k: A[j][k] for k in range(N)} for j in range(N)}
    adj = {v: {u for u in range(N) if u != v and A[u][v] != 0} for v in range(N)}
    clean, fill_free = True, True
    for p in order:
        if any(M[p][x] == 0 for x in adj[p]):   # a committed position of L is zero
            clean = False
        nbrs = adj[p]
        for a, b in itertools.combinations(nbrs, 2):
            if b not in adj[a]:                 # the pivot creates a fill edge
                fill_free = False
                adj[a].add(b); adj[b].add(a)
        for v in nbrs:
            adj[v].discard(p)
        del adj[p]
        rest = [v for v in M if v != p]
        M = {j: {k: M[j][k] - M[j][p] * M[k][p] / M[p][p] for k in rest} for j in rest}
    return clean, fill_free

pattern = {frozenset((i, j)) for i in range(N) for j in range(i + 1, N) if A[i][j] != 0}
assert is_spd(A), "matrix is not SPD"
assert pattern == STRIP, "pattern is not the 2-path of 5 triangles"
nbr = {v: {u for u in range(N) if frozenset((u, v)) in pattern} for v in range(N)}
simplicial = [v for v in range(N)
              if all(frozenset((a, b)) in pattern for a, b in itertools.combinations(nbr[v], 2))]
# C_v at the empty level: pairs {j,k} with A_vv A_jk = A_vj A_vk (Definition 2.19)
clean_simplicial = [v for v in simplicial
                    if not any(A[v][v] * A[j][k] == A[v][j] * A[v][k]
                               for j, k in itertools.combinations(range(N), 2)
                               if v not in (j, k) and frozenset((j, k)) in pattern)]

orders = clean_count = peo_count = clean_peo_count = 0
for order in itertools.permutations(range(N)):
    orders += 1
    clean, fill_free = eliminate(order)
    clean_count += clean
    peo_count += fill_free
    clean_peo_count += clean and fill_free

```

```
print(f"Matrix is SPD with the pattern of the 2-path of 5 triangles on {N} vertices")
print(f"Simplicial vertices: {simplicial} (expected [0, 6])")
print(f"Clean simplicial vertices at the empty level: {len(clean_simplicial)} (expected 0)")
print(f"Enumerated {orders} orderings: {peo_count} perfect elimination orderings, "
      f"{clean_peo_count} of them clean (expected 0)")
print(f"Clean orderings: {clean_count} (expected > 0; each creates fill)")
```

Appendix F. Source of `fixed_ordering_witnesses.py`

```
"""
fixed_ordering_witnesses.py -- patterns of Theorem 4.8 on which no fixed ordering is clean
Tests Remark 4.9: for the triangle K3, and for the one-triangle pattern E on five vertices,
every ordering fixed in advance is unclean for some SPD matrix with that pattern.

An ordering is run through the elimination game and the Schur updates of Definition 2.6. It is
clean (Definition 2.24) if every position committed to L is nonzero, that is every pair {p, x}
with x a neighbour of the pivot p in the current game graph. Arithmetic is exact (fractions),
so every zero test is exact.

K3: the witness for the vertex a has unit diagonal, 1/2 on the two entries at a and 1/4 on the
opposite entry, so pivoting a first cancels the opposite pair: 1/4 - (1/2)(1/2) = 0.

E = {03, 04, 14, 23, 34}: the five witnesses of Remark 4.9. The witness for an ordering is
chosen by its first triangle vertex u and, for u = 3 (resp. 4), by whether the pendant 2
(resp. 1) precedes the earlier end of the pair that u zeroes. The script checks that this
choice defeats all 120 orderings, and that choosing by whether the pendant precedes u does not.
"""
import itertools
from fractions import Fraction as F

def pivots(A):
    """Pivots of Gaussian elimination in natural order, stopping at the first nonpositive one."""
    n = len(A)
    M = [row[:] for row in A]
    out = []
    for p in range(n):
        out.append(M[p][p])
        if M[p][p] <= 0:
            break
        for j in range(p + 1, n):
            f = M[j][p] / M[p][p]
            for k in range(p, n):
                M[j][k] -= f * M[p][k]
    return out

def is_spd(A):
    """Symmetric, and every pivot of Gaussian elimination in natural order is positive."""
    n = len(A)
    symmetric = all(A[i][j] == A[j][i] for i in range(n) for j in range(n))
    piv = pivots(A)
    return symmetric and len(piv) == n and all(x > 0 for x in piv)

def leading_minors(A):
    out, d = [], F(1)
    for x in pivots(A):
        d *= x
        out.append(d)
    return out
```

AI

```python
def pattern(A):
    n = len(A)
    return {frozenset((i, j)) for i in range(n) for j in range(i + 1, n) if A[i][j] != 0}

def pivot(M, p):
    """The pivot update of Definition 2.6 at p, on the survivors of the current matrix M."""
    rest = [v for v in M if v != p]
    return {j: {k: M[j][k] - M[j][p] * M[k][p] / M[p][p] for k in rest} for j in rest}

def schur(A, S):
    """The entries of A/A_S, by successive pivots (Lemma 2.4: the order within S is immaterial)."""
    M = {j: {k: F(A[j][k]) for k in range(len(A))} for j in range(len(A))}
    for p in S:
        M = pivot(M, p)
    return M

def committed_zeros(A, order):
    """The committed positions {p, x} that are zero when `order` is used (none iff clean)."""
    n = len(A)
    M = {j: {k: F(A[j][k]) for k in range(n)} for j in range(n)}
    adj = {v: {u for u in range(n) if u != v and A[u][v] != 0} for v in range(n)}
    zeros = []
    for p in order:
        zeros += [(p, x) for x in sorted(adj[p]) if M[p][x] == 0]
        for a, b in itertools.combinations(adj[p], 2):    # complete the neighbourhood of p
            adj[a].add(b)
            adj[b].add(a)
        for v in adj[p]:
            adj[v].discard(p)
        del adj[p]
        M = pivot(M, p)
    return zeros

# --- The triangle K3 ---
h, q = F(1, 2), F(1, 4)
K3 = {frozenset(e) for e in itertools.combinations(range(3), 2)}
T = {a: [[F(1) if i == j else (h if a in (i, j) else q) for j in range(3)] for i in range(3)]
     for a in range(3)}
for a, A in T.items():
    b, c = [v for v in range(3) if v != a]
    assert is_spd(A) and pattern(A) == K3
    abc = [[A[i][j] for j in (a, b, c)] for i in (a, b, c)]
    assert leading_minors(abc) == [1, F(3, 4), F(9, 16)]
    assert schur(A, [a])[b][c] == 0
orders3 = list(itertools.permutations(range(3)))
defeated = sum(bool(committed_zeros(T[order[0]], order)) for order in orders3)
clean = [sum(not committed_zeros(A, order) for order in orders3) for A in T.values()]
print("K3: three SPD witnesses with pattern K3, leading minors 1, 3/4, 9/16 in the order a, b, c;")
print("    pivoting a first cancels the opposite pair")
print(f"K3: orderings unclean for the witness of their first vertex: {defeated} of 6 (expected 6)")
print(f"K3: clean orderings of each witness: {clean} (expected [4, 4, 4])")

# --- The one-triangle pattern E of Remark 4.9 ---
E = {frozenset(map(int, e)) for e in ["03", "04", "14", "23", "34"]}
t = F(1, 3)
A0 = [[3, 0, 0, 1, 1], [0, 3, 0, 0, 1], [0, 0, 3, 1, 0], [1, 0, 1, 3, t], [1, 1, 0, t, 3]]
A3 = [[3, 0, 0, 1, h], [0, 3, 0, 0, 1], [0, 0, 3, 1, 0], [1, 0, 1, 2, 1], [h, 1, 0, 1, 3]]
A4 = [[3, 0, 0, h, 1], [0, 3, 0, 0, 1], [0, 0, 3, 1, 0], [h, 0, 1, 3, 1], [1, 1, 0, 1, 2]]
A3p = [row[:] for row in A3]
A3p[3][3] = F(7, 3)
A4p = [row[:] for row in A4]
A4p[4][4] = F(7, 3)
W = {name: [[F(x) for x in row] for row in A]
     for name, A in {"A(0)": A0, "A(3)": A3, "A(3)'": A3p, "A(4)": A4, "A(4)'": A4p}.items()}
```

```python
MINORS = {"A(0)": [3, 9, 27, 63, 147], "A(3)": [3, 9, 27, 36, F(297, 4)],
          "A(3)'": [3, 9, 27, 45, F(195, 2)], "A(4)": [3, 9, 27, F(279, 4), F(297, 4)],
          "A(4)'": [3, 9, 27, F(279, 4), F(195, 2)]}
for name, A in W.items():
    assert is_spd(A) and pattern(A) == E and leading_minors(A) == MINORS[name], name
assert schur(W["A(0)"], [0])[3][4] == 0 and schur(W["A(3)"], [3])[0][4] == 0
assert schur(W["A(4)"], [4])[0][3] == 0
assert schur(W["A(3)'"], [2, 3])[0][4] == 0 and schur(W["A(4)'"], [1, 4])[0][3] == 0
print("E: five SPD witnesses with pattern E and the leading minors of Remark 4.9")

def witness(order, by_pair):
    """Remark 4.9's choice of witness. by_pair: compare the pendant with the earlier end of
    the pair that u zeroes (the corrected rule); otherwise compare it with u itself."""
    pos = {v: i for i, v in enumerate(order)}
    u = min((0, 3, 4), key=pos.get)
    if u == 0:
        return "A(0)"
    pendant, pair = (2, (0, 4)) if u == 3 else (1, (0, 3))
    first = min(pos[v] for v in pair) if by_pair else pos[u]
    return f"A({u})'" if pos[pendant] < first else f"A({u})"

orders = list(itertools.permutations(range(5)))
fixed = [o for o in orders if not committed_zeros(W[witness(o, True)], o)]
naive = [o for o in orders if not committed_zeros(W[witness(o, False)], o)]
print(f"E: orderings escaping the corrected choice of witness: {len(fixed)} of 120 (expected 0)")
print(f"E: orderings escaping the choice by 'pendant precedes u': {len(naive)} (expected 20)")
print("E: sigma = (3,2,0,1,4) with the displayed A(3): (A/A_3)_04 =", schur(W["A(3)"], [3])[0][4],
      "is refilled to (A/A_{2,3})_04 =", schur(W["A(3)"], [3, 2])[0][4],
      "| committed zeros:", committed_zeros(W["A(3)"], (3, 2, 0, 1, 4)))
```

Appendix G. Source of `slot_counts.py`

```python
"""
slot_counts.py -- feasible slot families at four and five survivors, and the slot search on K4
Tests Lemma 5.5 and Appendix H (the counts at a four-vertex level and on K4) and Remark 5.7
(no scheme leaving one survivor at every vertex at order five).

A slot (u, {j, k}) at a level with survivor set W says that pivoting the apex u cancels the pair
{j, k}; with nonzero legs it imposes t_jk = t_uj + t_uk (Lemma 2.33) on the log-magnitudes of the
pairs inside W. A family is feasible if these relations have a solution with every t > 0
(Definition 2.29). This is decided exactly: t > 0 may be replaced by t >= 1, the system being
homogeneous; equations are substituted away, then Fourier-Motzkin runs in rational arithmetic.
"""
import itertools
from fractions import Fraction as F

def level(nv):
    """Slots and relation rows at a level with survivors 0..nv-1."""
    pairs = list(itertools.combinations(range(nv), 2))
    index = {p: i for i, p in enumerate(pairs)}
    slots = [(u, p) for u in range(nv) for p in pairs if u not in p]
    def row(slot):
        u, (j, k) = slot
        r = [0] * len(pairs)
        r[index[(j, k)]] += 1
        r[index[tuple(sorted((u, j)))]] -= 1
        r[index[tuple(sorted((u, k)))]] -= 1
        return r
    return pairs, slots, row
```

AI

```python
def feasible(rows, nvar):
    """Whether rows . t = 0 has a solution with every t_v >= 1."""
    ineq = [([F(int(i == v)) for i in range(nvar)], F(1)) for v in range(nvar)]   # a . t >= b
    eqs = [[F(x) for x in r] for r in rows]
    while eqs:
        e = eqs.pop()
        piv = next((i for i, x in enumerate(e) if x != 0), None)
        if piv is None:
            continue
        sub = lambda a: [ai - a[piv] / e[piv] * ei for ai, ei in zip(a, e)]
        eqs = [sub(r) for r in eqs]
        ineq = [(sub(a), b) for a, b in ineq]
    for v in range(nvar):
        pos = [(a, b) for a, b in ineq if a[v] > 0]
        neg = [(a, b) for a, b in ineq if a[v] < 0]
        new = [(a, b) for a, b in ineq if a[v] == 0]
        for (ap, bp), (an, bn) in itertools.product(pos, neg):
            cp, cn = ap[v], -an[v]
            new.append(([cn * x + cp * y for x, y in zip(ap, an)], cn * bp + cp * bn))
        ineq, seen = [], set()
        for a, b in new:
            if not any(a):
                if b > 0:
                    return False                    # 0 >= b > 0
                continue
            if (tuple(a), b) not in seen:
                seen.add((tuple(a), b))
                ineq.append((a, b))
    return True

# --- Four survivors (Lemma 5.5) ---
pairs, slots, row = level(4)
fam = [[slots[i] for i in range(12) if m >> i & 1] for m in range(1 << 12)]
feas = [f for f in fam if feasible([row(s) for s in f], 6)]
top = max(len(f) for f in feas)
maxi = [f for f in feas if len(f) == top]
allapex = [f for f in maxi if len({u for u, _ in f}) == 4]
def partner(f):
    """For a family with one slot per apex: u -> the vertex lying neither at u nor in its pair."""
    return {u: next(x for x in range(4) if x != u and x not in p) for u, p in f}
involution = all(pi[pi[u]] == u for pi in map(partner, allapex) for u in pi)
print(f"Four survivors: {len(feas)} of 4096 slot sets feasible (expected 182), maximum size {top} "
      f"(expected 4), {len(maxi)} of that size (expected 15), {len(allapex)} using all four apexes "
      f"(expected 3), each a fixed-point-free involution: {involution}")

# --- The order-four slot search on K4 (Appendix H) ---
V = range(4)
slots4 = [({a}, jk) for a in V for jk in itertools.combinations([x for x in V if x != a], 2)]
slots4 += [(set(S), tuple(x for x in V if x not in S)) for S in itertools.combinations(V, 2)]
def spoils(slot, order):
    """On K4 every pair is predicted nonzero: the slot spoils the ordering iff S is the set of
    predecessors of the earlier of j, k (Definition 2.29)."""
    S, (j, k) = slot
    return set(order[:min(order.index(j), order.index(k))]) == S
masks = [sum(1 << i for i, s in enumerate(slots4) if spoils(s, o))
         for o in itertools.permutations(V)]                  # the slots spoiling each ordering
blocking = {m for m in range(1 << len(slots4)) if all(m & sm for sm in masks)}
minimal = [m for m in blocking
           if not any(m & ~(1 << i) in blocking for i in range(len(slots4)) if m >> i & 1)]
print(f"K4 slot search over {len(slots4)} slots: {len(blocking)} blocking families "
      f"(expected 69075), {len(minimal)} minimal (expected 633)")

# --- Five survivors (Remark 5.7) ---
pairs, slots, row = level(5)
triple = lambda s: frozenset((s[0],) + s[1])
same_triple = [(s, t) for s, t in itertools.combinations(slots, 2) if triple(s) == triple(t)]
```

AI

```python
assert len(same_triple) == 30 and not any(feasible([row(s), row(t)], 10) for s, t in same_triple)
print("Five survivors: two slots on one triple are never feasible (30 cases), so at most 10 slots")
line = [(u, (j, k)) for (j, k) in pairs for u in range(5) if j < u < k]      # points 0..4 on a line
assert len(line) == 10 and feasible([row(s) for s in line], 10)
A = [[F(1, 2 ** abs(j - k)) for k in range(5)] for j in range(5)]            # A_jk = 2^-|j-k|
M = [r[:] for r in A]
for p in range(5):
    assert M[p][p] > 0                                                      # SPD: positive pivots
    for j in range(p + 1, 5):
        M[j] = [x - M[j][p] / M[p][p] * y for x, y in zip(M[j], M[p])]
assert all(x != 0 for r in A for x in r)                                    # complete pattern
assert all(A[j][k] - A[j][u] * A[u][k] / A[u][u] == 0 for u, (j, k) in line)
print("Five survivors: the 10 slots (u, {j,k}) with u between j and k on a line are feasible, and")
print("    realized by the SPD matrix 2^-|j-k| (complete pattern, all ten cancellations exact)")
# One survivor at the apex u: two of its pairs, covering exactly three of the other four vertices.
options = {}
for u in range(5):
    own = [jk for jk in pairs if u not in jk]
    options[u] = [((u, p), (u, r)) for p, r in itertools.combinations(own, 2)
                  if len(set(p) | set(r)) == 3]
assert all(len(o) == 12 for o in options.values())
schemes = [sum(c, ()) for c in itertools.product(*options.values())
           if len({triple(s) for s in sum(c, ())}) == 10]       # one slot on each of the 10 triples
nfeas = sum(feasible([row(s) for s in c], 10) for c in schemes)
print(f"Five survivors: {len(schemes)} configurations leave one survivor at every apex with one "
      f"slot per triple (expected 288); feasible: {nfeas} (expected 0)")
```

Appendix H. Independent recomputation of the numerical claims

Human

The numerical and combinatorial assertions listed below have been recomputed independently, in exact arithmetic, by an implementation written from the statements alone (no shared code with the primary verification of Appendix A). Rational quantities were handled with exact rationals; the irrational parameter was handled by exact arithmetic in the quadratic field $\mathbb{Q}(\sqrt{73})$, representing every quantity as $p + q\sqrt{73}$ with $p, q \in \mathbb{Q}$, so that all zero tests are exact and no floating-point tolerance is used.

The recomputation covers the spectrum and positive definiteness of A^* and of M_{2m} for $m = 2, 3, 4$, the two parameter identities and the vanishing of $A^2 + (1 - B^2)A - 1$ at $A = (-3 + \sqrt{73})/8$, the clean-ordering counts $0/24$ and $8/24$ for A^* and $0/720$ and $0/40320$ for M_6 and M_8, the pairing classification of Lemma 5.5, and the square-count identity $n\lceil (n-2)/2 \rceil = \binom{n}{2}$ for odd n in the range $3 \le n \le 11$. Every asserted value was reproduced exactly. One quantity emerged from the recomputation that the text uses only in the discussion of Question 7.1: there are 15 feasible slot sets of the maximum size 4 at a four-vertex level, of which only 3 use all four pivots, which is why Lemma 5.5 has to impose the all-four-pivots condition explicitly. The recomputation also separates the two necessary conditions of that lemma from feasibility itself: 218 of the 4096 subsets have an acyclic arc digraph, so the General Cycle Lemma 3.16 leaves 36 infeasible subsets undetected, and the feasible count 182 is a linear-programming count rather than a cycle count. None of these counts is used in the proof of Lemma 5.5, which is elementary; they are recorded here as corroboration and as context for the size bound that the lemma does prove. This recomputation is not included in the ancillary material; `slot_counts.py` (Appendix G) reproduces the counts 182, 15 and 3 in exact arithmetic.

It is worth recording why random search does not find the counterexamples of Section 5. Adversarial sampling on dense K_6, in which the entries of a random SPD matrix are perturbed towards

Human

the cancellation conditions, bottoms out near 296/720 clean orderings in our runs and never approaches the 0/720 of M_6, because M_6 lies on the measure-zero curve $A^2 + (1 - B^2)A - 1 = 0$. We report this as an indication of the difficulty rather than as evidence: the perturbation scheme and the number of draws are implementation choices, and no claim in this paper depends on them. These runs are not included in the ancillary material. This is the coexistence noted in Section 1.4: unfaithful distributions form a null set while strong-faithfulness fails on a set of positive measure [URBY13], so the genericity heuristic and the existential failure are consistent with one another.

An exhaustive slot search at order four was also carried out, over the 18 slots with $1 \le |S| \le 2$ on $G_A = K_4$: of the $2^{18} = 262144$ slot families, 69075 are blocking and 633 of these are minimal, and all 633 are refuted by exact rational certificates (605 by Stiemke's lemma, 28 by zero-leg propagation). The script `slot_counts.py` reproduces the counts 69075 and 633; the certificates are not included in the ancillary material. This corroborates Theorem 5.2 but is no longer part of its proof, which is elementary.

References

[BGM05] A. Becker, D. Geiger, and C. Meek. Perfect tree-like Markovian distributions. *Probability and Mathematical Statistics*, 25(2):231–239, 2005. An earlier version appeared in *Proceedings of the Sixteenth Conference on Uncertainty in Artificial Intelligence* (UAI 2000), 19–23.

[Bod98] H. L. Bodlaender. A partial k-arboretum of graphs with bounded treewidth. *Theoretical Computer Science*, 209:1–45, 1998.

[BP93] J. R. S. Blair and B. Peyton. An introduction to chordal graphs and clique trees. *The IMA Volumes in Mathematics and its Applications*, 56:1–29, 1993.

[CH69] D. E. Crabtree and E. V. Haynsworth. An identity for the Schur complement of a matrix. *Proceedings of the American Mathematical Society*, 22:364–366, 1969.

[Dav06] T. A. Davis. *Direct Methods for Sparse Linear Systems*. SIAM, 2006.

[Dem24] J. Demmel. MA221 lecture notes. Lecture notes, University of California, Berkeley, 2024. Fall 2024; accessed 2026.

[Dir61] G. A. Dirac. On rigid circuit graphs. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 25:71–76, 1961.

[FG65] D. R. Fulkerson and O. A. Gross. Incidence matrices and interval graphs. *Pacific Journal of Mathematics*, 15:835–855, 1965.

[GJSW84] R. Grone, C. R. Johnson, E. M. Sa, and H. Wolkowicz. Positive definite completions of partial Hermitian matrices. *Linear Algebra and its Applications*, 58:109–124, 1984.

[Gol80] M. C. Golumbic. *Algorithmic Graph Theory and Perfect Graphs*. Academic Press, 1980.

[Gor73] P. Gordan. Ueber die auflösung linearer gleichungen mit reellen coefficienten. *Mathematische Annalen*, 6:23–28, 1873.

[Hay68] E. V. Haynsworth. Determination of the inertia of a partitioned Hermitian matrix. *Linear Algebra and its Applications*, 1:73–81, 1968.

[HH18] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. The dual simplex implementation underlying the HiGHS solver.

[HNP91] M. T. Heath, E. Ng, and B. W. Peyton. Parallel algorithms for sparse linear systems. *SIAM Review*, 33(3):420–460, 1991.

[JW05] B. Jones and M. West. Covariance decomposition in undirected Gaussian graphical models. *Biometrika*, 92:779–786, 2005.

[Lau96] S. L. Lauritzen. *Graphical Models*. Oxford University Press, 1996.

[LM07] R. Lnenicka and F. Matus. On Gaussian conditional independence structures. *Kybernetika*, 43(3):327–354, 2007.

[Mee95] C. Meek. Strong completeness and faithfulness in Bayesian networks. In *Proceedings of the Eleventh Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 411–418, 1995.

[MR11] D. Malouche and B. Rajaratnam. Gaussian covariance faithful Markov trees. *Journal of Probability and Statistics*, 2011:152942, 2011. Preprint arXiv:0912.2407, 2009.

[RTL76] D. J. Rose, R. E. Tarjan, and G. S. Lueker. Algorithmic aspects of vertex elimination on graphs. *SIAM Journal on Computing*, 5(2):266–283, 1976.

[Sch25] Johannes Schmitt. Extremal descendant integrals on moduli spaces of curves: An inequality discovered and proved in collaboration with ai. *arXiv preprint arXiv:2512.14575*, 2025.

[STD10] S. Sullivant, K. Talaska, and J. Draisma. Trek separation for Gaussian graphical models. *The Annals of Statistics*, 38(3):1665–1685, 2010.

[Sti15] E. Stiemke. über positive lösungen eines systems linearer gleichungen. *Mathematische Annalen*, 76:340–342, 1915.

[URBY13] C. Uhler, G. Raskutti, P. Buhlmann, and B. Yu. Geometry of the faithfulness assumption in causal inference. *The Annals of Statistics*, 41(2):436–463, 2013.

[Yan81] M. Yannakakis. Computing the minimum fill-in is NP-complete. *SIAM Journal on Algebraic and Discrete Methods*, 2:77–79, 1981.

[ZS08] J. Zhang and P. Spirtes. Strong faithfulness and uniform consistency in causal inference. *Journal of Machine Learning Research*, 9:2013–2030, 2008.